TECHNISCHE UNIVERSITÄT
ILMENAU

Fakultät für Informatik und Automatisierung
Data-intensive Systems and Visualization Group

Master Thesis

Robustness and Adaptability in Multi-Agent Reinforcement Learning: A Unified Analysis and a Proposed Method

Registration Date:	[FILL IN]
Submission Date:	[FILL IN]
Academic Chair:	Prof. Dr.-Ing. Patrick Mäder
Academic Supervisor:	[FILL IN]
Submitted by:	Abdullah Alalwani born [FILL IN] in [FILL IN] abdullah.alwani@hotmail.com
Course of Studies:	[FILL IN]

Acknowledgments

[FILL IN: acknowledgements text]

Abstract

This thesis proposes a multi-agent reinforcement learning (MARL) method that adapts to a change in the task while staying robust to a bounded perturbation, both at the same time and without giving up performance in nominal conditions. A MARL policy that performs well in its training simulator is not automatically useful once it leaves that simulator. Two things go wrong that never appeared during training. First, the world is not the simulator: sensors are noisy, a teammate can fail or be attacked, and the assumed dynamics are only an approximation of the real ones. Second, the world does not hold still: traffic patterns shift over the day, teammates keep updating their own policies, and equipment degrades after deployment. The first problem is treated in the literature by *robust* MARL, which trains a policy to withstand a bounded, worst-case disturbance fixed in advance; the second by *adaptive* MARL, which tracks a disturbance that keeps changing after deployment without knowing the change in advance. The two are developed almost in isolation, on disjoint benchmarks and with metrics that do not translate, and no method handles a bounded perturbation and an ongoing task change together.

The proposed method is a unified framework built on the FMASAC cooperative backbone. A per-agent contrastive context encoder, in the style of contrastive context-based meta-RL and equipped with an exchangeable-process Gaussian posterior, supplies each agent with a local task representation and a closed-form information-gain reward; separate information-gain exploration and task-reward execution policies are trained on that context. An M3DDPG-style worst-case (MAAL) perturbation of the teammate actions, with a single team-shared variational encoder that infers the adversary's budget and scoped to the execution stream, provides robustness while leaving the exploration stream and the nominal case untouched. A representation-learned (InDiD-style) change-point detector, trained in a second phase, resets an agent's context posterior and re-triggers its exploration when that agent's regime switches. Both inferred latents condition the execution policy and the centralized critic as full posteriors, VariBAD-style, so conservatism is scaled to the disturbance that is actually inferred. The contribution is this combination and the coupling of its parts, not any single mechanism.

To ground the design, the thesis reviews the robust-MARL and adaptive-MARL literatures and organizes them into one framework around a shared cause, non-stationarity, whether imposed by an adversary or arising naturally as agents and

environments evolve. Each family is described by its mechanism, from adversarial training and adversarial regularization through distributionally robust value factorization and sim-to-real transfer on the robust side, and from change-point detection and sliding-window estimation through meta-reinforcement learning task inference on the adaptive side, and the two sides are compared on a common set of axes: objective, adaptation timing, disturbance assumption, mechanism, and failure mode. The comparison makes explicit the nominal-performance cost that worst-case optimization pays and yields four concrete gaps that the proposed method targets: robust and adaptive methods are tested on disjoint disturbance types, they report metrics that do not translate, sim-to-real robustness is never tested against continued post-deployment drift, and there is almost no multi-agent credit assignment for the source of an observed change. A Search-and-Rescue evaluation, with scenario dimensions and a provisional metric set, is designed, and a first implementation of the evaluation environment — a vectorized multi-robot task with a two-channel partial-observability model and the distributionally robust baseline — is reported and smoke-tested. Every quantitative figure attributed to prior work is the one reported by its original authors. The proposed method is specified in full but not implemented; layering the non-stationarity, implementing the method, and running the evaluation is the next step.

Contents

1. Introduction

1.1. A Deployment Scenario

Consider a team of delivery robots working together in a warehouse. During training, the simulator gives them clean sensor readings, a fixed layout, and teammates that never fail. None of this holds after deployment. A camera becomes dusty and reports noisy positions. A shelf is moved and the stored map is wrong. One robot loses power during a task and the others must take over its work. The team learned a single policy for a single environment, and the real environment does not match it.

This situation is common. It occurs in autonomous-vehicle fleets, drone swarms, and wireless sensor networks, that is, in any system where several agents learn a joint policy and then run it in conditions that differ from training [1], [2]. Because the agents depend on one another, a disturbance to one agent does not stay local. Each agent's best action depends on what its neighbours do, so a small local error can spread and cause a system-level failure.

1.2. Why a Trained Policy Is Not Enough

A multi-agent reinforcement learning (MARL) policy is trained to maximize the expected return in one fixed environment. Convergence guarantees, where they exist, are stated for that environment. After deployment, two different kinds of change can break the guarantee.

The environment is not the simulator. Sensors are noisy. A teammate's message is dropped or corrupted. An adversary may try to mislead the system. The environment itself has not changed, but the observations and actions of each agent have moved away from their training values by a *bounded* amount. A policy that is optimal only for the nominal case has no reason to perform well at the worst point of this bounded set. Work on adversarial examples in supervised learning and in single-agent RL shows that this worst point can be far from the nominal one, even for a small perturbation [3]–[5].

The environment does not stay fixed. Traffic patterns change over the day. A teammate updates its own policy and begins to act differently, which makes the environment seen by each agent non-stationary [6]. Equipment wears out. Here the environment itself differs from the training one, and it keeps changing after deployment, with no fixed bound on how far it can drift. The only assumption that can be made is that the change has some structure, such as slow drift, occasional jumps, or the return of a previously seen regime.

The first kind of change is a robustness problem and the second is an adaptability problem. The literature treats them separately, with different benchmarks, different metrics, and little cross-citation. This separation is a problem in practice. A policy that resists a fixed perturbation budget but cannot follow a drifting environment will eventually be pushed outside its budget and fail. A policy that follows benign drift but has no defence against an adversarial teammate will be exploited as soon as one appears. A deployable system needs both properties, and it needs them evaluated together. Chapter 3 makes both problems precise; the rest of this chapter states how the thesis approaches them.

1.3. Robustness and Adaptability as One Problem

This thesis treats robustness and adaptability as two responses to one underlying problem: *non-stationarity*, the fact that the environment of a MARL policy, including the other agents, does not stay fixed. The two responses differ in what they assume about the change and in when they act on it.

Robustness. The change from training conditions is *bounded* and unknown in advance, and it may be adversarial. The response is to prepare for the worst case *before* deployment, by optimizing a worst-case objective over the set of allowed changes. Robustness asks: how can a policy stay effective when the deployment condition deviates from training?

Adaptability. The change can grow *without bound* over time, but it has exploitable structure, such as slow drift, abrupt regime changes, recurrence, or membership in a family of related tasks. The response is to track this structure *after* deployment, by detecting the change and re-estimating, or by running a learned adaptation procedure. Adaptability asks: how can a policy change its behaviour when the underlying condition itself evolves?

Hypothesis. Robustness and adaptability are complementary mechanisms for handling non-stationarity under different assumptions about the disturbance. One assumption is bounded and prepared for; the other is evolving and tracked. Because both address the same cause, methods from the two families can be described in a

common vocabulary and compared on a common set of axes. This comparison shows where the two families already coincide and where current evaluation practice keeps them apart.

This view is more than a label. A distributionally robust method keeps an *ambiguity set*, a set of environment models the policy is protected against. An adaptive method keeps a *confidence set*, a set of models consistent with the data seen so far. The two sets are built in almost the same way; the difference is whether the set is fixed once or updated as data arrive [7], [8]. Similarly, a meta-learned task encoder is trained to infer which task from a distribution it currently faces; changing that distribution to include adversarially perturbed tasks turns the same encoder toward the robust objective. Chapter 8 develops these connections, and Chapter 10 builds the proposed method on them. The chapters between set out the literature the method draws on and the gaps it targets.

1.4. Research Questions

Four questions structure the thesis. The first two are settled from the literature, through a structured review that organizes existing work into one framework. The last two are raised by that review and answered at the level of a design: the thesis specifies a method for them but does not run the experiment that would confirm it. Each question is stated together with the form its answer must take, so that Chapter 11 can be checked against it.

RQ1 — What is the problem?

What sources of uncertainty and non-stationarity threaten a MARL policy once it leaves a fixed training simulator, and how does the literature classify them?

A useful answer is a classification with a principle behind it, not a list. It should state *where* each disturbance enters the system, *whether* it is modelled as bounded or as unbounded over time, and *why* the multi-agent setting makes non-stationarity unavoidable even without an external disturbance. The classification must also be the one that the rest of the thesis uses to sort methods. This question is answered in Chapter 3 and Section 4.7, with Table 4.2.

RQ2 — How is bounded uncertainty handled, and at what cost?

What mechanisms make a MARL policy robust to a bounded, unknown disturbance, and what do they trade off against one another?

The answer is organized by *mechanism*, the general idea that a group of methods shares, rather than by paper. For each mechanism, the answer states the assumption it makes about the disturbance, its cost, the guarantee it provides, and the case in which it fails. The test of the answer is whether a reader can use it to choose a robust method for a stated deployment condition and predict how it will break. This question is answered by the review of Chapter 5.

RQ3 — Can one policy be good in both disturbed and nominal conditions?

Does training a MARL policy against a disturbed or adversarial environment reduce its performance when no disturbance is present, and can a policy perform well in both the disturbed and the nominal case?

This is the practical objection to robustness. Adversarial training and worst-case optimization gain a floor on bad-case return by giving up expected-case return, and a deployed system spends most of its time in the nominal case. The first part of the question, how large the nominal-performance cost is, is answered from the literature in Chapter 5, where the trade-off of each robust family is recorded, and again in Chapter 8. The second part, whether the cost can be avoided, is a design question. Chapter 10 proposes a method that trains the execution critic against an M3DDPG worst-case (one-step MAAL) teammate-action perturbation and conditions both the execution policy and the centralized critic on the full posterior of an inferred perturbation radius (VariBAD-style), so behaviour returns to nominal when no disturbance is present; its evaluation is future work.

RQ4 — Can the policy adapt quickly when the whole environment changes?

What mechanisms let a MARL policy adapt, using as little deployment data as possible, when the environment or the other agents change after deployment; how do these relate to robustness; and can adaptation speed and robustness be obtained together?

The review part is organized by mechanism, with assumptions, cost, and failure mode. It spans Chapter 6 (online re-estimation) and Chapter 7 (amortized meta-learning). Chapter 8 then shows how these mechanisms relate to the robust ones. The design part, whether a single method can be both fast-adapting and robust, is answered by Chapter 10: a cooperative policy in which each agent infers its own local task context through an exchangeable-process posterior and detects its own regime switch with a representation-learned change-point detector, and which is robustified M3DDPG-style by a worst-case teammate-action perturbation scoped to the execution stream, so it adapts and stays robust at once.

Chapter 2 states the research plan behind these questions: which steps the thesis completes, and which it leaves as future work.

1.5. Contributions

This thesis makes the following contributions. The first is its original contribution; the others are the literature review and analysis that ground it and that answer RQ1 and RQ2 in their own right.

1. **A proposed method (the central contribution).** Chapter 10 presents a unified framework for a policy that is nominal-performant, robust, and fast-adapting at once: on the FMASAC cooperative backbone, a per-agent contrastive context encoder with an exchangeable-process Gaussian posterior gives each agent a local task representation and a closed-form information-gain reward; separate information-gain exploration and task-reward execution policies are trained on that context; an M3DDPG-style worst-case (one-step MAAL) teammate-action perturbation, with a single team-shared variational encoder that infers the adversary's budget and scoped to the execution stream, provides robustness; and a second training phase fits a representation-learned change-point detector that resets an agent's context posterior and re-triggers its exploration when that agent's regime switches. The contribution is the combination and its coupling. The method is specified in full but not implemented.
2. **A concrete gap analysis.** Chapter 9 identifies four specific gaps that block obtaining nominal performance, robustness, and fast adaptation together, and that block evaluating methods which claim to do so. These are the gaps the proposed method targets.
3. **One framework for two literatures.** Robust MARL and adaptive MARL are placed under a single framing: non-stationarity handled before deployment

versus after. The chapters that develop them (Chapters 5, 6, and 7) are written to be read against each other.

4. **A mechanism-level comparison.** Every family is described with a fixed template, and the two literatures are then compared on one set of axes. A specific method from one family can therefore be placed against a specific method from the other, including on the nominal-performance cost each one pays (RQ3).

5. **A curated set of exemplar methods.** Each branch of the framework is represented by a few methods, chosen for what they assume, what they cost, and how they fail. This is a shortlist that could be used to select baselines for the evaluation.

The literature-review chapters (Chapters 5–9) contain no experiments of their own: every quantitative result attributed to a method is the one reported in that method's own source paper, in the environment that paper used. The proposed method of Chapter 10 is the thesis's original contribution; it is specified in full but not implemented, and its evaluation is future work.

1.6. Approach at a Glance

The problem, in one sentence. A cooperative MARL team trained in a fixed simulator must keep performing when a *bounded*, possibly adversarial perturbation is applied at deployment *and* the task keeps drifting without bound, and neither defence may cost performance in the undisturbed case. No method surveyed here delivers all three of nominal performance, bounded-perturbation robustness, and fast adaptation at once (Chapter 9).

The idea, in one sentence. Do not hedge against the worst case unconditionally; instead infer online *how far* each agent's environment has drifted and *how large* the perturbation budget is, and condition behaviour on those inferences, so the policy defends only as much as the inferred disturbance warrants and returns to nominal behaviour when it infers none.

How the thesis gets there.

1. **Formalize** the two kinds of departure — bounded-and-static versus unbounded-but-structured — in one decentralized partially observable model (Chapter 3).

2. **Fix a cooperative backbone** by comparing the CTDE baselines and selecting FMASAC, the one that gives per-agent credit assignment, sample-efficient off-policy updates, and tunable exploration (Chapter 4).
3. **Review robustness** as four mechanisms and record the nominal-performance cost each pays (Chapter 5); this produces requirements R1–R2.
4. **Review adaptation** as online re-estimation and as meta-learning (Chapters 6–7); this produces requirements R3–R5.
5. **Synthesize and locate the gap** (Chapters 8–9), which turns the review into seven design requirements R1–R7 and four evaluation gaps (Section 9.2).
6. **Design the method** that meets R1–R7 (Chapter 10): on the FMASAC backbone, a per-agent contrastive context encoder with an exchangeable-process posterior and a closed-form information-gain reward; separate exploration and execution policies; an M3DDPG-style (MAAL) worst-case teammate-action perturbation with a team-shared budget encoder, scoped to the execution stream; and a second-phase, representation-learned change-point detector that resets an agent's context posterior and re-triggers its exploration on a regime switch.
7. **Specify the evaluation** — a Search-and-Rescue task with independent disturbance and drift dials and a joint metric set, plus a first implementation of the environment — and leave running it as the next step (Chapter 10).

Each literature-review chapter closes by naming the requirement it produces, and Chapter 10 opens by restating R1–R7 and showing where each is met. The survey establishes what is known; the method is what the thesis adds; the evaluation is what remains.

1.7. Roadmap

Chapter 2 states the research plan and which steps are completed. Chapter 3 formalizes the deployment problem: the decentralized partially observable model, the bounded setting with its robust objective and robust Bellman operator, and the unbounded task-distribution setting that motivates meta-learning. Chapter 4 introduces the formal vocabulary the rest of the thesis uses: Markov games, partial observability, the MARL objective, the non-stationarity that other learning agents create, and the CTDE baselines that later methods extend; it compares those baselines and selects FMASAC as the base learner for the proposed method, and it completes the classification of uncertainty sources (RQ1). Chapters 5–9 are the literature review that grounds the method. Chapter 5 organizes robust MARL into four

solution families and records the nominal-performance cost of each (RQ2, RQ3). Chapter 6 organizes adaptive MARL into change-point detection, sliding-window estimation, and structured adaptation, starting from why robustness alone is not enough (RQ4). Chapter 7 covers meta-reinforcement learning as a third adaptation mechanism (RQ4). Chapter 8 places the two families side by side and shows where they already meet. Chapter 9 states the gaps that the proposed method targets. Chapter 10 presents that method, a two-phase exchangeable-context CCM–FMASAC framework, and outlines its evaluation. Chapter 11 answers the four research questions directly.

Part I.

Background and Problem Formulation

2. Methodology

This thesis follows a five-step research plan:

1. review the literature on robustness and adaptability in MARL;
2. organize it into a taxonomy and identify what it misses;
3. design a method that addresses the gap;
4. evaluate that method against representative baselines in a simulated multi-agent task;
5. define a common metric and compare the results.

The first three steps are carried out in this thesis: Chapter 10 presents the proposed method in full. Steps 4 and 5, the evaluation, are future work. Table 2.1 lists each step, its status, and where it appears. The rest of this chapter gives the detail behind the completed steps: the literature search, the inclusion and exclusion criteria, how the taxonomy and the comparison axes were built, and the limitations of a single-author review.

2.1. Review Methodology

Literature reviews can generally follow either a systematic or a narrative approach. A systematic review uses a predefined search and screening protocol to identify and evaluate the literature in a reproducible manner. In contrast, a narrative review selects and organizes relevant studies in order to develop a conceptual understanding of a research problem.

This thesis follows a narrative review approach. Its objective is not to quantify the literature on robust and adaptive multi-agent reinforcement learning (MARL), nor to claim exhaustive coverage of all published work. Instead, the review is structured to provide a conceptual organization of the main approaches to robustness and adaptation in MARL. This organization allows different methods to be compared according to the problems they address, their underlying assumptions, and their mechanisms for achieving robustness or adaptation.

Table 2.1. – The five-step research plan and its status in this thesis.

	Step	Status	Where
1	Review the robustness and adaptability literature for MARL and adjacent single-agent work.	Completed	Ch. 5–7
2	Synthesize the two literatures into one taxonomy, compare them on a common set of axes, and identify the concrete gaps that block a joint treatment.	Completed	Ch. 8, Ch. 9
3	Design a method aimed at the gap: a single MARL policy that keeps its nominal performance while being robust to bounded disturbance and fast to adapt to a changed environment.	Completed (design); not implemented	Ch. 10
4	Evaluate the method and representative survey exemplars in a Search-and-Rescue multi-agent simulator.	Future work; outline given	Ch. 10
5	Define evaluation metrics that score robustness and adaptation together, and compare all methods under them.	Future work; metrics given	Ch. 10, Ch. 9

The comparison is then used to identify limitations that remain unresolved across the existing approaches. In particular, the review seeks to determine whether current methods can simultaneously provide robustness to uncertainty or perturbations and adaptation to changing task or environment conditions.

Because this is a narrative rather than a systematic review, the selection of literature is guided by relevance to the research questions rather than by a fully predefined and exhaustive search protocol. Consequently, the thesis does not claim a reproducible estimate of the percentage of the literature covered. To improve transparency, however, the main databases, search terms, inclusion criteria, and rationale for selecting representative methods are documented below.

Sources and span. Searches were run on Google Scholar and the arXiv listings, cross-checked against DBLP for venue and version, over 1993–2026. The lower bound is the first framing of Markov games for multi-agent RL; the upper bound is this thesis's knowledge cutoff and includes one venue-accepted 2026 paper.

Venues. Material came primarily from the algorithm and theory tracks of NeurIPS, ICML, ICLR, and AAMAS, from the associated journals, and from a small number of preprints where a preprint is the standard reference for a method (for example

M3DDPG and MIR3). Robotics and applied venues were consulted for sim-to-real and multi-agent meta-learning methods.

Search terms. Seed queries combined a mechanism term (*robust*, *adversarial*, *distributionally robust*, *domain randomization*, *non-stationary*, *change-point*, *sliding window*, *meta-reinforcement learning*, *task inference*) with a setting term (*multi-agent*, *cooperative*, *Markov game*, *Dec-POMDP*).

Snowballing and stopping rule. From each method that met the inclusion criteria, backward citations were followed to its prerequisites and forward citations to its variants. Search along a branch stopped when new papers only added further variants of a mechanism already represented by an exemplar. This is a saturation criterion on *mechanisms*, not on papers, and is why the exemplar set is small while the appendix catalogue is long.

2.1.1. Inclusion criteria

A method was included if it met one of the following.

- **Robust MARL.** It explicitly optimizes, bounds, or regularizes against a disturbance to a named channel, observations, actions, communication, transition dynamics, or reward, and states what happens when its own assumption about that disturbance is violated.
- **Adaptive MARL.** It explicitly tracks a non-stationary quantity online, rather than assuming a fixed environment and relying on generalization.
- **Meta-reinforcement learning.** Its *adaptation mechanism*, not merely its architecture, is relevant to tracking task or environment change. This separates the meta-RL material in Chapter 7 from the larger body of meta-RL work on representation learning and optimizer design for their own sake.
- **Prerequisite.** It is a single-agent mechanism that several MARL methods covered here build on directly, for example UCRL2's confidence-set planning [9] or the DDPG/TD3/SAC backbones, and is covered only to the depth those dependents require.

Methods were grouped by the problem they solve and the mechanism they use, not by venue or year. The reason is that the same mechanism recurs across otherwise unrelated papers. Examples are an information bottleneck, a confidence set, and a contrastive encoder.

2.1.2. Exclusion criteria

Some topics that appeared during the search were reduced to background or left out, because they do not bear on robustness or adaptability once the non-stationarity motivation is established. These are:

- the general game-theoretic taxonomy of solution concepts beyond the minimax value;
- tabular joint-action learning beyond what motivates the non-stationarity problem;
- communication-protocol design for its own sake;
- mean-field MARL;
- Bayesian meta-learning that refines the task-uncertainty estimate rather than the adaptation mechanism;
- fully offline meta-RL.

Single-agent value-based and policy-gradient algorithms appear only as prerequisites (Chapter 4). Table 2.2 names, for each branch, one method that was considered and not included.

2.1.3. Relation to existing surveys

Hernandez-Leal, Kaisers, Baarslag, *et al.* [6] survey learning in multi-agent environments around non-stationarity, and Zhang, Yang, and Başar [1] give a broad overview of MARL algorithms. Neither treats robustness to a bounded adversarial disturbance as part of the same problem as adaptability to an unbounded one. Moos, Hansel, Abdulsamad, *et al.* [16] survey robust reinforcement learning in similar depth to Chapter 5, but for the single-agent case, and without the multi-agent credit-assignment question raised in Chapter 8. This thesis differs in three ways. It places robust MARL and adaptive MARL under one taxonomy, it compares named methods on the same axes, and it uses that comparison to state concrete gaps and to motivate a specific design.

Table 2.2. – Representative methods considered but not carried into the taxonomy, with the selection criterion each fails.

Branch	Method considered	Reason not included
Adversarial training	Action-Robust RL / NR-MDP [10]	Perturbs only the acting agent's own action, a narrower single-agent threat model than RARL or M3DDPG, which perturb dynamics or teammates.
Adversarial regularization	CROP [11]	Certifies robustness by randomized smoothing, a different mechanism from RADIAL-RL's interval bound propagation; single-agent, with no established multi-agent extension.
Distributionally robust factorization	Robust Dynamic Programming [12]	Foundational single-agent ambiguity-set theory that DrIGM builds on, but predates multi-agent value factorization; prerequisite theory, not a MARL method.
Change-point detection	CUSUM [13]	Classical change detector; lacks the Bayesian run-length posterior that later RL work builds on directly.
Sliding-window estimation	Sliding-Window UCB [14]	Developed for the bandit setting; SW-UCRL2 is its direct MDP-level descendant and is covered instead.
Task-inference meta-RL	FOCAL [15]	Task inference is fully offline, whereas every task-inference method covered here assumes online interaction at test time.

2.2. Taxonomy and Gap Analysis

Within robust MARL and adaptive MARL, methods were grouped by *solution principle*: the general idea a group shares, independent of implementation. On the robust side there are four principles: adversarial training, adversarial regularization, distributionally robust factorization, and sim-to-real transfer. On the adaptive side there are three: change-point detection, sliding-window estimation, and structured adaptation. Within meta-RL there are three: policy-parameter gradient, black-box recurrence, and task inference. Each group is written to a fixed template, so that groups can be read against one another: *problem*; *why conventional MARL fails*; *solution principle*; *algorithm family*; *representative methods*; *comparison* (objective, assumption, mechanism, cost, guarantee, failure mode); and *takeaway*.

Selecting representatives. A method was made an exemplar, and developed in the main text, if it introduced the family's mechanism or is its clearest instance, and if its assumptions, cost, and failure mode differ from the other exemplars of that family. Most families have one to three exemplars. Adversarial regularization has four, because the same principle, sensitivity control, appears there in four different forms. Every other included method that repeats an exemplar's mechanism is recorded with a one-line note on what it changes, either in the relevant chapter or in Appendix A.

Comparison axes. The axes used in Chapter 8 were not fixed in advance. They are the dimensions on which the per-method records from step 1 differ: the *objective* optimized, *when* the disturbance is handled relative to deployment, what is *assumed* about it, the *computational mechanism*, and the *failure mode*. Two further axes, *information availability* and *temporal behaviour*, were added in Chapter 8 when the five-axis view showed they were needed. An axis was kept only if a specific exemplar from each side could be scored on it and the scores differed.

Statement types. Because a synthesis of several papers can be mistaken for a claim one of them makes, Chapter 8 labels every non-trivial statement as [**Literature**] (a result an existing paper establishes), [**Synthesis**] (a claim that follows from comparing several papers but is stated by none), or [**Proposed**] (a direction this thesis argues is worth investigating and does not test). No combination of methods is called novel unless the cited literature already establishes it.

2.3. Method Design

The gap analysis of Chapter 9 identifies where current methods and their evaluations fall short of a policy that is at once nominal-performant, robust, and fast-adapting. Chapter 10 gives the method that targets that gap, in the notation of Chapter 3: a unified framework on the FMASAC backbone that pairs a contrastive context encoder with an exchangeable-process Gaussian posterior, separate information-gain exploration and execution policies, an M3DDPG-style worst-case (MAAL) teammate-action perturbation with a variational encoder that infers the adversary's budget, scoped to the execution stream, and a second-phase, representation-learned change-point detector that resets the context and re-triggers exploration. It is presented at the level of an architecture, two training phases, and three algorithms. The method is not implemented in this thesis.

2.4. Planned Evaluation (Future Work)

Chapter 10 also outlines the evaluation that would test the method. It uses a *Search-and-Rescue* multi-agent environment, chosen because it carries both kinds of change that a deployed team faces: bounded disturbances, such as sensor noise, intermittent communication among rubble, and a robot that fails; and task change, such as the layout being resampled or switching within an episode. The chapter lists a set of scenario dimensions along which the non-stationarity is varied, a set of metrics (nominal return, worst-case return, degradation, recovery time, adaptation regret) measured on the same runs, and ablations that remove one component at a time. Implementing the method, running this evaluation, and reporting the comparison is the next stage of the work.

2.5. Limitations and Threats to Validity

Single author and non-systematic selection. Inclusion decisions were made by one person, using the criteria of Section 2.1.1, without a second reviewer and without a pre-registered protocol. Coverage therefore cannot be quantified. Every category-level exclusion is stated and justified, and every branch names at least one excluded method, so the boundary of what was left out can be checked even though it is not reproducible.

Selection bias toward methods that report failure modes. The inclusion criteria favour methods whose papers state what happens when their assumption is violated. This may make the literature look more self-critical than it is.

No reproduction. Every quantitative claim in the survey chapters is the one reported by the method's own authors, in their own environment. Differences between papers are discussed in Chapter 9; this thesis did not re-run any experiment to resolve them.

The proposed method is specified but not tested. Chapter 10 gives the method in full and argues from the surveyed evidence that it could obtain nominal performance, robustness, and adaptation speed together. This is a hypothesis. Until the evaluation of steps 4 and 5 is carried out, any advantage over the surveyed methods is conjectural, and the coupling of its parts into a stable two-phase pipeline is unverified.

Moving target, language, and access. Robust and adaptive MARL are active fields. The 2026 cutoff means that some gaps identified here may be partly closed by later work. Only English-language work reachable through the sources listed above was considered.

3. Problem Formulation

This chapter makes the deployment problem of Chapter 1 precise. It fixes the multi-agent model (Section 3.1) and then describes two ways the deployed environment can differ from the training one. The first is a *bounded* disturbance that leaves the environment intact but corrupts what agents sense and do (Section 3.2). The second is a *structural* change that places the agents in a different task (Section 3.3). Section 3.4 states the distinction the thesis is built around, and Section 3.5 states the combined problem it targets. How each kind of change is *handled* is left to later chapters: robust methods in Chapter 5, adaptive and meta-learning methods in Chapters 6 and 7.

3.1. The Multi-Agent Model

A cooperative multi-agent system is modelled as a decentralized, partially observable Markov decision process (Dec-POMDP). This is the appropriate model when each agent acts on its own private observation rather than the full state, which is the usual case for a deployed team.

Definition 3.1 (Dec-POMDP)**.** *A Dec-POMDP is a tuple*

$$\mathcal{M} = \left\langle N,\ S,\ \{A_i\}_{i=1}^{N},\ P,\ R,\ \{\Omega_i\}_{i=1}^{N},\ O,\ \gamma \right\rangle,$$

where N is the number of agents, S the global state space, A_i the action space of agent i with joint action space $A = \prod_{i=1}^{N} A_i$, $P(s' \mid s, a)$ the transition function for a joint action $a = (a_1, \ldots, a_N)$, $R(s, a) \in \mathbb{R}$ the shared team reward, Ω_i the observation space of agent i, $O(o \mid s, a)$ the observation function giving each agent's observation $o_i \in \Omega_i$, and $\gamma \in [0, 1)$ the discount factor. Each agent i acts on its own history $h_i = (o_i^0, a_i^0, \ldots, o_i^t)$ through a policy $\pi_i(a_i \mid h_i)$, and the joint policy is $\pi = (\pi_1, \ldots, \pi_N)$.

Training solves for a joint policy that maximizes the expected discounted team return under one fixed *nominal* model $\mathcal{M}^0 = \langle N, S, \{A_i\}, P^0, R^0, \{\Omega_i\}, O^0, \gamma \rangle$,

$$J(\pi) = \mathbb{E}_{\tau \sim \pi,\, \mathcal{M}^0}\left[\sum_{t=0}^{\infty} \gamma^t R^0(s_t, a_t)\right], \qquad \pi^0 = \arg\max_{\pi} J(\pi). \tag{3.1}$$

Everything that follows is about what happens when the environment the policy actually runs in is not $\mathcal{M}^0$.

3.2. Bounded Disturbance: the Robustness Problem

In the first case the environment is still $\mathcal{M}^0$, but every channel an agent depends on can be corrupted by a bounded amount. Let $\delta_{o,i}$, $\delta_{a,i}$, and δ_s denote perturbations to agent i's observation, agent i's executed action, and the global state:

$$\hat{o}_i = o_i + \delta_{o,i}, \qquad \hat{a}_i = a_i + \delta_{a,i}, \qquad \hat{s} = s + \delta_s. \tag{3.2}$$

When agents share information, for instance a message m_{ij} from agent j aggregated by agent i through an attention weight α_{ij},

$$g_i = \sum_j \alpha_{ij}\, m_{ij}, \tag{3.3}$$

that aggregated message is also a channel that can be corrupted in the same way. In every case the perturbation is assumed *bounded*: $\delta \in \Delta$, where Δ is usually a product of ℓ_p-balls $B_\epsilon(\cdot)$ of fixed radius ϵ. The disturbance is unknown in advance, but it cannot be arbitrarily large.

A robust policy optimizes for the worst case inside this bound rather than the average case. For a fixed policy π, its robust value is the lowest return that any allowed perturbation can cause,

$$V^{\mathrm{rob}}(\pi) = \min_{\delta \in \Delta} \mathbb{E}_{\mathcal{M}^0}\left[\sum_{t=0}^{\infty} \gamma^t R^0(s_t, a_t) \,\middle|\, \pi, \delta\right], \tag{3.4}$$

and training solves for the policy with the best worst case,

$$\pi^{\mathrm{rob}} = \arg\max_{\pi} V^{\mathrm{rob}}(\pi) = \arg\max_{\pi} \min_{\delta \in \Delta} \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R^0(s_t, a_t) \,\middle|\, \pi, \delta\right]. \tag{3.5}$$

When the disturbance acts on the transition model rather than on an observation or action channel, the same idea is written as a robust Bellman operator. Let $\mathcal{P}(s,a)$ be the set of transition kernels regarded as plausible around the nominal $P^0(\cdot \mid s,a)$:

$$\left(T^{\mathrm{rob}}Q\right)(s,a) = R(s,a) + \gamma \inf_{P(\cdot|s,a) \in \mathcal{P}(s,a)} \mathbb{E}_{s' \sim P(\cdot|s,a)}\left[\max_{a' \in A} Q(s', a')\right]. \tag{3.6}$$

If the set $\bigcup_{s,a} \mathcal{P}(s,a)$ is rectangular [12], then T^{rob} is a γ-contraction and its fixed point is the robust value. It evaluates every action against the single worst transition model in $\mathcal{P}(s,a)$ instead of the nominal one. Chapter 5 explains how $\mathcal{P}(s,a)$ is chosen and how the inner infimum is made tractable.

Realizing the worst case with an adversary. Equation (3.5) is only useful once the inner minimization can be computed. One way to do this is to introduce an adversary that plays the role of $\min_\delta$. The Dec-POMDP is extended so that every protagonist agent i has an adversary i' with the same state and observation, whose goal is to reduce the protagonist's return:

$$\mathcal{M}^{\text{adv}} = \left\langle \{(i, i')\}_{i=1}^{N},\ S,\ \{A_i\},\ \{A'_i\},\ \{\Omega_i\},\ O,\ P,\ R \right\rangle, \tag{3.7}$$

where A'_i is the adversary's action space, typically the same bounded perturbation set $B_\epsilon(s) \times B_\epsilon(a)$ used in Equation (3.2). The game is zero-sum: the adversary's reward is the negative of the protagonist's,

$$R^{i'}(s, a, \delta) = -R^i(s, a, \delta), \qquad \text{so that} \qquad Q^{i'} = -Q^i \tag{3.8}$$

under the shared dynamics. Training alternates between the protagonist and the adversary,

$$\pi^{\text{rob}} = \arg\max_{\pi} \min_{\delta_s \in B_\epsilon(s),\ \delta_a \in B_\epsilon(a)} \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t \,\middle|\, \pi, \delta_s, \delta_a \right]. \tag{3.9}$$

Training a full adversary is expensive. A cheaper option approximates the worst-case perturbation with a first-order attack such as the Fast Gradient Sign Method,

$$\delta = \epsilon \cdot \operatorname{sign}\big(\nabla_o \mathcal{L}\big), \tag{3.10}$$

which moves in the direction that increases the training loss $\mathcal{L}$ the most, instead of solving the inner minimization exactly [3], [4]. A third option replaces the attack with a regularizer that penalizes the policy's sensitivity to a bounded input change [5]. RARL [17] is the standard instance of the game form. All three appear in Chapter 5.

The cost. Every method here optimizes for the worst $\delta \in \Delta$. When the deployed system runs with no disturbance ($\delta = 0$), it still follows a policy shaped by a case that is not occurring, so its return can be lower than that of a policy fitted to $\mathcal{M}^0$. Measuring this cost, and asking whether it can be avoided, is RQ3.

3.3. Structural Change: the Adaptability Problem

In the second case the disturbance is not a bounded corruption of $\mathcal{M}^0$. The agents face a different task. This is modelled as a distribution over related Dec-POMDPs. Let $p(\mathcal{T})$ be a distribution over a task space $\mathcal{M}$, where each task

$$\mathcal{T} = \left\langle N,\ S,\ \{A_i\},\ P_{\mathcal{T}},\ R_{\mathcal{T}},\ \{\Omega_i\},\ O,\ \gamma \right\rangle, \qquad \mathcal{T} \sim p(\mathcal{T}), \tag{3.11}$$

shares the state, action, and observation spaces of every other task, but can have a different transition function $P_{\mathcal{T}}$, a different reward $R_{\mathcal{T}}$, or both. Unlike the bounded case, there is no limit on how far $P_{\mathcal{T}}$ can move from task to task. The only assumption is that the tasks share enough structure to be worth learning from together.

No single fixed policy performs well across every $\mathcal{T} \sim p(\mathcal{T})$, so the objective is no longer to solve one task. It is to learn an *adaptation procedure*: a function f_θ that maps a small amount of interaction data $D_{\mathcal{T}}$ from a new task to a policy for that task,

$$\phi = f_\theta(D_{\mathcal{T}}), \qquad \max_\theta \; \mathbb{E}_{\mathcal{T}\sim p(\mathcal{T})}\left[J_{\mathcal{T}}\left(\pi_{f_\theta(D_{\mathcal{T}})}\right) \right]. \tag{3.12}$$

This is the meta-reinforcement-learning view of adaptability. Instead of preparing for the worst case inside a bounded set, the team learns in advance, across many related tasks, how to update its behaviour quickly once a new task appears [18]–[20]. Chapter 7 develops the meta-learning families. Chapter 6 develops the alternative approach of detecting the change explicitly and then re-estimating.

3.4. The Central Distinction

Sections 3.2 and 3.3 answer two different questions with two different kinds of guarantee.

Robustness	**Adaptability**
The disturbance is *bounded* and unknown in advance, possibly adversarial.	The disturbance can be *unbounded over time*, but is structured as a task distribution $p(\mathcal{T})$.
Prepare for the worst case inside the bound *before* deployment (Eq. (3.5)).	Learn an adaptation procedure and update the policy *after* deployment (Eq. (3.12)).
Guarantee: a floor on return over Δ.	Guarantee: fast convergence to near-optimal return for $\mathcal{T} \sim p(\mathcal{T})$.

This thesis treats both as responses to the same fact: a policy trained on $\mathcal{M}^0$ does not automatically work in the conditions the agents actually meet. The two problems differ only in what they assume about how those conditions fail to match. Chapter 8 shows that they are closer still. Both keep a set of environment models that the policy is protected against; robustness fixes this set, while adaptability re-estimates it.

3.5. The Combined Problem

The two settings are usually treated separately, and a method built for one does not solve the other. A worst-case-robust policy has no mechanism to track an unbounded change. A fast-adapting policy has no protection against a bounded adversarial perturbation while it is still adapting. A deployable multi-agent system needs one joint policy π that, for a team of N agents under centralized training with decentralized execution (Section 4.8):

1. retains near-nominal team return when $\delta = 0$ and the task is the nominal one (RQ3);
2. does not collapse under a bounded perturbation $\delta \in \Delta$ on the observation, action, or state channel of Equation (3.2) (RQ2, RQ3);
3. reaches near-optimal return for a previously unseen task $\mathcal{T} \sim p(\mathcal{T})$ within a small amount of deployment interaction, for both abrupt and gradual change (RQ4).

No method surveyed in Chapters 5–7 is shown to meet all three. In addition, meta-learning, which the author intends to use for point 3, has been developed almost entirely for the single-agent case. Chapter 9 states these shortfalls as research problems, and Chapter 10 proposes a method that addresses them by combining contrastive task inference over an exchangeable-process posterior with an M3DDPG-style worst-case teammate-action perturbation and a representation-learned change-point detector on a shared cooperative backbone. The method is specified in full; its evaluation is future work.

4. Foundations

Chapter 3 stated the deployment problem and its two settings. This chapter introduces the formal vocabulary the survey uses to compare solutions to it, and nothing more. Single-agent reinforcement learning is summarized only to the depth that later chapters need. Game theory is reduced to a single concept, the minimax value, which the robust chapter uses. Two topics are treated at more length because the thesis depends on them: the non-stationarity that other learning agents create (Section 4.7), which is the common cause behind Chapters 5–7, and the centralized-training baselines (Section 4.8) that the later methods extend. The chapter closes by comparing those baselines and selecting the one this thesis builds its own method on (Section 4.9).

4.1. Markov Decision Processes and the Learning Objective

A Markov decision process (MDP) is a tuple $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ with state set $\mathcal{S}$, action set $\mathcal{A}$, transition kernel $P(s' \mid s, a)$, reward function $r(s, a)$, and discount $\gamma \in [0, 1)$. A policy $\pi(a \mid s)$ induces a distribution over trajectories; the objective is to maximize the expected discounted return

$$J(\pi) \;=\; \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t \, r(s_t, a_t)\right],$$

with state value $V^{\pi}(s) = \mathbb{E}_{\pi}[\sum_t \gamma^t r_t \mid s_0 = s]$ and action value $Q^{\pi}(s, a)$ defined in the same way. The optimal value functions satisfy the Bellman optimality equation $Q^{\star}(s, a) = r(s, a) + \gamma \, \mathbb{E}_{s'}[\max_{a'} Q^{\star}(s', a')]$. Two points matter later. First, the objective is an expectation under a *single fixed* P. Robust methods (Chapter 5) replace it with a worst case over a set of kernels, and adaptive methods (Chapter 6) replace the fixed P with a time-indexed sequence P_t. Second, V^{π} and Q^{π} are defined relative to a policy. When the policy that generates the data differs from the one being evaluated, which happens whenever another agent is also learning, the value estimates are biased.

4.2. Stochastic Games

The multi-agent generalization is the stochastic (Markov) game [21], a tuple $(\mathcal{N}, \mathcal{S}, \{\mathcal{A}^i\}, P, \{r^i\}, \gamma)$ with agent set $\mathcal{N} = \{1, \dots, n\}$. At each step every agent i selects $a^i \in \mathcal{A}^i$; the joint action $\mathbf{a} = (a^1, \dots, a^n)$ drives a shared transition $P(s' \mid s, \mathbf{a})$ and yields a per-agent reward $r^i(s, \mathbf{a})$. Write $\mathbf{a} = (a^i, \mathbf{a}^{-i})$ to separate agent i's action from the others'. The game is *cooperative* (team) when all r^i coincide, *zero-sum* when they sum to zero, and *general-sum* otherwise. This thesis is concerned mainly with cooperative and team settings, with zero-sum structure appearing inside robust methods as the protagonist–adversary game of Section 5.4.

4.3. Partial Observability

In most multi-agent settings no agent sees the full state. The decentralized partially observable MDP (Dec-POMDP) adds an observation set $\mathcal{O}^i$ and an observation function $O^i(o^i \mid s)$ for each agent. Agent i acts on its action–observation history $h_t^i = (o_0^i, a_0^i, \dots, o_t^i)$ rather than on s, so its policy is $\pi^i(a^i \mid h_t^i)$. A *joint history* $\mathbf{h}_t = (h_t^1, \dots, h_t^n)$ collects them.

Two consequences of partial observability recur later. First, the Markov property no longer holds at the level of observations: o_t^i is not a sufficient statistic for the future, so a policy that uses only the current observation is in general suboptimal. The methods of Chapters 6 and 7 spend most of their effort building a *compressed* summary of the history that recovers sufficiency approximately, such as a sliding window, a run-length posterior, or a latent task code. Second, a distributionally robust method must hedge over a distribution on next *joint histories* $\Delta(\mathcal{H})$, not next states $\Delta(\mathcal{S})$ (Section 5.6). The two coincide only when observations reveal the state, and Chapter 8 uses this gap when it compares a robust ambiguity set with an adaptive confidence set. Where the distinction does not matter, this thesis writes s for the argument of a policy or value function and means "state or history, as appropriate".

4.4. The MARL Objective

A joint policy $\boldsymbol{\pi} = (\pi^1, \dots, \pi^n)$ gives each agent a return $J^i(\boldsymbol{\pi}) = \mathbb{E}_{\boldsymbol{\pi}}[\sum_t \gamma^t r_t^i]$. In the cooperative case all J^i are equal and the target is simply $\max_{\boldsymbol{\pi}} J(\boldsymbol{\pi})$. The difficulty is that $\boldsymbol{\pi}$ is optimized in a decentralized way: each agent controls only its own π^i. In the general-sum case there is no single scalar to maximize, and "optimal" is replaced by an equilibrium (Section 4.5). The structural point, developed in Section 4.7, is

that from agent i's viewpoint the term $P(s' \mid s, a^i, \mathbf{a}^{-i})$ depends on the other agents' policies through $\mathbf{a}^{-i}$, so agent i does not face a stationary MDP unless the other agents are held fixed.

4.5. Equilibrium Concepts: Only What Is Used

Given a fixed opponent profile $\boldsymbol{\pi}^{-i}$, agent i's *best response* maximizes $J^i(\pi^i, \boldsymbol{\pi}^{-i})$. A *Nash equilibrium* is a joint policy in which every agent plays a best response to the others [22], so no agent gains by unilateral deviation. *Correlated* and *coarse correlated* equilibria generalize this to distributions over joint actions coordinated by a shared signal [23], [24], and are the natural targets of no-regret learning dynamics [25].

This thesis does not organize MARL around equilibrium selection, for two reasons that also bound its scope. First, equilibria in general-sum games are not unique and are hard to compute or select, so "the agents reach an equilibrium" is not a useful description of deployed behaviour. Second, an equilibrium is a statement about a *fixed* game, while the subject of this thesis is what happens when the game does not stay fixed. Equilibrium concepts are therefore treated as background.

The one concept used directly is the *minimax* (max–min) value. For a zero-sum game, agent i's security level

$$\underline{V}^i \;=\; \max_{\pi^i} \min_{\boldsymbol{\pi}^{-i}} J^i(\pi^i, \boldsymbol{\pi}^{-i})$$

is the return it can guarantee whatever the opponents do. By the minimax theorem it equals the value with the order of play swapped, and a policy that attains it is safe against a worst-case opponent without needing to predict that opponent. The robust chapter is the pursuit of a security level in which the "opponent" is something else: an external force in RARL, the teammates' actions in M3DDPG, the transition kernel in distributionally robust MARL, and the simulator's physical parameters in sim-to-real work (Chapter 5). Reading robustness as a security level also lets Chapter 8 place it next to the adaptive objective, which replaces the worst-case opponent with a *drifting* one.

4.6. Single-Agent Deep RL Backbones

Later chapters extend a small number of single-agent algorithms. This section states what each one contributes and where it is reused. Derivations are omitted.

Table 4.1. – Single-agent backbones and the robust or adaptive MARL methods that extend them.

Backbone	What it adds	Extended by
DQN	Neural Q-learning with replay and target network	Value decomposition (VDN, QMIX); RADIAL-RL certification
PPO/TRPO	Trust-region / clipped on-policy updates	MAPPO; MAML and meta-gradient estimators
DDPG	Deterministic actor trained through a critic	MADDPG; M3DDPG
TD3	Twin critics, delayed and smoothed updates	Adversarial-regularization and sim-to-real backbones
SAC	Maximum-entropy actor–critic, off-policy	FMASAC; PEARL and off-policy task-inference meta-RL

Value-based. DQN [26] fits $Q^\star$ with a neural network trained on the temporal-difference target $r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a')$, using a replay buffer and a slowly updated target network $\bar{\theta}$ for stability. Value decomposition for cooperative MARL (Section 4.8) and the certified-robust method RADIAL-RL (Chapter 5) build on this backbone.

Policy-gradient. REINFORCE [27] ascends $\nabla_\theta J = \mathbb{E}[\nabla_\theta \log \pi_\theta(a \mid s)\, A(s,a)]$ with an advantage estimate A [28]. TRPO [29] and PPO [30] constrain each update so that the new policy stays close to the old one, which makes on-policy learning practical at scale. MAPPO (Section 4.8) and the meta-gradient methods of Chapter 7 use this machinery.

Actor–critic for continuous control. The deterministic policy gradient (DPG) [31] and its deep form DDPG [32] learn a deterministic actor $\mu_\theta(s)$ trained through a critic Q_ϕ. TD3 [33] adds twin critics and delayed updates to reduce overestimation. SAC [34] adds an entropy term for exploration and stability. Table 4.1 lists which robust or adaptive MARL method builds on each. These are the only single-agent algorithms the thesis assumes.

4.7. Non-Stationarity in MARL: The Root Cause

Fix agent i and marginalize the other agents out of the joint transition. The environment i actually experiences has transition kernel

$$\tilde{P}_i(s' \mid s, a^i) \;=\; \sum_{\mathbf{a}^{-i}} \Big(\textstyle\prod_{j \neq i} \pi^j(a^j \mid h^j) \Big)\, P(s' \mid s, a^i, \mathbf{a}^{-i}),$$

and a reward averaged in the same way. This kernel depends on the other agents' policies $\boldsymbol{\pi}^{-i}$. While those policies are being learned, $\boldsymbol{\pi}^{-i}$ changes between updates, so $\tilde{P}_i$ changes between updates. Agent i is therefore learning against a *moving* environment, even when the underlying game $(\mathcal{S}, \{\mathcal{A}^i\}, P, \{r^i\})$ is stationary and there is no adversary or external drift [1], [6].

This internal non-stationarity has three consequences that matter later. *Convergence is not guaranteed.* Independent Q-learners, each treating $\tilde{P}_i$ as a fixed MDP, can cycle instead of converging, because each agent's update breaks the stationarity assumption the others just used. *Experience becomes stale.* A transition collected under an old $\boldsymbol{\pi}^{-i}$ is a sample from a different $\tilde{P}_i$ than the current one, so a replay buffer contains samples from many environments. This is the same problem that an adaptive learner faces under external drift (Chapter 6). *The target is a set, not a point.* An agent that will face a range of teammate policies is effectively facing a set of environments, which is the object a robust learner hedges over (Chapter 5). CTDE (Section 4.8) is the standard first response: condition the critic on $\mathbf{a}$, so that the target is stationary once the joint action is given. This removes the internal non-stationarity during training, but not the external kind, which is why the rest of the thesis is needed.

On top of this internal non-stationarity, deployment adds external changes from the training conditions. Table 4.2 classifies them by the channel through which they enter, by whether they are treated as bounded or as unbounded but structured, and by the chapter that handles each. This table answers RQ1 and is referenced, not repeated, in later chapters.

4.8. Centralized-Training Baselines

Nearly every method in this thesis is built on *centralized training with decentralized execution* (CTDE). During training a learner may use the joint state and all agents' actions, but each agent's deployed policy $\pi^i(a^i \mid h^i)$ uses only local information. CTDE addresses the non-stationarity of Section 4.7 directly: a centralized critic that conditions on $\mathbf{a}$ sees a stationary target even while the individual policies change. Four baselines recur.

Value decomposition (VDN, QMIX). For cooperative teams the joint action value is learned centrally but must be maximized decentrally. The *individual-global-max* (IGM) property is the condition that makes this possible:

$$\arg\max_{\mathbf{a}} Q_{\text{tot}}(\mathbf{h}, \mathbf{a}) \;=\; \Big(\arg\max_{a^1} Q^1(h^1, a^1),\; \ldots,\; \arg\max_{a^n} Q^n(h^n, a^n) \Big),$$

Table 4.2. – Sources of uncertainty and non-stationarity for a deployed MARL policy, by entry channel. "Bounded" departures are the province of robust MARL (Chapter 5); "structured, unbounded" departures are the province of adaptive MARL (Chapters 6–7).

Channel	Disturbance	Typical assumption
Observation	Sensor noise, occlusion, or adversarial perturbation of o^i within an ℓ_p ball	Bounded
Action	Actuator fault or adversarial perturbation of the executed a^i	Bounded
Communication	Delayed, dropped, or corrupted messages between agents	Bounded (rate or budget)
Transition dynamics	Sim-to-real gap; parameter shift; wear	Bounded set at deployment; structured drift after
Reward	Task or objective change over time	Structured, unbounded
Other agents	Teammate failure or adversarial control; opponents and teammates that keep learning	Bounded (worst-case teammate) or structured (drifting policies)
Task identity	Deployment on a new but related task drawn from a family	Structured (task distribution)

so each agent can act greedily on its own utility Q^i and the team still maximizes Q_{tot}. VDN [35] enforces IGM with the sufficient condition $Q_{\text{tot}} = \sum_i Q^i(h^i, a^i)$. QMIX [36] relaxes the sum to a state-conditioned mixing network $Q_{\text{tot}} = f_s(Q^1, \ldots, Q^n)$ that is constrained to be monotonic, $\partial f_s / \partial Q^i \geq 0$, which is a larger IGM-safe class. *Relevance:* DrIGM (Section 5.6) is the class of mixing functions for which IGM still holds *after* the robust Bellman operator is applied, that is, for which robustness and factorization commute. RADIAL-RL certifies each per-agent Q^i network separately and relies on the mixing being monotonic so that the per-agent certificates combine.

MADDPG. MADDPG [37] gives each agent a DDPG actor $\mu^i_\theta(o^i)$ and a centralized critic $Q^i_\phi(s, a^1, \ldots, a^n)$ that conditions on every agent's action, so the critic target is stationary despite the other actors changing. *Relevance:* M3DDPG (Chapter 5) replaces the critic target's expectation over teammate actions with a worst case over them.

MAPPO. MAPPO [38] runs PPO with a shared centralized value function $V_\psi(s)$ as the advantage baseline, and is a strong cooperative baseline despite its simplicity. *Relevance:* the adversarial-regularization methods of Chapter 5 attach their penalty to a MAPPO- or QMIX-style backbone rather than replacing it.

FMASAC. Factored Multi-Agent Soft Actor-Critic [39] combines soft actor-critic with value factorization. Each agent has a stochastic (maximum-entropy) actor $\pi_{\theta_i}(a^i \mid h^i)$ and a utility $Q^i(h^i, a^i)$, and a monotonic mixing network $Q_{\text{tot}} = g_\omega(s, \{Q^i\})$ combines them while preserving IGM. The factored soft policy objective is

$$\mathcal{L}(\pi_\theta) = -\,g_\omega\Big(s,\ \big\{\,\mathbb{E}_{\pi_\theta}\big[\,Q^i(h^i,a^i) - \alpha \log \pi_{\theta_i}(a^i \mid h^i)\,\big]\,\big\}_{i=1}^{n}\Big),$$

so each agent's update is shaped by its own entropy-regularized utility and by the mixer's state-dependent weighting: credit is assigned to each agent implicitly through the factorization. Training is off-policy from a replay buffer, with the soft Bellman target

$$y_{\text{tot}} = r + \gamma\Big(Q_{\text{tot}}^{\bar\phi,\bar\omega}(\mathbf{h}', \mathbf{a}', s') - \alpha \log \pi_{\bar\theta}(\mathbf{a}' \mid \mathbf{h}')\Big), \qquad \mathbf{a}' \sim \pi_{\bar\theta},$$

and the temperature α is tuned automatically against a target entropy H_0. It targets the two weaknesses of ordinary multi-agent actor-critic: high policy-gradient variance and poor credit assignment. *Relevance:* it is the backbone this thesis adopts for its own method (Section 4.9).

All four follow the same pattern. The problem is that decentralized policies make each agent's learning target non-stationary. The idea is to centralize the critic during training so that the target is stationary. The objective is the same expected return, estimated with a centralized baseline. For this thesis, the point is that the robust and adaptive methods that follow modify the *target* or add a *penalty*, and inherit everything else from the baseline.

4.9. Comparison of the Baselines and the Choice for This Thesis

The proposed method (Chapter 10) must be robust to a bounded perturbation and fast to adapt to a changed task at the same time. That places three demands on whatever CTDE learner it is built on. Table 4.3 scores the four baselines against them.

Demand 1: off-policy and sample-efficient. Both sides of the proposed method multiply the interaction a learner needs. Adaptation must be learned across a distribution of tasks, and after deployment it operates under a small interaction budget; robustness, if it uses adversarial training, adds an inner optimization per step. A learner that reuses a replay buffer is far cheaper than one that discards on-policy data. This rules out MAPPO.

Demand 2: principled multi-agent credit assignment. From a single team reward, each agent's policy update needs a signal that reflects its own contribution, not just the team's. A backbone whose critic is decomposed per agent through a monotonic mixer provides this directly, and it is also the natural starting point for the change-attribution problem that Chapter 9 identifies as unsolved. This rules out MADDPG, whose centralized critic produces a single global signal.

Demand 3: a stochastic policy with tunable exploration. After a change, the policy must keep exploring to re-adapt. The maximum-entropy objective with an automatically tuned temperature does this without a hand-set schedule, which is what VDN and QMIX lack: they act ε-greedily on a value function.

Why FMASAC (R4). FMASAC is the only one of the four that meets all three demands. It is off-policy and sample-efficient (SAC), it factorizes a *soft* critic so credit is assigned per agent through the mixer while IGM keeps execution decentralized, and its entropy temperature is tuned automatically. It is therefore adopted as the base learner for the method of Chapter 10 — the cooperative backbone that requirement R4 (Section 9.2) calls for: the robustness and adaptation components are added on top of it, and it also serves as the plain-baseline reference in the planned evaluation. Its one inherited weakness, the monotonic-mixer restriction it shares with QMIX, is accepted because decentralized execution under IGM is a hard requirement here.

Table 4.3. – The four CTDE baselines on the axes that matter for a robust, fast-adapting method. FMASAC is the only one that meets all three demands stated below the table.

Baseline	**Data use**	**Policy**	**Multi-agent credit assignment**	**Main limitation**
VDN / QMIX	Off-policy; sample-efficient	Value-based, ε-greedy	Implicit, through additive (VDN) or monotonic (QMIX) factorization; IGM preserved	No stochastic policy; exploration is a hand-set schedule; monotonic mixer restricts representable values
MADDPG	Off-policy; moderate	Deterministic actor	Global signal only: the centralized critic conditions on all actions but is not decomposed per agent; no IGM	Weak credit signal; unstable with many agents
MAPPO	On-policy; sample-inefficient	Stochastic (PPO)	A shared value baseline only; no per-agent decomposition	Discards data every iteration, so it needs many environment interactions
FMASAC	Off-policy; sample-efficient	Stochastic, maximum-entropy, auto-tuned α	Implicit, through a monotonic factorization of the *soft* critic; IGM preserved	Inherits QMIX's monotonic-mixer restriction

Part II.

Literature Review

5. Robust Multi-Agent Reinforcement Learning

This chapter answers RQ2 and the first part of RQ3. Section 5.1 defines robustness. Sections 5.2–5.7 review the methods. Each method is stated as the *problem* it solves, the *mechanism*, and the *equation* that expresses the solution. Section 5.8 compares them, and Section 5.9 states what they leave open. Chapter 10 proposes a method that addresses that gap.

5.1. Definition of Robustness

Robustness is good performance under a constrained perturbation. The deployment environment is the nominal Dec-POMDP $\mathcal{M}^0$ of Chapter 3 with a bounded, unknown disturbance $\delta \in \Delta$ applied to an agent's observation, action, or state channel (Eq. (3.2)), or to the transition kernel. A robust policy maximizes the worst-case return over Δ (Eq. (3.5)),

$$\pi^{\text{rob}} = \arg\max_{\pi} \min_{\delta \in \Delta} \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R^0(s_t, a_t) \,\middle|\, \pi, \delta \right],$$

or, when the disturbance acts on the transition model, satisfies the robust Bellman fixed point (Eq. (3.6))

$$\left(T^{\text{rob}} Q\right)(s, \mathbf{a}) = R(s, \mathbf{a}) + \gamma \inf_{P(\cdot|s,\mathbf{a}) \in \mathcal{P}(s,\mathbf{a})} \mathbb{E}_{s' \sim P}\left[\max_{\mathbf{a}'} Q(s', \mathbf{a}') \right]$$

over an uncertainty set $\mathcal{P}$ built around the nominal kernel [12]. The set Δ (or $\mathcal{P}$) is fixed before training. The families below differ in how this set is chosen and how the inner minimization is made tractable.

5.2. Types of Uncertainty

The robust literature designs against the bounded rows of Table 4.2: *observation* noise or attack, $\hat{o}_i = o_i + \delta_{o,i}$ with $\|\delta_{o,i}\|_p \le \epsilon$; *action* corruption, $\hat{a}_i = a_i + \delta_{a,i}$; *state or*

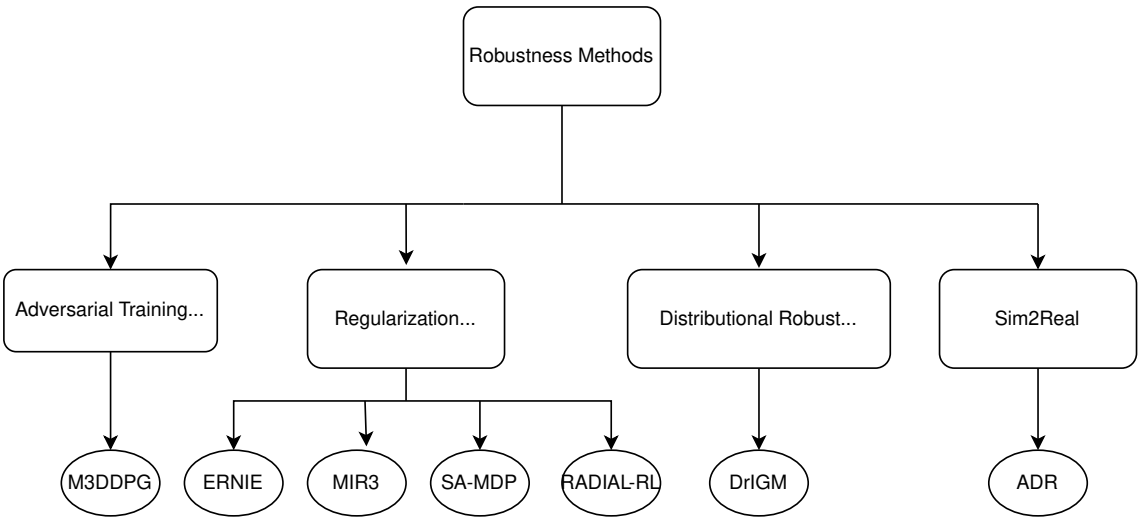

Figure 5.1. – Taxonomy of robust MARL methods: adversarial training (M3DDPG), robustness through regularization (SA-MDP, ERNIE, MIR3, RADIAL-RL), distributionally robust value factorization (DrIGM), and sim-to-real transfer (ADR).

message corruption, $\hat{s} = s + \delta_s$ (Eq. (3.2)–(3.3)); a *transition kernel* drawn from a set $\mathcal{P}$ around P^0 (sim-to-real gap, unmodelled physics); and a *teammate* whose policy may be anything within a set, including adversarial. Each family below commits to one or two of these channels; none covers all [40].

5.3. Taxonomy of Methods

Figure 5.1 groups the reviewed methods into four families by the form of the uncertainty set and how the worst case is realized.

5.4. Adversarial Training

Problem it solves. A policy trained only on the nominal model never sees a hard disturbance. In a team, a change in teammate behaviour is such a disturbance: an agent that is optimized for cooperative teammates can be exploited by an uncooperative one.

Mechanism. M3DDPG [41] extends MADDPG. It replaces the centralized critic's expectation over teammate actions with a worst case, so that agent i trains against the joint teammate action that minimizes its own value. The exact inner minimization is intractable, so *multi-agent adversarial learning* (MAAL) approximates the adversarial teammate action with one gradient-descent step on the critic.

Equation. The robust critic target and its one-step approximation are

$$y^i = r^i + \gamma \min_{\mathbf{a}^{-i}} Q^i_{\bar{\phi}}\big(s',\, a'^i,\, \mathbf{a}^{-i}\big), \qquad \mathbf{a}^{-i} \leftarrow \mathbf{a}^{-i} - \alpha\, \nabla_{\mathbf{a}^{-i}} Q^i_{\bar{\phi}}\big(s',\, a'^i,\, \mathbf{a}^{-i}\big), \tag{5.1}$$

with $a'^i = \mu^i_{\bar{\theta}}(o'^i)$. RARL [17] is the single-agent antecedent, in which a separate adversary policy applies the perturbation and the two are trained by alternating updates.

5.5. Robustness Through Regularization

The four methods here share one idea. Instead of training an adversary, they add a term to the objective that limits the *sensitivity* of the policy or value to a bounded input perturbation,

$$\max_{\theta}\ \mathbb{E}\big[\textstyle\sum_t \gamma^t r_t\big] - \lambda\, \mathbb{E}_s\Big[\sup_{\hat{s}\in B_\epsilon(s)} \Phi\big(f_\theta(s),\, f_\theta(\hat{s})\big)\Big], \tag{5.2}$$

and differ only in the quantity f_θ held insensitive and the measure Φ.

SA-MDP — policy consistency. *Problem:* the policy can map two nearly identical observations to very different action distributions. *Mechanism:* penalize the divergence of the action distribution over a bounded observation ball [5]. *Equation*, for a team:

$$L^{\text{robust}}_{\text{MARL}} = L_{\text{MARL}} + \lambda \sum_{i=1}^{N} \max_{\hat{o}^i \in B_\epsilon(o^i)} D_{\text{KL}}\big(\pi_i(\cdot \mid o^i)\,\big\|\,\pi_i(\cdot \mid \hat{o}^i)\big), \tag{5.3}$$

with $\|\mu_i(o^i) - \mu_i(\hat{o}^i)\|_2$ replacing the KL term for a deterministic policy.

ERNIE — Lipschitz smoothness. *Problem:* SA-MDP's penalty is empirical, and the min–max problem it creates is unstable. *Mechanism:* enforce local Lipschitz continuity of the policy with an adversarial regularizer, and write the min–max as a Stackelberg game so that no separate adversary policy is trained [42]. *Equation:*

$$L = L_{\text{task}} + \lambda \big\|\pi_\theta(\cdot \mid s) - \pi_\theta(\cdot \mid s+\delta^\star)\big\|, \qquad \delta^\star = \arg\max_{\|\delta\|\le\epsilon} \big\|\pi_\theta(\cdot \mid s) - \pi_\theta(\cdot \mid s+\delta)\big\|. \tag{5.4}$$

MIR3 — information bottleneck. *Problem:* an agent that relies heavily on its full history is fragile to perturbed or non-stationary teammate information, and defending one named worst case is expensive. *Mechanism:* minimize the mutual information between an agent's trajectory τ^i and its action. The mutual information is estimated by the CLUB upper bound [43] (Appendix B), which the authors show is a lower bound on a robust objective [44]. *Equation:*

$$J(\pi) = \mathbb{E}[R] - \lambda\, I(\tau^i; a^i), \qquad \hat{I}_{\text{CLUB}} = \frac{1}{N^2} \sum_j \sum_k \Big[\log q_\phi(a_j \mid \tau_j) - \log q_\phi(a_k \mid \tau_j) \Big]. \tag{5.5}$$

RADIAL-RL — certified margin. *Problem:* a sampled attack proves nothing; the goal is a guarantee that the greedy action does not change for *any* δ in the budget. *Mechanism:* interval bound propagation gives provable bounds $[\underline{Q}, \overline{Q}]$ on each action's value under $\|\delta\|_p \le \epsilon$. A margin loss keeps the lower bound of the certified best action above the upper bound of every other action [45]. *Equation*, per QMIX agent:

$$L^i_{\text{margin}} = \sum_{a \neq a_i^\star} \max\Big(0,\ \overline{Q}_i(o_i, a) - \underline{Q}_i(o_i, a_i^\star)\Big), \qquad L_{\text{QMIX-R}} = L_{\text{QMIX}} + \lambda_M \sum_{i=1}^{N} L^i_{\text{margin}}. \tag{5.6}$$

5.6. Distributionally Robust Value Factorization

Problem it solves. A single ℓ_p-ball is a rigid perturbation model. A worse problem is that if each agent computes its own worst-case kernel $P_i^\star$, the per-agent worst-case models disagree ($P_1^\star \neq P_2^\star$) and the greedy joint action is no longer consistent.

Mechanism. DrIGM [46] makes the *joint* value robust against one shared worst-case model, and then derives each agent's robust Q from that same model. The uncertainty set is a product of per-history-action sets $\mathcal{P}_{\mathbf{h},\mathbf{a}} \subseteq \Delta(\mathcal{H})$, and the robust Bellman operator minimizes over it. DrIGM is the condition under which the per-agent robust greedy actions still maximize the robust joint value, so VDN, QMIX, and QTRAN need no change to their architecture.

Equation.

$$(\boldsymbol{T}Q)(\mathbf{h},\mathbf{a}) = r(s,\mathbf{a}) + \gamma \inf_{P\in\mathcal{P}_{\mathbf{h},\mathbf{a}}} \mathbb{E}_{\mathbf{h}'\sim P}\left[\max_{\mathbf{a}'} Q(\mathbf{h}',\mathbf{a}')\right], \qquad Q_i^{\mathrm{rob}}(h_i,a_i) := Q_i^{P^{\mathrm{worst}}(\mathbf{h},\bar{\mathbf{a}})}(h_i,a_i), \tag{5.7}$$

with $P^{\mathrm{worst}}(\mathbf{h},\mathbf{a}) \in \arg\inf_{P\in\mathcal{P}} Q_{\mathrm{tot}}^{P}(\mathbf{h},\mathbf{a})$ and the DrIGM property $\left(\arg\max_{a_1} Q_1^{\mathrm{rob}},\ldots,\arg\max_{a_N} Q_N^{\mathrm{rob}}\right) \subseteq \arg\max_{\mathbf{a}} Q_{\mathrm{tot}}^{\mathcal{P}}(\mathbf{h},\mathbf{a})$. Two set choices are used: ρ-contamination, $\mathcal{P}_{\mathbf{h},\mathbf{a}} = \{(1-\rho)P^0_{\mathbf{h},\mathbf{a}} + \rho\,\nu : \nu \in \Delta(\mathcal{H})\}$, giving the cheap target $y = r + \gamma(1-\rho)Q_{\mathrm{tot}}(s',\mathbf{a}')$; and a total-variation ball, $\mathrm{TV}(P, P^0_{\mathbf{h},\mathbf{a}}) \le \rho$, which is stronger but costlier.

Estimating the set from data. DrIGM fixes $\mathcal{P}_{\mathbf{h},\mathbf{a}}$ before training. A separate line of work keeps the same worst-case objective but *estimates* the uncertainty set from interaction rather than setting it by hand. Wang and Zou [7] do this for single-agent robust RL, updating the model-uncertainty set from streaming data while still optimizing the worst case over it. Farhat, Ghosh, Atia, *et al.* [8] give a sample-efficient version for the multi-agent case, solving a distributionally robust Markov game through online interaction rather than a fixed offline set. These methods sit between the robust families of this chapter and the online estimators of Chapter 6: the objective is still a minimax over a set, but the set is re-estimated as data arrive. Chapter 8 builds on this observation.

5.7. Sim-to-Real Robustness

Problem it solves. The policy is trained in a simulator whose physics, latency, and noise are inaccurate. Uniform domain randomization transfers better, but it spends most of its budget on configurations that are already easy.

Mechanism. Adversarial Domain Randomization [47] plays a minimax game between the team policy and a "Nature Player" that selects simulator parameters ξ to minimize return, constrained by a divergence $D(\xi,\xi_0) \le \kappa$ to stay physically plausible.

Equation.

$$\max_{\boldsymbol{\pi}} \min_{\xi\,:\,D(\xi,\xi_0)\le\kappa} \mathbb{E}_{P_\xi}\left[\sum_t \gamma^t R(s_t,\mathbf{a}_t)\right]. \tag{5.8}$$

A prioritized-replay variant in the same work weights domain-randomized transitions by TD-error and by how under-represented their parameter region is.

Table 5.1. – Robust MARL families: the uncertainty set, the mechanism, and the solving equation. All optimize a worst case over a set fixed before training.

Family	Uncertainty set / channel	Mechanism	Eq.
Adversarial training (M3DDPG)	Implicit; teammate-action channel	Worst-case critic target; one-step MAAL approximation	(5.1)
Regularization (SA-MDP, ERNIE, MIR3, RADIAL-RL)	ℓ_p ball on observations / history	Sensitivity penalty: KL, Lipschitz, mutual information, or certified Q-margin	(5.2)–(5.6)
Distributionally robust (DrIGM)	TV or ρ-contamination ball over transition kernels	Robust joint Bellman target; DrIGM keeps IGM	(5.7)
Sim-to-real (ADR)	Plausible band of simulator parameters ξ	Minimax vs. a constrained Nature Player	(5.8)

5.8. Comparison

Table 5.1 summarizes the review. Two points carry into the next section. First, the mechanisms differ, but the objective is the same in every case: one policy that is optimal for the *worst* allowed disturbance. Second, the uncertainty set, together with its radius ϵ or ρ, its divergence, or its parameter band, is a hyperparameter that is fixed before training and never updated.

5.9. Research Gap: What These Methods Do Not Consider

[**Synthesis**] Taking the review as a whole, four points lie outside the scope of every method.

1. **The perturbation intensity is assumed, not inferred.** Every method fixes the bound (ϵ, ρ, κ) by hand. None estimates from deployment data how large the disturbance actually is. The policy is therefore either too conservative, when the true disturbance is smaller than assumed, or unprotected, when it is larger.

2. **A single fixed policy.** Each method returns one worst-case policy. None keeps several policies and *selects* among them based on the current condition, so it cannot act boldly when there is no disturbance and cautiously only when attacked.

3. **The nominal-performance cost is not addressed.** At $\delta = 0$ the worst-case policy still pays for a case that is not occurring (RQ3). No method returns to nominal behaviour when the disturbance is absent.
4. **The disturbance is a fixed set, not a player.** Adversarial training uses an adversary only as a computational device. It does not treat the protagonist–disturbance interaction as a game to be solved for its equilibrium, and it does not ask whether that equilibrium is a Nash point that could be reached and reused.

Implications for the proposed method (R1, R2). The method proposed in Chapter 10 targets the first three points. It trains the execution critic against the M3DDPG worst-case teammate action of Eq. (5.1), and a second variational encoder infers the perturbation radius; its full posterior conditions both the execution policy and the critic, VariBAD-style, so conservatism is scaled to what is inferred and behaviour returns to nominal when none is present (RQ3); the exploration stream is left unperturbed. A representation-learned change-point detector separates a bounded perturbation from a genuine task switch. The fourth point is only partly closed: the inner minimization is taken as one linearized MAAL step, a local rather than a global worst case. The design and its evaluation are future work.

6. Adaptive Multi-Agent Reinforcement Learning

Chapter 5 ended on one shared failure mode: every robust method fails when the real disturbance exceeds the uncertainty set it was trained against. This chapter starts from that failure. It reviews the methods that assume the disturbance *will* exceed any fixed set, because the environment keeps changing after deployment, and that track the change online instead of preparing for its worst case. Together with Chapter 7, it answers RQ4: what mechanisms let a MARL policy adapt online when the environment or the other agents change after deployment, and what do they trade off.

6.1. Why Robustness Is Not Enough

A robust policy is the solution to a max–min problem over an uncertainty set $\mathcal{P}$ that is fixed before training (Section 5.1). Its guarantee is a floor on return *for every model in* $\mathcal{P}$. That guarantee is silent about any model outside $\mathcal{P}$.

Now suppose the deployment environment is not a single unknown model but a *sequence* $\{(P_t, r_t)\}_{t\geq 1}$ that drifts. Traffic demand rises through the morning, a motor wears out, and, in the multi-agent case, every other agent keeps updating its own policy, so the effective kernel $\tilde{P}_{i,t}$ of Section 4.7 moves between episodes. Whatever radius ρ was chosen for $\mathcal{P}$, a drift that grows without bound will eventually carry (P_t, r_t) outside it, and from that point the robust guarantee no longer applies. Enlarging ρ to cover all future drift has two problems. If the drift is truly unbounded, no finite ρ is enough. And a large ρ makes the policy so conservative that it gives up most of its nominal performance from the first step, to protect against conditions that will not occur for hours.

The alternative is to stop treating the change as a bounded set to be covered in advance, and to treat it instead as a process to be tracked. This is the distinction the thesis is built on:

> *Robustness protects against deviation: a bounded, static departure from training conditions, prepared for before deployment. Adaptability tracks*

evolution: an unbounded but structured departure that accumulates after deployment, followed online.

Adaptability is not free. It moves the cost from training-time conservatism to deployment-time tracking. The learner needs data from the new regime before it can respond, so there is always a reaction lag. It also needs the change to have some structure, such as slow drift, occasional jumps, or a finite set of recurring regimes, so that a bounded amount of recent data is informative about the present. The rest of this chapter is organized by the structural assumption that each family makes and how it uses it.

6.2. Non-Stationary Markov Decision Processes

A non-stationary MDP replaces the fixed pair (P, r) with a time-indexed sequence $\{(P_t, r_t)\}_{t=1}^{T}$ over a horizon T. If there is no restriction on how fast the sequence changes, no learner can do better than one that ignores its past. The literature therefore bounds the total change by a *variation budget*

$$B_T \;=\; \sum_{t=1}^{T-1} \sup_{s,a} \Big(\big\| P_{t+1}(\cdot \mid s,a) - P_t(\cdot \mid s,a) \big\|_1 + \big| r_{t+1}(s,a) - r_t(s,a) \big| \Big),$$

which caps the accumulated drift but not its shape [48]. The performance target is stated against the *dynamic* optimum. Let $\pi_t^\star$ be optimal for the instantaneous MDP (P_t, r_t) and $V_t^\star$ its value. A learner that plays π_t at step t incurs *dynamic regret*

$$\mathrm{DR}(T) \;=\; \sum_{t=1}^{T} \big(V_t^\star - V_t^{\pi_t} \big),$$

or, reported per step, the *average dynamic suboptimality gap* $\frac{1}{T}\sum_t (V_t^\star - V_t^{\pi_t})$.

Dynamic regret is a harder target than the *static* regret of stationary RL, which compares only against the best *fixed* policy in hindsight. This is what makes non-stationary RL a separate problem rather than a special case. In a stationary MDP the best fixed policy is optimal, so static and dynamic regret are equal. Under drift, the best fixed policy can be far from $\pi_t^\star$ at every t, so a learner with sublinear static regret can still have linear dynamic regret. A sublinear dynamic-regret bound is therefore only possible with a restriction on how fast the environment moves, and the bound always states that restriction. For the slowly varying class, bounds scale as $\tilde{O}\big(B_T^{1/3} T^{2/3}\big)$ or $\tilde{O}\big(B_T^{1/4} T^{3/4}\big)$ in the variation budget. For the piecewise-stationary class, they scale as $\tilde{O}(\sqrt{L\,T})$ in the number of changes L. In both cases the regret rate gets worse as the environment moves faster, and it becomes trivial, $O(T)$, once

Table 6.1. – Structure classes for a non-stationary MDP, and the adaptive family that assumes each.

Class	Assumption on $\{(P_t, r_t)\}$	Exploited by
Slowly varying	Per-step change bounded, $\|P_{t+1} - P_t\|_1 \le \delta$; $B_T \approx \delta T$	Sliding-window estimation (§6.5)
Piecewise stationary	Constant except at $L \ll T$ abrupt change points	Change-point detection (§6.4)
Switching	$(P_t, r_t) \in \{M_1, \dots, M_K\}$, a finite, possibly recurring set of regimes	Context detection and policy reuse (§6.4)
Dynamic-parameter	$(P_t, r_t) = (P, r)(\cdot\,; \theta_t)$ with θ_t low-dimensional and itself evolving	Structured / factored adaptation (§6.6)

B_T or L is a constant fraction of T: no learner can track a target that jumps every few steps.

A concrete example makes these objects usable. Consider a two-signal traffic-light task whose arrival rates rise linearly through a rush hour and fall afterwards, so P_t drifts smoothly with $\|P_{t+1} - P_t\|_1 \approx \delta$ for a small δ and $B_T \approx \delta T$. The dynamic optimum $\pi_t^\star$ re-times the lights continuously. The best fixed policy uses one compromise timing and is suboptimal all day. A sliding-window learner (Section 6.5) with a suitable window tracks $\pi_t^\star$ up to a lag set by the window length, and its dynamic regret grows sublinearly in T because δ is small.

The families below do not target an arbitrary bounded-B_T sequence; each assumes more structure. Table 6.1 lists the four structure classes the literature uses and the family that exploits each.

6.3. A Taxonomy of Adaptive MARL

Adaptive methods answer one question, "the environment I am estimating is not the one I will act in, so which data should I trust?", in three ways (Figure 6.1). *Change-point detection* trusts all data since the last detected change and discards everything before it. *Sliding-window estimation* trusts a fixed window of recent data and continuously forgets the rest. *Structured adaptation* trusts a low-dimensional summary of the change and re-estimates only that. Meta-reinforcement learning, developed separately in Chapter 7, adds a fourth answer: trust a learned adaptation procedure that was trained across many related environments.

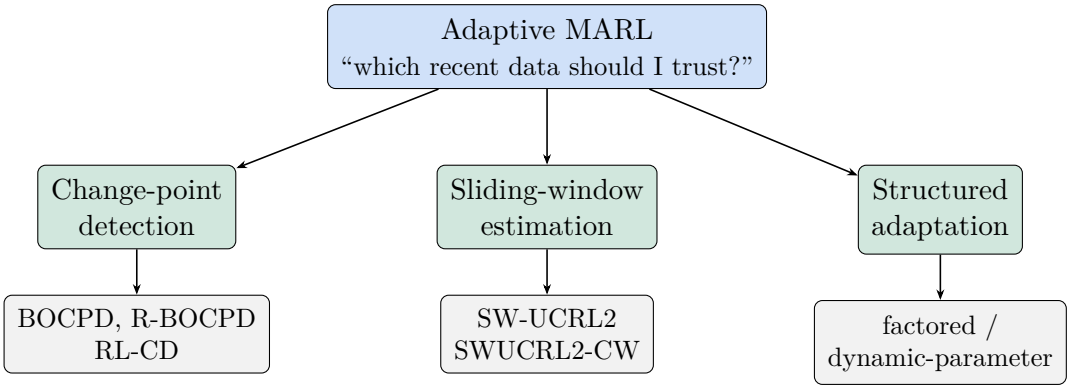

Figure 6.1. – The three adaptive MARL families of this chapter, by the structural assumption each makes about the non-stationarity (Table 6.1).

6.4. Family I: Change-Point Detection

Problem. The environment is stationary for long stretches and then changes abruptly, either once or among a finite set of regimes.

Why conventional MARL fails. A learner that averages over its whole history mixes pre-change and post-change data, so its estimate of the current MDP is a stale blend and its policy lags every change indefinitely.

Solution principle. Run a statistical test on the data stream. When it flags a change, either *reset* the learner's statistics so it re-estimates the new regime from scratch, or, if regimes recur, *recognize* the regime and switch to the policy already learned for it.

Algorithm family. A base RL learner with regret guarantees, plus a detector. The base learner is almost always UCRL2.

Prerequisite: UCRL2

UCRL2 [9] is the optimism-in-the-face-of-uncertainty algorithm for stationary MDPs, and the base learner almost every method in this chapter is built on. It proceeds in episodes. Within an episode it holds a policy fixed; at each episode boundary it recomputes, from the visit counts $N_t(s,a)$ accumulated so far, an ℓ_1 confidence set for each $P(\cdot \mid s,a)$ and an interval for each $r(s,a)$, both with radius

$\mathcal{O}\big(\sqrt{\log(t/\delta)/\max(1, N_t(s,a))}\big)$. It then forms the set $\mathcal{M}_t$ of all MDPs consistent with those confidence sets and computes the policy that is optimal for the *most favourable* MDP in $\mathcal{M}_t$, $\tilde{\pi}_t = \arg\max_\pi \max_{M \in \mathcal{M}_t} V_M^\pi$, by *extended value iteration*, which folds the inner maximization over transition kernels into the Bellman backup by shifting probability mass toward the highest-value successor within each confidence set. The episode ends when the visit count of some (s,a) doubles, which bounds the number of policy switches at $\mathcal{O}(SA \log T)$. This optimism drives exploration: a poorly known (s,a) has a wide confidence set, hence a high optimistic value, hence gets visited. The regret is $\tilde{\mathcal{O}}(DS\sqrt{AT})$, where the *diameter* D is the largest expected time to move between any two states under the best policy; D enters because the optimistic MDP can be traversed no faster than the true one, and a large D slows the propagation of value through extended value iteration. The adaptive methods below reuse this machinery unchanged and alter only which data feeds the confidence sets: everything since the last detected change (change-point detection) or everything in a sliding window (Section 6.5). The diameter term, harmless in a stationary MDP, is the quantity that the sliding-window methods have to fight.

Representative: BOCPD

Bayesian Online Change-Point Detection [49] maintains a posterior over the *run length* $r_t \in \{0, 1, 2, \dots\}$, the number of steps since the last change. Two ingredients define it. The *hazard function* $H(r)$ is the prior probability that a run of length r ends at the next step; a constant hazard $H(r) = \lambda$ corresponds to geometrically distributed inter-change intervals with mean $1/\lambda$. The *run predictive* $p(x_t \mid x_{t-r:t-1})$ models the observations within a stationary run, and is taken from an exponential family so that the run parameters integrate out in closed form and each candidate run carries only a set of conjugate hyperparameters updated by accumulation (Appendix D). The joint recursion over $(r_t, x_{1:t})$ then splits into a *growth* term, the run continues,

$$p(r_t = r_{t-1} + 1,\, x_{1:t}) \;=\; p(r_{t-1},\, x_{1:t-1})\,\big(1 - H(r_{t-1})\big)\,p\big(x_t \mid x_{t-r_{t-1}:t-1}\big),$$

and a *change-point* term that collapses all run lengths to zero,

$$p(r_t = 0,\, x_{1:t}) \;=\; \sum_{r_{t-1}} p(r_{t-1},\, x_{1:t-1})\,H(r_{t-1})\,p\big(x_t \mid x_{t-r_{t-1}:t-1}\big).$$

Normalizing gives $p(r_t \mid x_{1:t})$ in one pass; the maximum-a-posteriori run length locates the most recent change, and truncating the run-length support keeps the update constant-time per step. In an RL setting the observations x_t are the transition and reward samples, so a change in P_t or r_t shows up as a drop in the run predictive and pushes posterior mass onto $r_t = 0$.

Assumption: piecewise-stationary regimes with exponential-family observations and, implicitly, a geometric prior on inter-change intervals. *Failure mode:* BOCPD detects a change but supplies no adaptation of its own, it must be paired with a learner that acts on the signal; a mismatched hazard or likelihood model either delays detection or raises false alarms, and each false alarm throws away useful data.

Restart integration: R-BOCPD

Restarted BOCPD for non-stationary MDPs [50] closes BOCPD's open end. It runs a regret-optimal base learner (UCRL2-style) and a change detector on the learner's own transition and reward estimates in parallel; when the detector fires, the learner's statistics are restarted so it re-estimates the new regime, while previously learned regime models can be retained for reuse. The resulting dynamic-regret bound scales with the *number of changes* L rather than with a continuous variation budget, which is the right dependence when the environment really is piecewise stationary.

Representative: RL-CD

Reinforcement Learning with Context Detection [51] targets the *switching* class directly. It maintains a library of partial environment models, each with a running measure of how well it has predicted recent transitions. At each step the best-predicting model is made active and its policy is used; if no model predicts the recent data well enough, a new model is created and trained. When a previously seen regime recurs, its model reactivates and its policy is reused with no relearning.

Assumption: the environment switches among a finite, possibly recurring set of stationary contexts. *Failure mode:* continuously drifting dynamics never match a stored context well and force endless model creation; the prediction-quality threshold that decides "new context" is a sensitive hyperparameter.

Takeaway. Change-point detection is suited to change that is rare and sharp. It adapts in two steps: detect, then reset or reuse. Its dynamic regret depends on the number of changes. It fails on gradual drift, where there is no single point to detect, and its detector is only as good as its assumed observation model.

6.5. Family II: Sliding-Window Estimation

Problem. The environment drifts continuously, so there is no change point to detect, but old observations still become misleading.

Why conventional MARL fails. For the same reason as before: a full-history average is dominated by stale data. Now there is also no event to tell a detector when to reset.

Solution principle. Weight recent experience more heavily. In the simplest form, estimate the current MDP from only the last W transitions and discard everything older. Exponential weighting is a smooth version of the same idea.

Algorithm family. UCRL2 with all statistics restricted to a window. The design question is the window size, and the central trade-off is between bias and variance, which here reads as a trade-off between tracking speed and estimation accuracy: a short window tracks a fast drift but estimates each MDP from little data, a long window estimates accurately but lags the drift. The optimal window $W^\star \sim (T/B_T)^{2/3}$-order depends on the horizon T and the variation budget B_T, neither of which is usually known in advance.

Representative: SW-UCRL2

Sliding-Window UCRL2 [52] runs UCRL2 with every count restricted to the most recent W steps: $N_t(s,a)$ becomes the number of visits in $(t-W,t]$, and $\hat{p}_t$, $\hat{r}_t$ are the corresponding windowed empirical estimates. Optimistic planning is unchanged. The window introduces a bias, the data mixes up to W steps of drift, of order $W \cdot (B_T/T)$, and a variance of order $1/\sqrt{W}$ from the reduced sample; balancing the two gives $W^\star$ of order $(T/B_T)^{2/3}$ and a dynamic-regret bound of order $\tilde{O}\big(D\,S\,\sqrt{A}\,B_T^{1/3}T^{2/3}\big)$, sublinear whenever $B_T = o(T)$. The bound exposes one structural weakness. A window that straddles a change contains transitions from two different MDPs, and the optimistic MDP built from that blend can have a *diameter* D far larger than either true MDP, because a state that is easy to reach in each regime separately may look almost unreachable under the averaged kernel. Since the regret scales with D, and since extended value iteration converges slowly (or, with an unbounded optimistic diameter, not at all) when D is large, the mixed window degrades planning at exactly the moments adaptation matters most.

Representative: SWUCRL2-CW

Sliding-Window UCRL2 with Confidence Widening [53] fixes the diameter problem by deliberately *widening* the transition confidence region. Keeping the window, it enlarges the ℓ_1 ball around the windowed transition estimate $\hat{p}_t$ by a fixed extra amount η:

$$H_{p,t}(s,a)(\eta) \;=\; \Big\{ \dot{p}(\cdot \mid s,a) \in \Delta_{\mathcal{S}} \;:\; \big\|\dot{p}(\cdot \mid s,a) - \hat{p}_t(\cdot \mid s,a)\big\|_1 \le \mathsf{rad}_{p,t}(s,a) + \eta \Big\},$$

where $\mathsf{rad}_{p,t}(s,a) \to 0$ as the windowed visit count grows, while the floor η does not. The floor keeps the optimistic MDP's diameter bounded even across a change, because it guarantees a minimum level of exploration-inducing optimism regardless of how confident the windowed estimate has become: with η fixed, extended value iteration always has enough transition slack to route value through any state, so the effective optimistic diameter stays $O(1/\eta)$ rather than blowing up. The price is paid in the regret bound, which picks up an additive term growing in η, so the two effects trade off: too small an η leaves the diameter problem unsolved, too large an η inflates regret on its own, and the regret-optimal $\eta \sim (B_T/T)^{1/4}$ again needs the variation budget in advance. With η tuned, the dynamic-regret rate matches SW-UCRL2's $\tilde{O}(B_T^{1/4} T^{3/4})$ order but without the unbounded-diameter failure case.

The object $H_{p,t}(s,a)(\eta)$ is worth stating precisely, because Chapter 8 shows that freezing its two moving parts, the re-centring on $\hat{p}_t$ and the shrinking $\mathsf{rad}_{p,t}$, turns it into exactly the fixed ambiguity set of a distributionally robust method.

Takeaway. Sliding-window estimation is suited to gradual drift. It adapts continuously, with no detector, and its dynamic regret depends on the variation budget. It has two parameters, the window size and the confidence widening. Both trade tracking speed against regret, and both can be tuned optimally only with prior knowledge of the horizon and the drift rate.

6.6. Family III: Structured and Factored Adaptation

Problem. Treating every change as an unstructured shift in a full transition table wastes data. If the environment varies along only a few interpretable axes, most of that table is irrelevant.

Solution principle. Assume the non-stationarity is low-dimensional. The environment is a *dynamic-parameter* MDP $(P, r)(\cdot\,; \theta_t)$ with a small θ_t, or it factors into components that change independently. Adaptation then estimates only θ_t, or only the factor that moved.

This principle is well developed in single-agent RL but thin in MARL, and this thesis does not overstate it. The clearest multi-agent case treats the drift of *other agents' policies* as the low-dimensional parameter. Al-Shedivat, Bansal, Burda, *et al.* [54] track a competitor's changing strategy with a meta-learned update; here θ_t stands for opponent behaviour, which connects to the meta-RL mechanisms of Chapter 7. Factored and causal variants, which ask *which* state factor's dynamics changed, exist for single-agent settings. The multi-agent version of that question, attributing an observed change to a specific teammate rather than to the physical environment, has no general answer in the literature. Chapters 8 and 9 return to it as an open problem.

Takeaway. When the non-stationarity is truly low-dimensional, structured adaptation is the most data-efficient of the three families, because it re-estimates a few numbers rather than a whole model. Its assumption, that the change lies in a known low-dimensional space, is also the strongest of the three, and in MARL it is mostly unverified.

6.7. The MARL-Native Case: Adapting to Other Agents

The three families above are single-agent adaptation mechanisms applied to a multi-agent environment. There is also a form of adaptation that is specific to MARL and older than any of them. It targets the internal non-stationarity of Section 4.7 directly: the other agents' policies are the part of the environment that is changing, so the idea is to model them and respond.

Opponent and teammate modelling keeps an explicit estimate $\hat{\pi}^{-i}$ of the other agents' policies, built from their observed actions, and best-responds to it. The classical form is fictitious play, where each agent best-responds to the empirical action frequencies of the others. The deep form learns $\hat{\pi}^{-i}$ as a neural network conditioned on the interaction history and feeds it to the acting agent as an extra input. Read as adaptation, this is a change-point-free, structured tracker whose latent parameter (Section 6.6) is opponent behaviour and whose "window" is the recency weighting of the frequency estimate. It inherits the same trade-off: weight recent actions heavily

Table 6.2. – Adaptive MARL taxonomy. All three families re-estimate the current environment from a chosen subset of recent data; they differ in how that subset is selected and in what must be known to select it well.

Family	Change type	Adaptation mechanism	Information required	Failure mode
Change-point detection (BOCPD, R-BOCPD, RL-CD)	Piecewise stationary or switching	Detect a change, then reset the learner or reuse a stored context policy	Observation model for the detector; hazard rate or a context-fit threshold	Gradual drift (no point to detect); detector model mismatch delays or false-triggers
Sliding-window estimation (SW-UCRL2, SWUCRL2-CW)	Slowly varying, bounded B_T	Restrict UCRL2's confidence sets to a window; widen them by a floor η	Window size W, and η; both optimal only given T and B_T	Window straddling a change inflates the diameter; wrong W or η inflates regret
Structured / factored adaptation	Dynamic-parameter; factored change	Estimate only the low-dimensional θ_t or the factor that moved	The parameterization or factor structure of the change	Change not actually low-dimensional; in MARL, source attribution is unsolved

and track a switching opponent quickly but noisily, weight them lightly and lag a drifting one.

Two limitations matter for the rest of the thesis. First, opponent modelling assumes the other agents' non-stationarity is the *only* non-stationarity; a simultaneous change in the physical environment is misattributed to the opponents. Second, it estimates *what* the other agents are doing, not *which* of them changed or *why*, so it does not solve the credit-assignment problem that Chapters 8 and 9 identify. It is the adaptive counterpart of M3DDPG's worst-case teammate model (Section 5.4): one tracks the teammate, the other bounds it, and neither attributes a change to a source.

6.8. Comparison and Takeaway

Table 6.2 lays the three families on one set of axes. Read against Table 5.1, the contrast with robust MARL is systematic rather than incidental:

- *Objective.* Robust MARL maximizes worst-case return over a fixed set; adaptive MARL minimizes dynamic regret against a moving optimum.
- *Timing.* Robust MARL acts before deployment; adaptive MARL acts after, and therefore always with a reaction lag.

- *Assumption.* Robust MARL assumes the departure is bounded and static; adaptive MARL assumes it is unbounded over time but structured (piecewise, slow, or low-dimensional).
- *Mechanism.* Robust MARL uses minimax training or a regularizer or a worst-case model; adaptive MARL uses a detector-plus-reset, a sliding window, or a low-dimensional estimator.
- *Failure mode.* Robust MARL fails when the disturbance exceeds its set; adaptive MARL fails when the change is faster than its reaction time or lacks the assumed structure.

The two failure modes are complementary: a robust policy handles a sharp bounded shock that an adaptive learner would react to too slowly, and an adaptive learner handles a slow drift that a robust policy's fixed set cannot contain. Chapter 8 makes this complementarity precise, and shows that on one axis, the shape of the model set the learner carries, the two families are running almost the same construction with one difference: whether the set is re-estimated as data arrives.

Implications for the proposed method (R3, R5). A method that must survive unbounded drift cannot rely on a radius fixed before deployment; it needs an online detector that flags a regime change and triggers re-estimation, and — because the change-attribution problem of Section 8.3.3 shows each agent's environment can move independently — that detector and its reset must act per-agent. Chapter 10 adopts a representation-learned detector on the context embeddings for exactly this. Before that, Chapter 7 develops the third adaptation mechanism, a learned adaptation procedure, that Table 6.2 only points to.

7. Meta-Reinforcement Learning as an Adaptation Mechanism

Chapter 6 treated adaptation as something added to an ordinary reinforcement-learning loop: detect a change and re-estimate, or keep a sliding window and re-estimate. Both re-estimate the environment online, from scratch, using only data from the current deployment. Ordinary reinforcement learning is adaptable in exactly this sense whenever new data can be collected at deployment, but it is sample-hungry: reaching a good policy this way can cost on the order of millions of environment steps, far more interaction than a deployed system can usually afford to spend relearning after every change. Meta-reinforcement learning targets that cost directly. Rather than relearn from scratch each time, it learns the *adaptation procedure itself*, offline, across a distribution of related tasks, so that a new task, or a new drift in the current one, can be handled from a handful of samples at deployment time, with no retraining from scratch. This chapter completes the answer to RQ4 begun in Chapter 6, and should be read against it. The two chapters together cover three adaptation mechanisms, and Table 7.1 at the end places all three on the same axes used for the robust and adaptive taxonomies.

The chapter is deliberately narrow. Meta-RL is a large field, much of it concerned with representation learning, optimizer design, and exploration for their own sake. Only the part whose *adaptation mechanism* relates to tracking task or environment change is developed here, following the inclusion criteria of Section 2.1.1. The remaining methods, such as attention and memory backbones, weight-space plasticity, discriminative and direct-regression task encoders, and meta-gradient estimators, are listed in Appendix A.

7.1. Meta-RL as Rapid Adaptation

Setup. A meta-RL problem is a distribution $p(\mathcal{M})$ over MDPs ("tasks"), where task $\mathcal{M}_i = (\mathcal{S}, \mathcal{A}, P_i, r_i, \gamma)$ shares the state and action spaces but has its own dynamics and reward. During *meta-training* the learner interacts with many tasks drawn from $p(\mathcal{M})$; at *meta-test* it is given a new task from $p(\mathcal{M})$ and must reach high return within a small budget of interaction. The object being learned is an *adaptation*

function $f\colon \tau_{1:k} \mapsto \pi$ that maps a short interaction history on the new task to a policy for it. What varies across the field is the form of f and what it updates.

Meta-RL as a Bayes-adaptive MDP. If the task identity is treated as an unobserved parameter w with prior $p(w)$ induced by $p(\mathcal{M})$, a meta-RL problem is a POMDP whose hidden state is w, and the equivalent belief MDP is the *Bayes-adaptive MDP* (BAMDP). Its state is the pair (s_t, b_t) where $b_t(w) = p(w \mid \tau_{1:t})$ is the posterior over the task given the history, updated by Bayes' rule after each transition,

$$b_{t+1}(w) \;\propto\; b_t(w)\, P_w(s_{t+1} \mid s_t, a_t)\,,$$

and its transitions carry the belief forward alongside the physical state. The BAMDP-optimal policy $\pi^*(a \mid s_t, b_t)$ is *Bayes-optimal*: it takes an exploratory action exactly when the expected value of the task information that action reveals, integrated over the rest of the episode, exceeds its immediate cost. This is the precise sense in which meta-RL "learns to explore". Exact BAMDP planning is intractable because the belief lives in an infinite-dimensional simplex; every method in this chapter is a tractable approximation, and they differ in *what stands in for* b_t: an adapted parameter vector (Section 7.2), a recurrent hidden state that is never explicitly a belief (Section 7.3), or an explicit low-dimensional latent (Section 7.4).

Few-shot and many-shot. The adaptation budget k separates two regimes. *Few-shot* meta-RL is given only a handful of episodes on the new task; the meta-learner must therefore encode a strong prior over $p(\mathcal{M})$ and spend its few episodes resolving *which* task it faces, not learning the task from scratch. This is the regime that matches a sudden task change at deployment and is the one this chapter develops. *Many-shot* meta-RL is given a long lifetime, so it can afford to learn more about each task online; it shades into continual RL, and its relevance to this thesis is to slow drift rather than abrupt change. The distinction is not just quantitative: few-shot methods are judged on *how fast* return rises in the first few episodes, many-shot methods on asymptotic return, so a method tuned for one regime is usually mediocre in the other.

Relation to Chapter 6. Meta-RL differs from change-point and sliding-window methods in *where* the adaptation knowledge comes from. Those methods build their model of the current environment entirely from deployment data, using a hand-specified update rule (a detector, a window). Meta-RL learns the update rule itself from a distribution of related tasks, and then applies it with very little deployment data. The price is a meta-training phase and the assumption that meta-test tasks resemble meta-training ones; the benefit is adaptation from a handful of transitions

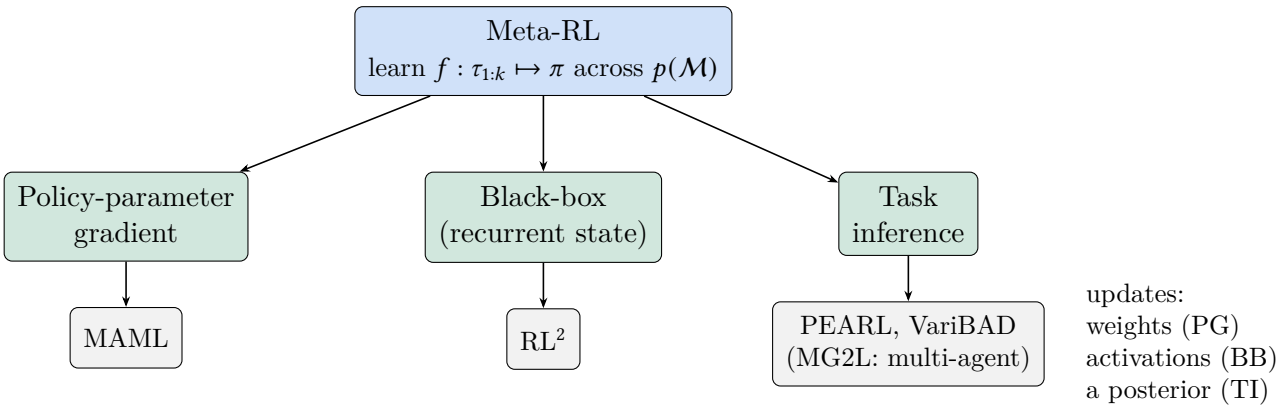

Figure 7.1. – The three meta-RL families, by what the adaptation function updates at deployment.

rather than the hundreds a windowed re-estimator needs. The two are complementary, not competing: a meta-learned prior gives a fast initial response, an online re-estimator gives an asymptotically correct one.

Three families. By what the adaptation function updates (Figure 7.1): the *policy parameters*, through a gradient step (Section 7.2); the *recurrent state* of a fixed-weight network, through the forward pass (Section 7.3); or an explicit *latent task variable* that the policy is conditioned on (Section 7.4).

7.2. Policy-Parameter Adaptation

Principle. Learn a policy *initialization* from which a few ordinary gradient steps, on data from a new task, reach a good task-specific policy.

Representative: MAML. Model-Agnostic Meta-Learning [18] learns an initialization θ such that, for any task $\mathcal{M}_i$, the *inner-loop* adapted parameters

$$\theta_i' \;=\; \theta + \alpha \nabla_\theta J_i(\theta), \qquad J_i(\theta) = \mathbb{E}_{\tau\sim\pi_\theta,\,\mathcal{M}_i}\Big[\textstyle\sum_t \gamma^t r_i(s_t,a_t)\Big],$$

achieve high return, and the *outer-loop* objective optimizes the post-adaptation performance of the initialization,

$$\max_\theta \; \mathbb{E}_{\mathcal{M}_i\sim p(\mathcal{M})}\big[\, J_i(\theta_i')\,\big], \qquad \theta \leftarrow \theta + \beta\, \nabla_\theta\, \mathbb{E}_i\big[J_i(\theta_i')\big].$$

The outer gradient differentiates through the inner step, so it contains a second-order term $\nabla_\theta \theta_i' = I + \alpha \nabla_\theta^2 J_i(\theta)$. At meta-test the adaptation *is* the inner step: collect a little data on the new task, take one or a few gradient steps from θ, act. MAML is "model-agnostic" because f is plain gradient descent and the adapted object is an ordinary policy, so it composes with any policy-gradient backbone and, through CTDE, with a centralized critic (Section 7.5).

Assumption: every parameter benefits from inner-loop adaptation, a few steps suffice, and $p(\mathcal{M})$ is representative of meta-test tasks. *Failure mode:* full-parameter second-order adaptation is computationally heavy and high-variance; the inner step needs fresh on-policy rollouts from the new task, which is the opposite of sample-efficient; and a task far from $p(\mathcal{M})$ is not reachable in a few steps. A sub-literature addresses the estimator rather than the mechanism: first-order approximations drop the Hessian term, ProMP [55] corrects the resulting bias in the pre-adaptation gradient, E-MAML [56] adds credit for exploratory pre-adaptation trajectories, and off-policy meta-gradient estimators reuse replay data. These are catalogued in Appendix A; none changes what is updated at deployment.

7.3. Black-Box Adaptation

Principle. Make the adaptation function a *recurrent network with fixed weights*. The history is fed in step by step; the hidden state accumulates whatever is useful about the current task; the policy reads the hidden state. Nothing is updated by gradient at deployment.

Representative: RL^2. RL^2 [19], and independently L2RL [57], organize meta-training into *trials*, each a fixed number of consecutive episodes of one sampled task. A recurrent policy $\pi_\theta(a_t \mid h_t)$ receives at each step the usual observation together with the previous action, previous reward, and an episode-termination flag, and its hidden state h_t is carried across episode boundaries *within* a trial, reset only when a new task begins. The outer objective is the ordinary policy-gradient objective on the total return over the whole trial,

$$\max_\theta \; \mathbb{E}_{\mathcal{M}_i \sim p(\mathcal{M})} \Big[\mathbb{E}\big[\textstyle\sum_{\text{trial}} \gamma^t r_i(s_t, a_t) \big] \Big].$$

Because the reward from later episodes depends on what the hidden state learned in earlier ones, gradient ascent on this objective forces the recurrent dynamics to implement, inside the forward pass, something like exploration in early episodes and exploitation later. The learned reinforcement-learning algorithm is thus *implicit* in θ,

and h_t is an implicit, never-normalized stand-in for the BAMDP belief. Adaptation at meta-test is the forward pass alone: no gradient step, no explicit inference.

Assumption: the task-relevant content of an arbitrarily long history compresses into a fixed-width hidden state, and meta-test tasks resemble meta-training tasks. *Failure mode:* the hidden state is a bottleneck; credit assignment across a long multi-episode history is hard for a recurrent network; performance falls off sharply for tasks outside $p(\mathcal{M})$, and unlike MAML there is no test-time gradient step to recover.

7.4. Task-Inference Adaptation

Principle. Maintain an explicit estimate of *which task* is in play, a latent variable z inferred from the history, and condition the policy $\pi(a \mid s, z)$ on it. This is a direct, tractable approximation to the BAMDP belief of Section 7.1: separate "infer the task" from "act given the task".

Grounding: approximate information states. The Approximate Information State framework [58] justifies replacing the exact belief with a learned compression $z_t = \sigma(\tau_{1:t})$ of the history. It requires z_t to be approximately *predictively sufficient*: the expected reward and the distribution of the next z must be recoverable from (z_t, a_t) up to small errors ε_r and ε_P. A contraction argument then bounds the value lost by planning on z_t instead of the true belief by $O\big((\varepsilon_r + \varepsilon_P)/(1-\gamma)\big)$. This is what licenses the practice below: the encoder output need not be the exact posterior, only a compression rich enough to predict reward and its own dynamics. The two design choices are the encoder architecture and its training signal.

Representative: PEARL. PEARL [20] encodes a *set* of recent transitions $c = \{(s_j, a_j, r_j, s'_j)\}_{j=1}^{N}$ into a latent z with an encoder $q_\phi(z \mid c) \propto \prod_j \Psi_\phi(z \mid s_j, a_j, r_j, s'_j)$, a product of per-transition Gaussian factors. The product form makes the encoder permutation-invariant and able to take a variable number of transitions, and makes the posterior sharpen automatically as N grows, since multiplying Gaussians shrinks the variance. The actor $\pi_\psi(a \mid s, z)$ and critic $Q_\omega(s, a, z)$ are trained off-policy with SAC, conditioned on z; the encoder is trained through the critic's Bellman error plus a $\mathrm{KL}\big(q_\phi(z \mid c) \,\|\, \mathcal{N}(0, I)\big)$ term that acts as an information bottleneck on the context. Exploration uses *posterior sampling*: draw one $z \sim q_\phi(z \mid c)$, act under $\pi_\psi(\cdot \mid \cdot, z)$ for a trajectory, add the new transitions to c, and repeat, so the task posterior contracts over a handful of trajectories. Crucially, c is drawn from recently collected on-policy data while the SAC updates use the full replay buffer, and this decoupling is what lets task inference sit on top of an off-policy, sample-efficient learner.

Assumption: the task is identified by its transition and reward statistics, and the context set is exchangeable. *Failure mode:* the encoder's training signal still flows through the critic, so a poor critic degrades inference; conditioning the policy on a single sampled z at action time discards the uncertainty the posterior was meant to carry, so PEARL is not Bayes-optimal; and inference is only as good as the coverage of $p(\mathcal{M})$.

Contrast: VariBAD. VariBAD [59] keeps the latent-task idea but changes both design choices. The encoder is a *sequential* RNN maintaining an approximate posterior $q_\phi(m \mid \tau_{1:t})$ over a latent task embedding m, trained with a variational lower bound on the probability of the trajectory,

$$\mathcal{L}_t = \mathbb{E}_{q_\phi(m|\tau_{1:t})}\Big[\textstyle\sum_k \log p_\theta\big(r_k, s_{k+1} \mid s_k, a_k, m\big)\Big] - \mathrm{KL}\big(q_\phi(m \mid \tau_{1:t}) \,\|\, p(m)\big),$$

where the decoder must reconstruct rewards and transitions for *all* timesteps $k = 1, \ldots, H$, past and future, from a belief formed at time t. This forces the belief to be predictive rather than merely descriptive. The policy is conditioned on the *entire* posterior $q_\phi(m \mid \tau_{1:t})$ (its parameters, not a sample), which is what lets it approximate the Bayes-optimal policy of Section 7.1 and explore specifically to shrink task uncertainty. The costs are that training is on-policy and that the reconstruction decoder over the full horizon is heavier than PEARL's set encoder. The pair spans the family's design space cleanly: PEARL is off-policy, set-encoded, point-conditioned; VariBAD is on-policy, sequence-encoded, belief-conditioned.

The rest of the design space. The two exemplars fix a variational objective and a probabilistic latent, but task inference does not require either. *Discriminative* encoders replace the reconstruction loss with a contrastive one, pulling together the embeddings of transitions from the same task and pushing apart those from different tasks; this drops the decoder entirely and is more robust to high-dimensional observations, at the cost of the calibrated uncertainty a variational posterior provides. *Direct-regression* methods skip the belief and regress from a short context straight to a task-specific quantity, a policy embedding scored by a value function, or a linear policy layer, giving adaptation in a single forward pass but no notion of how uncertain that estimate is. *Exchangeable-process* models build the permutation invariance PEARL approximates into the architecture exactly, via a stochastic process that is exchangeable by construction. A further off-policy variant, ELUE [60], decouples encoder training from the actor–critic loop with a belief-based conditional autoencoder on raw transitions, addressing PEARL's dependence on the critic. All of these keep the family's defining move, history → latent task → conditioned policy, and change only the encoder's objective or output type; Appendix A lists them method by method.

7.5. Multi-Agent Meta-RL

The single-agent mechanisms above transfer to MARL through CTDE (Section 4.8), but each family raises a distinct question about *what is shared* across agents. For **policy-parameter gradient**, the natural design meta-learns one shared initialization θ and adapts it per agent with a centralized critic in the outer loop; the open choice is whether the inner adaptation is also shared (all agents take the same gradient step, exploiting symmetry) or per-agent (each adapts to its own role). For **black-box**, a centralized recurrent critic stabilizes training while each agent keeps a decentralized recurrent actor; the hidden states are per-agent, so the team's adaptation is only as coordinated as the actors' inputs allow. For **task inference**, the encoder can condition on the joint trajectory during meta-training but must run on local history at execution, and closing that gap is a research problem in its own right, which is what the second method below addresses. Two methods are directly relevant to this thesis.

Adaptation to non-stationary opponents. Al-Shedivat, Bansal, Burda, *et al.* [54] treat a competitor whose policy keeps changing as a *sequence of tasks*, and meta-learn an update that adapts the agent's policy from one opponent version to the next. This is the structured-adaptation idea of Section 6.6 with the latent parameter standing for opponent behaviour, realized by a meta-learned adaptation step rather than an explicit estimator, and it is one of the few places the meta-RL and adaptive-MARL literatures actually meet.

Meta Global-to-Local (MG2L). MG2L [61] is the task-inference method built for MARL, and it makes explicit a difficulty that CTDE glosses over. During centralized meta-training a task encoder can condition on the joint trajectory of every agent and produce a *global* task representation z^{g}; at decentralized execution agent i has only its own history and must produce a *local* estimate z^{i}. The gap between them is not just missing data, it is that a single agent's local observations may be genuinely uninformative about parts of the team task. MG2L meta-trains a multilevel encoder that produces both, and adds a consistency loss that pulls each z^{i} toward z^{g} (with a mutual-information term to keep z^{i} from collapsing to a constant), so that at deployment each agent's local estimate is as close to the global one as its observations allow. MG2L engages multi-agent structure head-on, but for *task inference*: it estimates *what* task the team is in, not *which* agent or which part of the environment caused an observed change. That attribution problem is identified in Chapters 8 and 9 as unsolved on both sides of the taxonomy.

Table 7.1. – Meta-RL taxonomy. All three families amortize the adaptation procedure over meta-training on a task distribution $p(\mathcal{M})$; the shared failure mode is a meta-test task or drift unlike anything in $p(\mathcal{M})$.

Family	Adaptation mechanism	Updated at deployment	Data requirement	Failure mode
Policy-parameter gradient (MAML)	A few inner-loop gradient steps from a learned initialization	Policy weights	On-policy rollouts on the new task for the inner step	Second-order cost; task outside $p(\mathcal{M})$; needs a real gradient step
Black-box (RL^2)	Recurrent forward pass over the multi-episode history	Hidden activations only	One (multi-episode) rollout; no gradient	Hidden-state bottleneck; long-horizon credit assignment; brittle off-distribution
Task inference (PEARL, VariBAD, MG2L)	Encode history into a latent task z; condition the policy on z (or on the full belief)	A posterior over z	A few transitions to sharpen the posterior	Inference only as good as $p(\mathcal{M})$ coverage; PEARL discards belief uncertainty

7.6. Comparison and Takeaway

The three mechanisms differ in what they change at deployment, and therefore in what they cost. *Gradient adaptation* changes the weights. It can move far from the initialization, but it needs new-task gradient data and pays a second-order cost. *Memory-based adaptation* changes only the activations. It is almost free at deployment, but it pushes all the difficulty into an unstable meta-training problem and a fixed-width bottleneck. *Latent-task inference* changes a posterior. It needs only a few transitions and, done fully, is Bayes-optimal, but it assumes that the deployment task is drawn from, or close to, the meta-training distribution.

That last assumption is the shared limitation. All three families move the cost of adaptation offline, into meta-training on $p(\mathcal{M})$. In exchange they inherit one failure mode: a deployment task, or a post-deployment drift, unlike anything in $p(\mathcal{M})$. This is the meta-RL version of a robust method's disturbance exceeding its uncertainty set, and Chapter 8 uses this parallel to place meta-RL, adaptive MARL, and robust MARL under one comparison.

Implications for the proposed method (R4). Latent-task inference is the fastest of the three adaptation mechanisms and the one this thesis builds on, but it is almost entirely single-agent. The proposed method therefore places a task-context encoder inside the credit-assigning FMASAC backbone, and pairs it with the online detector of Chapter 6 to guard the "unlike anything in $p(\mathcal{M})$" failure mode.

8. Robustness and Adaptability: A Unified Synthesis

RQ4 asks how robustness and adaptability relate, and what it would take to evaluate methods from both families on common ground. The three preceding chapters built the material this chapter needs: a robust MARL taxonomy (Chapter 5), an adaptive MARL taxonomy (Chapter 6), and meta-RL as a third adaptation mechanism (Chapter 7). This chapter places them side by side.

Throughout, three kinds of statement are kept distinct. [**Literature**] marks a result an existing paper establishes. [**Synthesis**] marks a claim that follows from comparing several papers but is not stated by any one of them. [**Proposed**] marks a direction this thesis argues is worth investigating and does *not* test. The distinction matters most in Sections 8.3 and 8.4.

8.1. A Shared Root Cause

[**Synthesis**] Robust MARL and adaptive MARL exist for the same reason: a MARL policy's environment, including the other agents in it, does not stay fixed (Section 4.7). They diverge only in what they assume about the departure and when they act on it.

- Robust MARL assumes the departure is *bounded* and treats every model inside the bound as possible, including the worst. Its objective is a minimax over an ambiguity set $\mathcal{P}$, fixed before training (Section 5.1).
- Adaptive MARL assumes the departure grows *without bound* over time but carries exploitable structure, and spends its effort tracking that structure: a confidence set that shrinks as evidence accumulates (UCRL2), a window that forgets old data (SWUCRL2-CW), or a task posterior inferred online (PEARL).

This connection is not unique to this thesis. As reviewed in Section 5.6, Wang and Zou [7] and Farhat, Ghosh, Atia, *et al.* [8] keep the worst-case objective but estimate the uncertainty set from online interaction rather than fixing it in advance. Their results show that the boundary between the robust and adaptive objectives is not

structural: it is a modelling choice about whether the model set is estimated once or updated as data arrive. [**Synthesis**] What this chapter adds is the same reduction stated at the level of the taxonomies built here, so that it applies to the exemplar methods surveyed, DrIGM, SWUCRL2-CW, PEARL, and the rest, rather than to one algorithm at a time.

The meta-RL side fits the same picture with the model set replaced by a task distribution. A robust method hedges every model in a fixed set $\mathcal{P}$; a task-inference meta-learner hedges every task in a fixed distribution $p(\mathcal{M})$ and, at deployment, collapses that distribution to a posterior using data. Freezing the posterior update, never letting the context c grow, leaves a policy that acts under the prior $p(\mathcal{M})$ alone, which is a policy robust to the whole task distribution and adaptive to nothing. Allowing the update recovers the adaptive method. So across all three chapters the pattern is identical: a set or distribution of environments the policy is prepared for, and a switch that says whether deployment data is allowed to narrow it. Robust methods weld the switch open; adaptive and meta-RL methods let it close.

8.2. The Unified Framework

Every method in Chapters 5–7 can be located by two questions. *What does it assume about the departure from training conditions?*, on a scale from a bounded static set to an unbounded structured process. And *when is the model set it carries fixed?*, on a scale from "entirely before deployment" to "re-estimated at every step". Figure 8.1 places the families on these two axes; Table 8.1 states the seven axes on which specific methods can be compared.

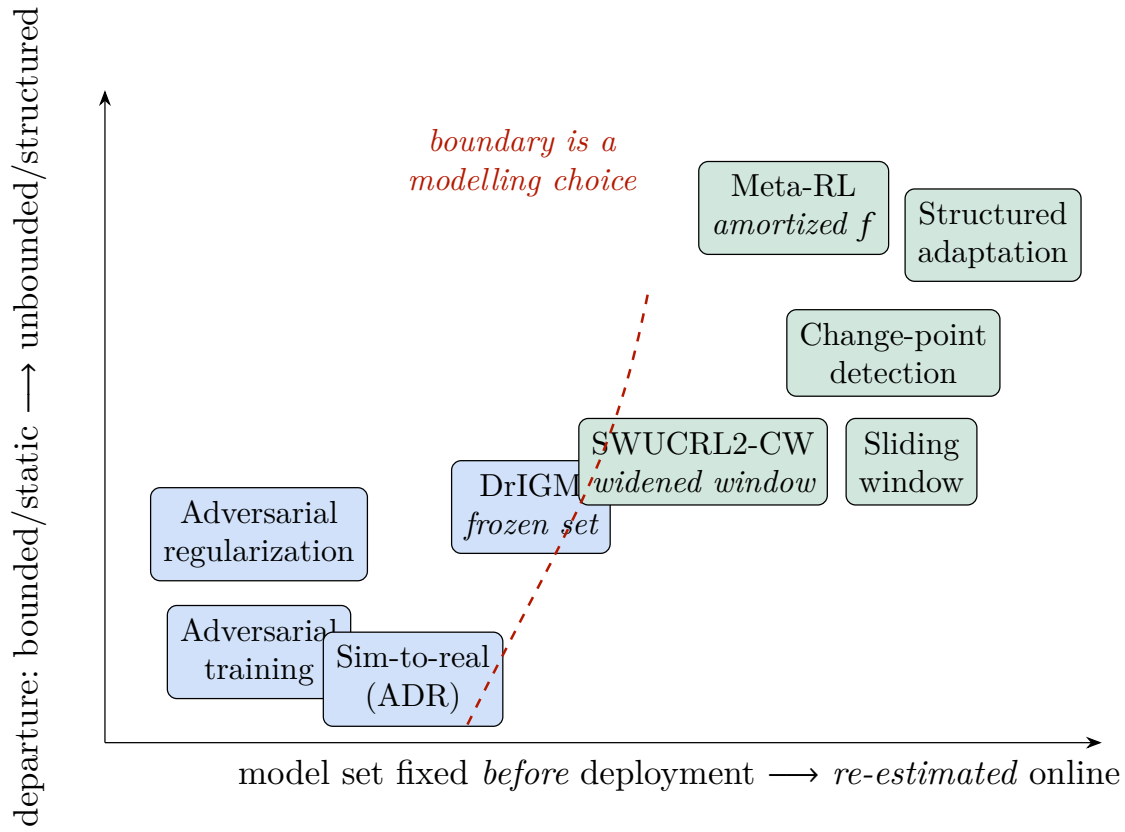

Figure 8.1. – Robust families (blue) and adaptive families (green) on two axes: what they assume about the departure from training, and when the model set they carry is fixed. DrIGM and SWUCRL2-CW sit on either side of a boundary that Section 8.3 shows is a single construction with one switch flipped.

Table 8.1. – Robust MARL and adaptive MARL (including meta-RL) compared on seven axes. Each axis is a column a specific method from either side can be scored on; the axes are used directly in Table 8.2.

Axis	**Robust MARL**	**Adaptive MARL / meta-RL**
Disturbance assumption	Bounded; adversarial or not; fixed for training	Unbounded over time; structured (piecewise, slow, low-dimensional, or task-distributed)
Temporal behaviour	Static: one departure, present throughout	Dynamic: a departure that accumulates or recurs
Optimization objective	Maximize worst-case return over $\mathcal{P}$	Minimize dynamic regret against a moving optimum; or few-shot return on a new task
Information availability	The uncertainty set and its radius, chosen in advance	A stream of deployment data; structure class known, its parameters (budget, hazard) often not
Adaptation timing	Before deployment, during training	After deployment, online; or amortized in meta-training and applied at test time
Computational mechanism	Minimax training, a sensitivity penalty, or a worst-case Bellman target	Detect-and-reset, windowed re-estimation, or a learned adaptation procedure
Failure mode	The real disturbance exceeds $\mathcal{P}$	The change outpaces the reaction time, or lacks the assumed structure / lies outside $p(\mathcal{M})$

Table 8.2. – The exemplar methods of Chapters 5–7, one row per method, on the axes of Table 8.1. Every cell restates what the method's own section already says.

Method	Family	Objective	Timing	Assumption	Mechanism / failure mode
RARL	Adversarial training	Max–min return, protagonist vs. adversary	Before deployment	Adversary's action space contains the test-time disturbance	Alternating zero-sum minimax training / robust only to disturbances the adversary can represent
M3DDPG	Adversarial training	Max return under worst-case joint teammate action	Before deployment	One-step local perturbation approximates the worst case	MAAL one-step gradient in the critic target / worst case only local, conservative on cooperative tasks
SA-MDP	Adversarial regularization	Return plus a penalty on policy divergence under bounded observation perturbation	Before deployment	True perturbation stays in the trained ℓ_∞ ball	Per-agent policy-consistency regularizer / observation channel only, guarantee = trained bound
ERNIE	Adversarial regularization	Return plus a Lipschitz-smoothness penalty	Before deployment	Reward and transitions are Lipschitz	Adversarial regularization, Stackelberg training / λ trade-off, holds only under smoothness
MIR3	Adversarial regularization	Return minus $\lambda\, I(\text{history}; \text{action})$	Before deployment	Robustness follows from reduced dependence on teammate information	CLUB-estimated MI penalty as modified reward / maximizes only a lower bound on the robust objective
RADIAL-RL	Adversarial regularization	Return plus a certified Q-margin under perturbation	Before deployment	Interval bounds hold for every perturbation in the budget	Interval bound propagation per agent, margin loss / certificate = trained budget, bounds loosen for deep nets

Table 8.2 – continued

Method	**Family**	**Objective**	**Timing**	**Assumption**	**Mechanism / failure mode**
DrIGM	Distributionally robust	Minimax return over a total-variation ball of transition models	Before deployment; set fixed once	True kernel inside the fixed TV ball of radius ρ	Robust Bellman target, IGM-preserving mixing / disturbance beyond ρ uncovered, set never updated
ADR / PERDRE	Sim-to-real	Max return against an adversarial, plausible domain-randomization generator	Before deployment, in simulation	Sim-to-real gap is a bounded physical-parameter shift	Minimax vs. a constrained Nature Player; prioritized DR replay / silent about post-deployment drift
BOCPD	Change-point detection	Localize a change via the run-length posterior	After deployment, online	Piecewise stationary, exponential-family observations	Recursive Bayesian run-length update / no adaptation of its own; model mismatch delays or false-triggers
RL-CD	Change-point detection	Recognize the current context and reuse its policy	After deployment, online	Finite, possibly recurring set of stationary contexts	Library of context models, best predictor active / continuous drift forces endless model creation
SW-UCRL2	Sliding window	Minimize dynamic regret against the changing optimum	After deployment, online	Bounded variation budget; windowed data represents the present	Windowed UCRL2 optimism / a window across a change inflates the diameter
SWUCRL2-CW	Sliding window	Same, corrected for the diameter blow-up	After deployment, online	A budget exists; η can be chosen to bound the extra pessimism	Confidence widening: floor η on the transition confidence region / optimal η needs the budget in advance

Table 8.2 – continued

Method	Family	Objective	Timing	Assumption	Mechanism / failure mode
MAML	Policy-parameter gradient	Learn an initialization reaching high return after a few gradient steps	Amortized; a few gradient steps at test time	All parameters need inner-loop adaptation; task distribution representative	Bi-level optimization / second-order cost, tasks outside $p(\mathcal{M})$ not covered
RL^2	Black-box	Max expected return across a task distribution	Amortized; test-time adaptation via activations only	Task information compresses into one recurrent state	Recurrent policy over the multi-episode history / hidden-state bottleneck, brittle off-distribution
VariBAD	Task inference	Act Bayes-optimally with respect to a belief-augmented objective	Amortized; online belief update at test time	Task variation captured by unknown reward/transition parameters	RNN encoder, ELBO plus RL objective, belief-conditioned policy / exact BAMDP only approximated, on-policy
PEARL	Task inference	Same task-inference objective, trained off-policy	Amortized (off-policy); online context inference at test time	Task fixed by its transition/reward statistics; context exchangeable	Permutation-invariant context encoder, SAC / encoder signal via the critic, one sampled latent discards belief uncertainty

Reading the seven axes in order shows they are not independent; they fall out of the one modelling choice of Section 8.1. *Disturbance assumption* and *temporal behaviour* are the choice itself, stated about the disturbance: bounded and static, or unbounded and dynamic. *Optimization objective* is its immediate consequence: hedging a static set is a minimax, tracking a moving target is a dynamic-regret problem. *Information availability* and *adaptation timing* follow from *when* the set is fixed: a set chosen in advance needs only its radius as input and is applied before deployment, whereas a set re-estimated online needs a data stream and acts with a lag. *Computational mechanism* is the implementation of whichever objective was selected. And *failure mode* is what happens when the modelling choice is wrong: a static set is exceeded, or a dynamic tracker is outrun. A method is therefore almost fully determined by two answers, is the departure bounded, and is the set frozen, and Table 8.2 is that pair of answers spelled out for sixteen methods.

8.3. Where the Two Taxonomies Already Touch

Holding the taxonomies against each other, rather than reading them as separate literatures, makes three points of contact visible that no single surveyed paper states.

8.3.1. Distributionally robust MARL is adaptive MARL with a frozen confidence set

[**Synthesis**] The claim can be made precise using the two methods' own formulations from Chapters 5 and 6.

DrIGM optimizes against the worst transition model in a per-history-action ambiguity set built once around a fixed reference model $P^0_{\mathbf{h},\mathbf{a}}$ at a fixed total-variation radius ρ,

$$\mathcal{P}_{\mathbf{h},\mathbf{a}} = \left\{ P \in \Delta(\mathcal{H}) \ : \ d_{\mathrm{TV}}\big(P, P^0_{\mathbf{h},\mathbf{a}}\big) \le \rho \right\},$$

and never updated as new transitions are observed. SWUCRL2-CW's transition confidence region at time t is

$$H_{p,t}(s,a)(\eta) = \left\{ \dot{p}(\cdot \mid s,a) \in \Delta_{\mathcal{S}} \ : \ \left\| \dot{p}(\cdot \mid s,a) - \hat{p}_t(\cdot \mid s,a) \right\|_1 \le \mathsf{rad}_{p,t}(s,a) + \eta \right\},$$

re-centred on the running windowed estimate $\hat{p}_t$ and shrinking with the windowed visit count through $\mathsf{rad}_{p,t}(s,a) \to 0$, with a fixed floor η added on top for exactly the reason $\mathcal{P}_{\mathbf{h},\mathbf{a}}$ is fixed in DrIGM: protection against a model that has moved since it was last estimated.

Freezing the re-centring step ($\hat{p}_t \equiv \hat{p}_0$, never updated) and disabling the shrinking term ($\mathsf{rad}_{p,t}(s,a) \equiv 0$, so only the constant floor remains) turns $H_{p,t}(s,a)(\eta)$ into an ℓ_1 ball of fixed radius η around a fixed centre, the same construction as $\mathcal{P}_{\mathbf{h},\mathbf{a}}$, whose radius ρ is stated in total-variation distance rather than ℓ_1. By the identity $d_{\mathrm{TV}} = \frac{1}{2}\|\cdot\|_1$ (Appendix C), a total-variation ball of radius ρ is an ℓ_1 ball of radius 2ρ, so the matching radius is $\rho = \eta/2$, not $\rho = \eta$.

The two sets also range over different spaces: $\mathcal{P}_{\mathbf{h},\mathbf{a}} \subseteq \Delta(\mathcal{H})$ is a set of distributions over joint histories, as DrIGM is built for a Dec-POMDP, while $H_{p,t}(s,a)(\eta) \subseteq \Delta_{\mathcal{S}}$ is a set of distributions over next states, as SWUCRL2-CW is built for a fully observable MDP. They coincide as sets, not only as constructions, when the Dec-POMDP is fully observable and a history reduces to the current state, so $\mathcal{H} = \mathcal{S}$. Outside that case, DrIGM's frozen set and SWUCRL2-CW's frozen limit are the *same mechanism*, an ℓ_1 / total-variation ball of fixed radius around a fixed centre, applied to different underlying spaces, not literally the same set. DrIGM is that special case mechanically; it is that special case set-theoretically only under full observability. Neither DrIGM nor SWUCRL2-CW is described this way by its own authors; the connection becomes visible only once both are read against Table 8.1.

What the equivalence buys is a shared vocabulary for the two failure modes. DrIGM fails when the true kernel leaves $\mathcal{P}_{\mathbf{h},\mathbf{a}}$; SWUCRL2-CW's confidence widening exists precisely to keep the true kernel inside $H_{p,t}(s,a)(\eta)$ *despite* drift, by re-centring $\hat{p}_t$ as data arrives. So the two are not competing designs but the two ends of one dial: fix the centre and you have a robust method whose set can be escaped by drift; move the centre and you have an adaptive method whose set tracks the drift, at the cost of the reaction lag built into $\hat{p}_t$. A method that re-centres *slowly* interpolates between them, and the choice of re-centring rate is exactly the window-size / variation-budget trade-off of Section 6.5 viewed from the robust side. This is the concrete content of the claim that robustness and adaptability are one construction with one switch: the switch is whether $\hat{p}_t$ is allowed to move.

8.3.2. Task-inference encoders can be meta-trained against adversarial tasks

[**Proposed**] PEARL and VariBAD (Section 7.4), and ELUE (Appendix A), meta-train a context encoder across a task distribution $p(\mathcal{M})$ so that a new task can be inferred from a handful of transitions. Nothing in that training procedure requires $p(\mathcal{M})$ to contain only benign variation in dynamics or reward. A task distribution that *includes adversarially perturbed variants*, generated the way ADR's Nature Player generates difficult but physically plausible domain-randomization parameters (Section 5.7), would push a task-inference method toward the robust objective of

Chapter 5 without changing its adaptation mechanism at all. This is a recombination of two existing pieces, an adversarial task generator and a task-inference encoder, not a new algorithm, and it is exactly the kind of possibility that stays invisible until the two taxonomies are compared directly. It is listed here as a research direction, not a validated method.

8.3.3. Multi-agent credit assignment is the missing piece on both sides

[**Synthesis**] M3DDPG (Section 5.4) trains each agent against the worst-case actions of its teammates, treating a shift in teammate behaviour as an adversarial disturbance to defend against. Change-point detectors and meta-RL task encoders, on the adaptive side, generally treat "the environment changed" as a single undifferentiated event, without asking whether the physical environment changed or a specific teammate's policy changed. MG2L (Section 7.5) is the one surveyed method that engages multi-agent structure directly, and it does so for task inference, not for attributing an observed change to a specific cause. Neither side of Table 8.1 has a general answer to *who, or what, caused a given disturbance*, which is a precondition for a team to respond to it proportionately.

8.4. Intersection Directions

The contact points above suggest combinations. Each is labelled by how much the literature already supports it.

Robust meta-RL. [**Proposed**] Meta-training across a task distribution is already close to training across an uncertainty set; the objective-level connection for non-meta methods is made by Wang and Zou [7] and Farhat, Ghosh, Atia, *et al.* [8] (Section 5.6). Combining an explicit adversarial task generator with a task-inference meta-learner, as in Section 8.3.2, is the concrete instantiation and is not yet tested.

Adversarial task distributions. [**Proposed**] Generating the meta-training $p(\mathcal{M})$ adversarially, rather than sampling it from a fixed benign prior, so that the encoder must infer tasks that were chosen to be hard.

Adaptive robust policies. [**Synthesis**] Section 8.3.1 shows DrIGM is SWUCRL2-CW with the re-centring and shrinking switched off. Switching them back on, letting a distributionally robust MARL method *re-estimate* its ambiguity set from deployment data, is the direct construction, and single-agent precedent exists [7].

Robustness plus online adaptation. [**Proposed**] Running a robust policy and an online adaptor together, the robust policy absorbing sharp bounded shocks while the adaptor tracks slow drift, matching the complementary failure modes noted at the end of Chapter 6. No surveyed method composes the two.

Task inference under adversarial perturbation. [**Proposed**] Whether a task-inference encoder can still identify the task when its context transitions are adversarially perturbed within a bounded set, that is, task inference with the observation-channel threat model of Section 5.5 applied to the encoder input.

[**Synthesis**] These directions answer RQ4's first half: robustness and adaptability relate through a single construction, a model set the policy is hedged against, that is either frozen (robust) or re-estimated (adaptive), and the interesting combinations all amount to unfreezing one and adding a bound to the other. The second half of RQ4, what it would take to evaluate methods from both families on common ground, is the subject of Chapter 9: none of these directions can be assessed today, because there is no benchmark on which a robust method and an adaptive method are even measured the same way.

9. Research Gaps and Evaluation Requirements

Chapter 8 showed that robustness and adaptability are two settings of one construction: a set of environment models the policy is protected against, either frozen before deployment or re-estimated during it. This chapter states what the survey leaves open. It gives four gaps that stand between the current literature and a single MARL policy that is at once nominal-performant, robust, and fast-adapting, and against which such a policy could be tested. Section 9.1 states the gaps as research problems. Section 9.3 gives the environment dimensions and Section 9.4 the metrics that any joint evaluation would have to control and report. Both are used by the evaluation outline of Chapter 10. Section 9.5 states what is and is not being claimed.

9.1. Four Gaps

9.1.1. No shared benchmark

Robust MARL methods, RARL, M3DDPG, ERNIE, MIR3, DrIGM, ADR, are almost universally evaluated against a *fixed, bounded* perturbation applied at test time: a fixed attack budget, a fixed domain-randomization range, a fixed adversarial opponent. On the adaptive side, the reference methods, BOCPD, RL-CD, SW-UCRL2, SWUCRL2-CW, together with the meta-RL methods, are almost universally evaluated against a *schedule of changes that is never adversarial*, only assumed piecewise stationary, slowly varying, or drawn from a benign task distribution. The first four are single-agent non-stationarity mechanisms applied to a multi-agent environment (Chapter 6); they stand in as the adaptive-side reference points here precisely because no multi-agent-native method pairs an unbounded, ongoing drift with an adversarial-robustness objective,[1] which is one more axis on which the two literatures fail to meet. No environment surveyed in this thesis applies both at once, a

[1] Rivas, Saika, Bakht, *et al.* [62] come closest: a red/blue adversarial setup layered on drift in a network-security MARL task. Their notion of drift, however, is the adversary's own co-adapting strategy, not an independent bounded perturbation applied on top of an unbounded task drift, so it does not supply the shared benchmark this section calls for.

bounded adversarial perturbation layered on top of an ongoing, unbounded drift, in the same multi-agent setting. A method evaluated against only one of the two has not been tested against the other, however strong its published guarantee looks in isolation.

The failure this hides is concrete. A distributionally robust policy tuned to a total-variation radius ρ has a guarantee that is vacuous the moment slow drift carries the true kernel past ρ, but no robust-MARL evaluation includes drift, so the guarantee is never seen to lapse. Conversely, a sliding-window learner with a well-chosen window tracks benign drift with low dynamic regret, but a single adversarial spike inside one window inflates the empirical diameter (Section 6.5) and can stall its planner for many episodes; no adaptive-MARL evaluation includes an adversary, so this is never seen either. Each method's published strength is measured on exactly the axis its weakness does not show. The gap is a single environment family that supports both kinds of departure simultaneously and independently, so that the two failure modes can appear in the same run.

9.1.2. No shared metric

Robust MARL reports worst-case return degradation under a specific named attack. Adaptive MARL reports dynamic regret, or an average dynamic suboptimality gap, across a sequence of tasks or environments (Section 6.2). These measure different things and neither can be read off the other: a small worst-case degradation says nothing about tracking speed, and a small dynamic regret says nothing about the worst single step under attack. The two even have opposite signs of desirable variance, a robust method wants low spread across adversarial conditions, an adaptive method accepts high spread early and low spread once it has tracked, so a single summary statistic favours one by construction. Comparing a robust method against an adaptive one today therefore requires re-running both under a metric neither was built to optimize, and choosing that metric is itself a contested modelling step, which is a large part of why no such comparison exists. The gap is an agreed metric set that scores worst-case degradation, recovery time, and cumulative dynamic regret on the same runs, so that a method's number on one does not silently depend on which the evaluator cared about.

9.1.3. Sim-to-real robustness is not post-deployment adaptability

ADR and PERDRE (Section 5.7) close the gap between a simulator and *one* physical deployment, fixed once training ends. They say nothing about an environment that

keeps changing after that point, which is exactly what the change-point and sliding-window methods of Chapter 6 target. Those methods, in turn, are validated almost entirely in tabular or low-dimensional settings, not in the continuous-control, deep-MARL simulators that sim-to-real work actually uses. No method surveyed here has been trained with adversarial domain randomization for a real-robot-style deployment and then evaluated on how well it tracks continued drift after deployment begins. The gap is an evaluation protocol that runs the sim-to-real transfer step and then keeps the environment moving.

9.1.4. No multi-agent credit assignment for the source of a change

Change-point detectors and meta-RL task encoders alike treat "the environment changed" as one event, without asking whether the physical environment changed or a specific teammate's or opponent's policy did (Section 8.3.3). MG2L engages multi-agent structure but for task inference, not attribution. A team that cannot tell *which* of its members, or which part of the world, caused a disturbance cannot respond to it proportionately: it must either over-react across all agents or under-react everywhere. The gap is a mechanism, and an evaluation of it, that attributes an observed change to an agent or an environment component.

9.2. Design Requirements for a Unified Method

The four gaps above are about *evaluation*. Read together with Chapters 5–8, the same review also pins down what a single method that is nominal-performant, robust, and fast-adapting would have to do. Seven requirements follow. Chapter 10 is organized around meeting them, and each is traced back to the chapter that produces it.

R1 — Robustness without a nominal-performance tax. From Chapter 5 (RQ3): every robust family buys a worst-case floor by giving up expected-case return, and none returns to nominal behaviour when no disturbance is present. The method must instead *condition* its conservatism on an online estimate of the disturbance, so it collapses to nominal behaviour at zero disturbance.

R2 — Perturb where the multi-agent threat enters, and contain it. From Chapter 5 and Section 8.3.1: in a cooperative team the bounded disturbance that matters is a deviation in *teammate* behaviour. The method must optimize against a worst-case teammate-action perturbation, not only observation noise, and must confine it so that exploration and task learning are not destabilized.

R3 — Track unbounded drift; do not assume it bounded. From Chapter 6: a fixed ambiguity set is vacuous once drift carries the environment past its radius. The method must detect regime change online and re-estimate, rather than rely on a radius fixed before deployment.

R4 — Amortized task inference inside a team learner. From Chapter 7: latent-task inference adapts fastest and is near Bayes-optimal, but it has been built almost entirely for a single agent. The method must carry a task-context encoder *inside* a credit-assigning cooperative backbone.

R5 — Per-agent, not team-wide, change. From Sections 4.7 and 8.3.3: each agent's effective environment can shift while the others are stationary. Context inference, change detection, and the post-change reset must all be per-agent.

R6 — Separate "the world changed" from "the adversary is strong". From Section 9.1.4: a team that conflates the two must over-react everywhere or under-react everywhere. The method must infer the task context and the perturbation budget as *distinct*, disentangled quantities.

R7 — Evaluate both properties on the same runs. From Sections 9.1.1–9.1.2: the method must be tested on an environment with independent bounded-shock and unbounded-drift dials (Section 9.3) under a metric set that scores both (Section 9.4).

R1–R6 shape the method of Chapter 10; R7 shapes its evaluation. The rest of this chapter specifies the environment and metrics that R7 calls for.

9.3. A Conceptual Benchmark

The four gaps share a cause: robust and adaptive MARL are evaluated in environments built for one property at a time. Closing them needs an environment family whose non-stationarity is *parameterized* along independent axes, so that a bounded adversarial shock and an unbounded natural drift can be dialled in together, and mapped onto a real-world task class such as multi-robot coordination, traffic signal control, or swarm foraging. Eight dimensions (Figure 9.1) span the space.

1. **Disturbance magnitude.** The size of the bounded perturbation, an ℓ_p radius on observations or actions, a transition-model divergence, a force bound. At zero this reduces to a purely adaptive benchmark.
2. **Disturbance duration.** How long each bounded perturbation lasts: a single step, a fixed window, or the whole episode. Separates transient shocks from sustained ones.

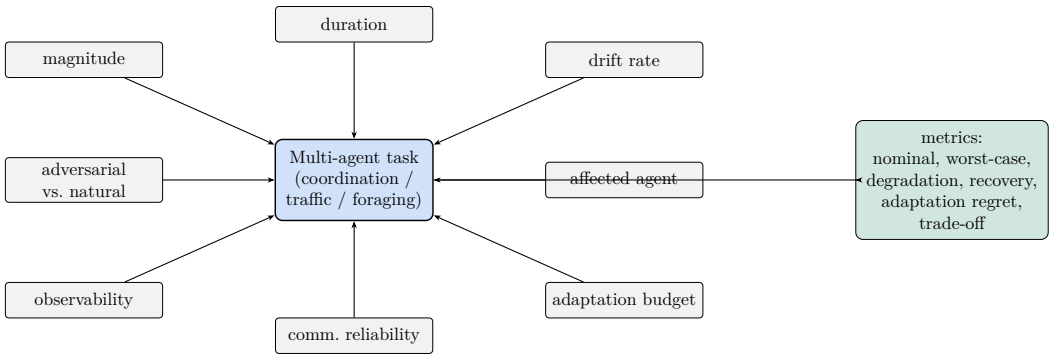

Figure 9.1. – The proposed benchmark: eight independently set dimensions of non-stationarity feed one multi-agent task, evaluated by the metrics of Section 9.4. Setting magnitude to zero recovers a purely adaptive benchmark; setting drift rate to zero recovers a purely robust one.

3. **Rate of environmental drift.** The per-step variation budget δ of the underlying non-stationary MDP (Section 6.2). At zero this reduces to a purely robust benchmark.
4. **Adversarial vs. natural change.** Whether the drift and the perturbations are generated by a return-minimizing adversary (as in ADR) or sampled from a benign stochastic process (as in the adaptive literature), on a continuum set by how much budget the adversary is given.
5. **Affected agent.** Which agents a given disturbance touches: one named agent, a random subset, or all of them. This is the axis the credit-assignment gap (Section 9.1.4) lives on.
6. **Observability.** Whether agents observe the full state, their own local observation only, or a degraded observation, and whether the disturbance itself is observable.
7. **Communication reliability.** The probability and pattern of message loss, delay, or corruption between agents, from a perfect channel to none.
8. **Adaptation budget.** How much interaction the learner is allowed in the new regime before it is scored: a few transitions (few-shot), a few episodes, or an unbounded online lifetime.

Two settings of these dials recover the existing literatures as special cases, which is what makes cross-family comparison possible: {magnitude > 0, drift rate $= 0$, adversarial} is the robust evaluation setting, and {magnitude $= 0$, drift rate > 0, natural} is the adaptive one. Every point in between is a test neither literature currently runs.

9.3.1. Instantiating the dimensions in a task class

The dimensions are abstract; a benchmark has to realize them in a concrete environment family. Three task classes that recur in the MARL literature each support all eight, which is what makes them suitable anchors.

Traffic signal control. Agents are intersection controllers; the shared state is queue lengths and phases. Drift rate is the rate of change of arrival demand over a simulated day; magnitude and duration are a bounded perturbation of a controller's sensed queue lengths (a miscalibrated loop detector) or of its phase timing; adversarial-vs-natural is whether demand surges are sampled from historical patterns or chosen to congest the network; the affected agent is which intersections a sensor fault or demand surge hits; observability ranges from network-wide state to a single intersection's detectors; communication reliability is the link between neighbouring controllers that exchange platoon-arrival predictions; the adaptation budget is how many signal cycles the learner gets before it is scored.

Multi-robot coordination. Agents are mobile robots with a joint task such as formation keeping or transport. Drift is a slowly changing payload mass or floor friction; the bounded perturbation is an ℓ_p ball on a robot's pose estimate or a bounded actuator bias; adversarial-vs-natural is whether the dynamics shift is sampled or driven by an ADR-style Nature Player; the affected agent is which robot's sensing or actuation degrades; observability is full state versus onboard sensing; communication reliability is the inter-robot radio; the adaptation budget is a number of episodes.

Swarm foraging. Agents are foragers collecting resources whose locations and yields change. Drift is the migration of resource patches; the bounded perturbation is noise on an agent's local resource sensor or a bounded number of spoofed detections; adversarial-vs-natural is whether patch dynamics follow an ecological model or an adversary that moves resources away from the swarm's learned routes; the affected agent, observability, and communication axes are as above.

In every case, setting drift to zero leaves a robust-MARL evaluation on that task, and setting the bounded perturbation to zero leaves an adaptive-MARL one, so a method from either literature can be dropped in unchanged and then tested along the axis it has never been tested on.

9.4. Evaluation Metrics

A benchmark that spans both properties needs metrics that score both. The set below is provisional; the exact metric definitions and their aggregation are left open

for the empirical stage. As a starting point, seven metrics are proposed, and the first six can be computed from the same set of runs.

- **Nominal performance.** Return with all disturbance dials at zero, the reference point.
- **Worst-case performance.** Lowest return over the adversarial settings of magnitude and duration, the robust literature's usual number.
- **Performance degradation.** Nominal minus worst-case, absolute or relative.
- **Recovery time.** Steps from the onset of a change until return returns to within a tolerance of its pre-change level, the tracking-speed number the robust literature never reports.
- **Adaptation regret.** Dynamic regret (Section 6.2) accumulated over the run, the adaptive literature's usual number.
- **Cumulative reward.** Total undiscounted return over the whole non-stationary run, the deployment-relevant bottom line.
- **Robustness–adaptability trade-off.** The pair (worst-case performance, adaptation regret), or a single scalar combining them, reported as a curve as the adversarial-vs-natural dial is swept. This is the number that does not exist today.

9.5. Status of the Requirements

[**Proposed**] The dimensions and metrics above are a specification, not an experiment. A first implementation of the evaluation environment — one task class, its partial-observability model, its reward, a domain-randomization hook, and the distributionally robust exemplar — is reported in Section 10.12, but no full evaluation has been run and no results are reported. The exemplar shortlist of Chapter 8 (Table 8.2) is assembled so that a future study can populate such an evaluation with baselines from both sides, for example DrIGM against SWUCRL2-CW. Chapter 10 presents the author's proposed method and outlines a Search-and-Rescue evaluation based on these dimensions and metrics. Layering the non-stationarity, adding an adaptive-side baseline, implementing the shared metric, and running the comparison is the next stage of the work.

Table 9.1. – The four gaps, why each blocks a robust-vs-adaptive comparison, and what closing it requires.

Gap	Why it blocks comparison	What closing it requires
No shared benchmark	The two families are tested on disjoint disturbance types	One environment family with independent bounded-shock and unbounded-drift dials (§9.3)
No shared metric	Worst-case degradation and dynamic regret do not translate	A metric set scoring both on the same runs (§9.4)
Sim-to-real ≠ post-deployment adaptability	Transfer is evaluated once; drift is evaluated in toy settings	A protocol that transfers, then keeps the environment moving, in a deep-MARL simulator
No multi-agent credit assignment	"The environment changed" is treated as one event	A mechanism, and a test, that attributes a change to an agent or a component (§9.1.4)

Part III.

Proposed Method and Outlook

10. Proposed Method: Two-Phase Exchangeable-Context CCM–FMASAC

The survey and the gap analysis identify one opening. Robust methods gain a worst-case floor at a nominal-performance cost (RQ3). Adaptive methods that detect a change and then re-estimate are slow, and in the multi-agent case they cannot attribute the change. Meta-learning, the mechanism best suited to fast adaptation, has been developed almost entirely for a single agent. No surveyed method obtains nominal performance, robustness, and fast adaptation together, and no surveyed method handles a bounded perturbation and a change in the task at the same time.

This chapter presents the method the author proposes to close that gap. It combines four mechanisms drawn from the survey into a single per-agent framework:

1. **contrastive context-based meta-RL** (CCM) [63], [64] for task inference and information-seeking exploration;
2. the **exchangeable-process view** of meta-RL [65], which replaces CCM's mean/product-of-Gaussians context pooling with an exact, $O(1)$-per-step recursive Gaussian posterior over the latent context;
3. the **factored multi-agent soft actor–critic** (FMASAC) [39] as the cooperative backbone (Section 4.9); and
4. **representation-learned change-point detection** (InDiD) [66], a supervised detector that reuses the context embeddings and, at deployment, resets the context posterior and re-triggers exploration.

Robustness is added M3DDPG-style: the execution critic is trained against a worst-case, one-step (MAAL) perturbation of the teammate actions, and a second variational encoder infers a posterior over the perturbation radius. Following VariBAD, the execution actor and the centralized critic are both conditioned on the *whole* posteriors — agent i's own context $(\mu_{z,i}, \Sigma_{z,i})$, which can change while other agents are stationary, and the single team-shared radius (μ_ρ, Σ_ρ), since the adversary's budget is the same for all — rather than on reparameterized samples; the latents are sampled only inside the reconstruction losses. Both the perturbation and its encoder are scoped to the execution stream only. Training is two-phase: Phase 1 jointly trains

the encoder, the perturbation branch, and both policies on episodes that contain a known task switch; Phase 2 freezes Phase 1 and trains the detector on the same episodes. The contribution is the combination and its coupling, expressed through two structural invariants (Section 10.2), not the individual mechanisms. The method is specified in full; it is not implemented in this thesis, and its evaluation is future work (Sections 10.11–10.14).

Requirements met. The design targets the seven requirements of Section 9.2. *R1* (robustness without a nominal-performance tax): the execution actor and critic condition on the full perturbation-budget posterior, so conservatism tracks the inferred radius and collapses when it is zero (Sections 10.5, 10.6). *R2* (teammate channel, contained): the worst-case MAAL perturbation is applied to teammate actions on the execution stream only (Section 10.5). *R3* (track unbounded drift): the second-phase InDiD detector flags a regime change online and resets the context posterior (Sections 10.9, 10.9.1). *R4* (amortized inference in a team learner): the contrastive context encoder sits on the FMASAC backbone (Sections 10.3, 10.6). *R5* (per-agent change): task sampling, the context posterior, the detector, and the reset are all per-agent (Sections 10.1, 10.3, 10.9.1). *R6* (world-change vs. adversary strength): the context latent z_i and the budget latent ρ are inferred as disentangled quantities (Section 10.8). *R7* (both properties on the same runs): the Search-and-Rescue outline instantiates the independent-dial benchmark and the joint metric set (Sections 10.11, 10.12).

10.1. Notation and Non-Stationary Task Sampling

The setting is the cooperative Dec-POMDP of Chapter 3, with agents $i = 1, \ldots, n$, local histories τ_i, joint history $\tau = (\tau_1, \ldots, \tau_n)$, and a global state s used only during centralized training. Each training episode of horizon H samples, *per agent*, two local environment regimes and a switch time, and one *team-wide* perturbation budget,

$$\mu_{1,i}, \mu_{2,i} \sim p(\mu), \qquad \vartheta_i \sim \mathrm{GEOM}(p_{\mathrm{sw}}), \quad \tilde{\vartheta}_i = \min(\vartheta_i, H), \qquad \epsilon_a \sim p_\epsilon, \tag{10.1}$$

so agent i's active regime is $\mu_i(t) = \mu_{1,i}$ for $t < \tilde{\vartheta}_i$ and $\mu_i(t) = \mu_{2,i}$ for $t \geq \tilde{\vartheta}_i$. The environment can therefore change for one agent while the others are stationary, matching the local changes of Chapter 3. Agents with $\tilde{\vartheta}_i = H$ are valid *no-switch* cases, giving Phase 2 stationary negatives. The budget ϵ_a is the radius the adversarial perturbation of Section 10.5 must respect, $\|\check{\boldsymbol{a}}^{-i} - \hat{\boldsymbol{a}}^{-i}\| \leq \epsilon_a$; it is a single team-wide draw (the adversary's ball is the same for every agent), independent of the tasks, and is the quantity the perturbation encoder infers.

Accordingly the two encoders of Sections 10.3 and 10.5 produce a *per-agent* Gaussian posterior over the local context and *one shared* Gaussian posterior over the perturbation budget; agent i's belief is

$$b_i = \big\{(\mu_{z,i}, \Sigma_{z,i}),\ (\mu_\rho, \Sigma_\rho)\big\}. \tag{10.2}$$

Following VariBAD [59], the execution policy and the execution critic are both conditioned on the *whole distribution* b_i, not on a reparameterized sample; the latents are sampled only inside the reconstruction losses of Sections 10.3 and 10.5. The two policies of agent i are then the exploration policy $\pi_{\theta_e,i}(a_i \mid o_i)$ and the execution policy $\pi_{\theta_x,i}(a_i \mid o_i, b_i)$. The exploration policy is *not* conditioned on b_i, matching CCM's design, in which exploration is driven purely by the intrinsic reward rather than by the current context estimate.

10.2. Architecture and Structural Invariants

Figure 10.1 shows the agent-level data flow. Rollouts feed two disjoint replay buffers. Each agent runs an environment encoder that maintains a *per-agent* exchangeable posterior $(\mu_{z,i}, \Sigma_{z,i})$ over its local context z_i (ELBO + contrastive-predictive-coding loss); one team-shared perturbation encoder maintains a single posterior (μ_ρ, Σ_ρ) over the perturbation-budget latent ρ (ELBO+$(\epsilon_a - \hat{\epsilon})^2$ loss on the episode's sampled adversary radius ϵ_a). VariBAD-style, the execution actor and the execution critic are both conditioned on the whole belief $b_i = \{(\mu_{z,i}, \Sigma_{z,i}), (\mu_\rho, \Sigma_\rho)\}$, not on a sample; the perturbation posterior is used only in the execution branch. Two invariants, fixed during the design of that diagram, are enforced throughout.

1. **Critic separation.** The factored mixing network Q_{tot} is built only from the execution critics $\{Q_i^{\text{exe}}\}_{i=1}^n$. The exploration critics $\{Q_i^{\text{exp}}\}_{i=1}^n$ never contribute to Q_{tot}, because they are trained on an intrinsic, information-theoretic reward that is not commensurable with task return.
2. **Perturbation scoping.** The M3DDPG worst-case teammate-action perturbation $\check{\boldsymbol{a}}^{-i}$ and the shared perturbation-budget latent ρ influence only the execution stream — the execution actor, its critic, and the critic target. The exploration policy and Q_i^{exp} never see $\check{\boldsymbol{a}}^{-i}$ or ρ, because robustifying exploratory behaviour would corrupt the information-gain signal it is trained to maximize.

10.3. Exchangeable Context Encoder

Each agent runs its own copy of this encoder on its own history. For agent i, each local transition is mapped by an invertible flow f_ϕ (shared parameters, per-agent

input) to a latent code $c_{i,t} = f_\phi(y_{i,t} \mid x_{i,t}) \in \mathbb{R}^d$, with inputs $x_{i,t} = (o_{i,t}, a_{i,t})$ and targets $y_{i,t} = (r_{i,t}, o_{i,t+1})$, so that within a local regime the sequence $\{c_{i,t}\}$ is modelled as an exchangeable Gaussian process, $\Sigma_{tt} = \nu$ and $\Sigma_{tt'} = \kappa$ for $t \neq t'$. By de Finetti's theorem this is equivalent to sampling the $c_{i,t}$ conditionally i.i.d. given a per-agent latent context z_i, whose posterior admits the exact recursive conjugate update

$$\mu_{i,t} = (1-\beta_{t-1})\mu_{i,t-1}+\beta_{t-1}c_{i,t-1}, \qquad \sigma^2_{i,t} = (1-\beta_{t-1})\big(\sigma^2_{i,t-1}-\nu+\kappa\big), \qquad \beta_{t-1} = \frac{\kappa}{\nu+\kappa(t-2)}, \tag{10.3}$$

initialized at $\mu_{i,0} = 0$, $\sigma^2_{i,0} = \kappa$ and *reset* to this prior whenever a change point is flagged for agent i (the ground-truth $\tilde{\vartheta}_i$ in Phase 1; the detector's own signal at test time), so that z_i is never estimated from a window that straddles two of that agent's regimes. The posterior parameters $(\mu_{i,t}, \sigma^2_{i,t})$ are the context part of the belief b_i that the execution policy and critic condition on (Section 10.1); a sample $z_{i,t} \sim \mathcal{N}(\mu_{i,t}, \sigma^2_{i,t} I)$ is drawn only for the reconstruction loss below.

Why an exchangeable posterior. Modelling the codes as exchangeable, rather than pooling them as in CCM or amortizing the belief in a recurrent network as in VariBAD, is what keeps the rest of the design exact and cheap. *(i)* The belief $(\mu_{i,t}, \sigma^2_{i,t})$ that the policy and critic condition on is the true Bayesian posterior under the model, in closed form — not a heuristic pooling or a network output that may be miscalibrated. *(ii)* Eq. (10.3) is an $O(1)$-per-step recursive update: the running posterior absorbs each new code without re-encoding the history, which matters because the intrinsic reward of Section 10.4 is recomputed at essentially every step, training is long-horizon and on-line, and the environment is run in large vectorized batches. *(iii)* Because the posterior is exactly Gaussian, the one-step information gain $I(z_i; c_{i,t} \mid c_{i,1:t-1})$ has the closed form of Eq. (10.5), so the exploration policy is driven by an exact signal rather than CCM's noisy InfoNCE bound-difference estimate. *(iv)* Exchangeability is the right inductive bias: within a stationary regime only *which* transitions have been seen should matter for the regime estimate, not their order, and the model builds that permutation-invariance in by construction, whereas a recurrent encoder must learn it. *(v)* $\sigma^2_{i,t}$ shrinks monotonically at a rate set by (ν, κ), a calibrated "belief sharpness" used both by the information-gain reward and by the meta-test stopping rule (Algorithm 10.3). *(vi)* A proper recursive filter makes the change-point response a one-line prior reset with the correct meaning: discard the pre-switch codes, re-accumulate from $(\mu, \sigma^2) = (0, \kappa)$. The one caveat — that all of this holds only while the flow can make $\{c_{i,t}\}$ exchangeable-Gaussian within a regime — is discussed under "When the contrastive term is needed" below and in Section 10.14.

Two admissible choices for context inference. The context latent z_i can be inferred in either of two ways, and the design is compatible with both; we adopt the second and record the trade-off here.

Option A — amortized variational inference (VariBAD/PEARL-style). A learned encoder $q_\phi(z_i \mid \tau_i)$ outputs the belief directly, trained by the ELBO alone (the first line of Eq. (10.4)), exactly as the perturbation-budget encoder of Section 10.5 is trained. *For:* it makes no exchangeability or Gaussian-in-code assumption, so it can absorb residual temporal structure in the codes and represent richer, nonlinear, or multimodal task variation through the decoder; and it uses one uniform inference mechanism for both latents. *Against:* the belief is a network output with no calibration guarantee; there is no closed-form one-step information gain (Eq. (10.5)), so the exploration reward must fall back to an approximate entropy- or bound-difference estimate; a recurrent amortization must *learn* permutation-invariance and a sensible mid-episode reset rather than getting them by construction; and the objective adds the usual variational-training difficulties (KL weighting, posterior collapse, encoder–policy interaction).

Option B — exchangeable-process posterior (BRUNO), adopted here. The flow f_ϕ is still a learned amortized encoder, but the per-transition codes are *combined* by the exact conjugate recursion of Eq. (10.3) rather than by a learned recurrence. *For:* the belief is the true Bayesian posterior under the model, in closed form and monotonically sharpening; the one-step information gain is exact (Eq. (10.5)); the update is $O(1)$ per step and order-invariant by construction; the change-point response is a one-line prior reset with the correct meaning; and there is no KL term to balance. *Against:* it assumes the flow can render $\{c_{i,t}\}$ conditionally i.i.d. Gaussian within a regime, which is only approximately true — the posterior variance contracts as $1/t$ regardless of the true correlation, so the belief can be overconfident, and a single Gaussian is misspecified under genuine multimodality (the case handled by $\lambda_{\text{CPC}} > 0$ below).

We adopt Option B because the exploration mechanism of Section 10.4 and the meta-test stopping rule of Algorithm 10.3 both consume closed-form posterior quantities that Option A would only approximate, because the change-point reset is exact, and because the regime structure of the target benchmark (Section 10.12) is low-dimensional and well identified, so Option B's misspecification risk is small while its overconfidence is bounded by the frequent resets. Option A becomes the preferable choice if the task variation is later made continuous and high-dimensional enough that z_i must reconstruct dynamics rather than select among a few regimes; the switch then mirrors the λ_{CPC} knob discussed below.

The training signal, however, stays CCM's, not the exchangeable model's own likelihood: the encoder is trained with a hybrid loss combining variational reconstruction and a contrastive task-discrimination term,

$$\mathcal{L}_{\text{enc}} = \underbrace{-\mathbb{E}_{q_\phi(z_i|\tau_i)}\big[\log p_\psi(y_{i,1:T} \mid z_i, x_{i,1:T})\big] + \mathrm{KL}\big(q_\phi(z_i \mid \tau_i) \,\|\, p(z)\big)}_{\mathcal{L}_{\text{ELBO}}} - \lambda_{\text{CPC}} \underbrace{\log \frac{\exp\big(f(z_q, z_{\text{pos}})\big)}{\sum_j \exp\big(f(z_q, z_j)\big)}}_{\mathcal{L}_{\text{CPC}}}, \tag{10.4}$$

summed over agents. In the ELBO term the decoder p_ψ reconstructs, from agent i's context z_i, the reward and next observation $y_{i,t} = (r_{i,t}, o_{i,t+1})$ at every step of the local regime — the VariBAD reconstruction target [59] — and the KL term regularizes the posterior toward the prior $p(z)$. For the CPC term, $f(z, z') = z^\top W z'$ with learnable W; $z_q = e_\phi(b_n^q)$ is the online embedding of a segment from an agent's current regime, $z_{\text{pos}} = \bar{e}_\phi(b_n^k)$ a momentum-encoder embedding of a different segment from the *same* regime, and $\{z_j\}$ momentum-encoder embeddings of segments from other regimes — including, for free, the pre- and post-switch halves of the same agent's episode. The momentum encoder is updated as $\bar{\phi} \leftarrow \tau_m \bar{\phi} + (1 - \tau_m)\phi$. The input $x_{i,t}$ is always the clean observation, and z_i is kept disentangled from the shared perturbation latent ρ (Section 10.8).

When the contrastive term is needed. The exchangeable-Gaussian posterior of Eq. (10.3) is a single Gaussian in z, which is the right model when the non-stationary environment parameters θ_{env} vary *continuously and unimodally* — a smooth drift with no distinct regimes, as in the physics randomization of Section 10.12.5, where drag, friction, and mass are drawn from connected uniform ranges. There the reconstruction and KL terms of $\mathcal{L}_{\text{ELBO}}$ already place nearby parameter values at nearby z, the contrastive term buys little, and it can be switched off, $\lambda_{\text{CPC}} = 0$.

The contrastive term earns its place when the non-stationarity is *multimodal*: a small number of qualitatively distinct regimes — an icy, a wet, and a dry surface, say — best described as a mixture $p(\theta_{\text{env}}) = \sum_{m=1}^{M} \pi_m \, p_m(\theta_{\text{env}})$ over M modes. A single Gaussian posterior is then misspecified, and $\mathcal{L}_{\text{ELBO}}$ alone tends to blur the modes toward an averaged z or to interpolate between them; the InfoNCE term $\mathcal{L}_{\text{CPC}}$ is exactly what prevents this, pulling same-mode segments together and pushing different-mode segments apart so that z becomes *mode-discriminative* rather than mode-averaged. The two-task episodes of Section 10.1, with $\mu_1, \mu_2 \sim p(\mu)$ and a sharp switch between them, are mode-structured in this sense, so $\lambda_{\text{CPC}} > 0$ is the default; the continuous-drift case is the exception in which it can be dropped. This

knob moves with the one in Section 10.4: when the posterior is misspecified by multimodality, the closed-form information gain of Eq. (10.5) likewise reverts to its InfoNCE bound-difference form.

10.4. Information-Gain Exploration

Because each agent's posterior in Eq. (10.3) is exactly Gaussian, the one-step information gain about that agent's own context has a closed form as the variance-reduction mutual information,

$$r_{i,t}^{\text{aux}} = I(z_i; c_{i,t} \mid c_{i,1:t-1}) = \tfrac{1}{2} \log\!\big(\sigma_{i,t-1}^2 / \sigma_{i,t}^2\big), \tag{10.5}$$

which replaces CCM's InfoNCE bound-difference estimator $L_{\text{upper}} - L_{\text{lower}}$ with an exact value under the exchangeable-Gaussian assumption; the bound-difference estimator remains a drop-in fallback if that assumption is violated in practice.

Derivation of Eq. (10.5). The one-step information gain about z_i from a single new code, given the codes already seen, is an entropy reduction,

$$I(z_i; c_{i,t} \mid c_{i,1:t-1}) = H(z_i \mid c_{i,1:t-1}) - H(z_i \mid c_{i,1:t}), \tag{10.6}$$

by the identity $I(X; Y \mid Z) = H(X \mid Z) - H(X \mid Y, Z)$: conditioning further on the new evidence $c_{i,t}$ can only shrink the uncertainty about z_i, and the amount it shrinks by is exactly the information $c_{i,t}$ carries about z_i, given what was already known. Both terms on the right are entropies of a Gaussian — $z_i \mid c_{i,1:t-1} \sim \mathcal{N}(\mu_{i,t-1}, \sigma_{i,t-1}^2)$, the running posterior *before* absorbing $c_{i,t}$, and $z_i \mid c_{i,1:t} \sim \mathcal{N}(\mu_{i,t}, \sigma_{i,t}^2)$, the posterior *after*, exactly the two states Eq. (10.3) produces one step apart. Using the differential entropy of a Gaussian, $H(\mathcal{N}(\mu, \sigma^2)) = \tfrac{1}{2} \log(2\pi e\, \sigma^2)$, Eq. (10.6) becomes

$$I(z_i; c_{i,t} \mid c_{i,1:t-1}) = \tfrac{1}{2} \log(2\pi e\, \sigma_{i,t-1}^2) - \tfrac{1}{2} \log(2\pi e\, \sigma_{i,t}^2) = \tfrac{1}{2} \log\!\big(\sigma_{i,t-1}^2 / \sigma_{i,t}^2\big), \tag{10.7}$$

the $2\pi e$ constant cancelling, which is Eq. (10.5).

As a consistency check, the same value follows directly from the update mechanics. Writing the running posterior as the prior for step t and the exchangeable-Gaussian model's per-code conditional as $c_{i,t} \mid z_i \sim \mathcal{N}(z_i, \nu - \kappa)$ (the "observation noise" implied by $\Sigma_{tt} = \nu$, $\Sigma_{tt'} = \kappa$), absorbing one new conditionally Gaussian observation is a standard conjugate update that combines precisions, $1/\sigma_{i,t}^2 = 1/\sigma_{i,t-1}^2 + 1/(\nu - \kappa)$, so $\sigma_{i,t-1}^2 / \sigma_{i,t}^2 = 1 + \sigma_{i,t-1}^2/(\nu - \kappa) > 1$ whenever $\nu > \kappa$. This is the familiar Gaussian-channel identity $I = \tfrac{1}{2} \log(1 + \text{SNR})$ with $\text{SNR} = \sigma_{i,t-1}^2/(\nu - \kappa)$, and it confirms both

the sign of $r^{\text{aux}}_{i,t}$ (always ≥ 0, an actual gain) and the monotonic shrinkage $\sigma^2_{i,t} \leq \sigma^2_{i,t-1}$ invoked throughout Section 10.3.

Dimensionality. If $z_i \in \mathbb{R}^d$ with isotropic posterior covariance $\sigma^2_{i,t} I$, the exact information gain carries a factor of d, $I = \frac{d}{2} \log(\sigma^2_{i,t-1}/\sigma^2_{i,t})$; Eq. (10.5) drops this constant factor. Since d is fixed across every step and every agent, this only rescales $r^{\text{aux}}_{i,t}$ by a constant, which is absorbed into α_{exp} and does not affect what the exploration policy is optimizing for.

The exploration reward for agent i is $r^e_{i,t} = r^{\text{env}}_{i,t} + \alpha_{\text{exp}}\, r^{\text{aux}}_{i,t}$, and each $\pi_{\theta_{e,i}}$ is updated independently by SAC on r^e_i through its own critic Q^{exp}_i and entropy temperature α_{exp} — with no mixing network, by the critic-separation invariant.

10.5. Adversarial Perturbation Branch (M3DDPG-Style)

The execution stream is robustified as in M3DDPG [41]. Write $\boldsymbol{b} = (b_1, \ldots, b_n)$ for the team-wide stack of the per-agent beliefs b_i of Eq. (10.2); since the budget posterior is shared this is $\boldsymbol{b} = \{(\mu_{z,i}, \Sigma_{z,i})_{i=1}^n,\ (\mu_\rho, \Sigma_\rho)\}$, which the execution critic consumes (Section 10.6). In the soft target and the factored soft-policy objective, the expectation over the other agents' actions is replaced by a worst case inside a bounded ball: for agent i,

$$Q^{\text{rob}}_i = \min_{\|\boldsymbol{a}^{-i} - \hat{\boldsymbol{a}}^{-i}\| \leq \epsilon_a} Q^{\bar{\zeta},\bar{\omega}}_{\text{tot}}\big(\tau,\ (a_i, \boldsymbol{a}^{-i}),\ s,\ \boldsymbol{b}\big), \qquad \hat{a}_j \sim \pi_{\theta_x,j}(\cdot \mid o_j, b_j), \tag{10.8}$$

so agent i trains against the joint teammate action that most reduces the team value. The inner minimization is intractable, so — exactly as in M3DDPG's multi-agent adversarial learning (MAAL) approximation — it is taken as a single projected gradient-descent step on the target critic,

$$\check{\boldsymbol{a}}^{-i} = \Pi_{\epsilon_a}\Big[\hat{\boldsymbol{a}}^{-i} - \alpha_{\text{adv}}\, \nabla_{\boldsymbol{a}^{-i}} Q^{\bar{\zeta},\bar{\omega}}_{\text{tot}}\big(\tau,\ (a_i, \hat{\boldsymbol{a}}^{-i}),\ s,\ \boldsymbol{b}\big)\Big], \tag{10.9}$$

with Π_{ϵ_a} the projection onto the ϵ_a-ball. The soft target y_{tot} (Eq. (10.15)) and the actor gradient (Eq. (10.14)) are then evaluated at the adversarial joint action $(a_i, \check{\boldsymbol{a}}^{-i})$ instead of the nominal one. This is the FMASAC counterpart of the M3DDPG critic target (Eq. (5.1)), with the centralized *mixed* value carrying the worst case in place of a per-agent joint critic. No separate adversary network is trained; unlike RARL [17] the perturbation is a closed-form one-step quantity.

Perturbation-budget encoder. A single, team-shared variational encoder, structured like the context encoder but scoped to the execution stream and pooling the execution histories $x^{\text{exe}} = (x_1^{\text{exe}}, \dots, x_n^{\text{exe}})$, maintains one Gaussian posterior over the current perturbation *budget*,

$$q_\eta(\rho \mid x^{\text{exe}}) = \mathcal{N}(\mu_\rho, \Sigma_\rho), \tag{10.10}$$

because the adversary's ball radius ϵ_a is the same for every agent (Eq. (10.1)). Its decoder p_η reconstructs the *radius* ϵ_a, not the realized perturbation: from a sample $\rho \sim q_\eta$ it predicts $\hat{\epsilon} = p_\eta(\rho)$, trained against the episode's sampled budget,

$$\mathcal{L}_\rho = \underbrace{\mathrm{KL}\big(q_\eta(\rho \mid x^{\text{exe}}) \,\|\, p(\rho)\big)}_{\text{ELBO regularizer}} + \underbrace{(\epsilon_a - \hat{\epsilon})^2}_{\text{radius reconstruction}} . \tag{10.11}$$

The target is the ball radius that bounds the worst case ($\|\check{\boldsymbol{a}}^{-i} - \hat{\boldsymbol{a}}^{-i}\| \le \epsilon_a$), not the one-step displacement $\|\check{\boldsymbol{a}}^{-i} - \hat{\boldsymbol{a}}^{-i}\|$: the latter is a noisy, gradient- and critic-dependent by-product of a single MAAL step, whereas ϵ_a is the stationary quantity that defines the threat model and is what the policy needs in order to set its conservatism. This is the perturbation-side counterpart of the environment encoder, whose decoder p_ψ reconstructs the reward and next observation (Section 10.3); here the target is the scalar radius ϵ_a rather than the transition. Because the encoder conditions on the history and never on ϵ_a itself, and ϵ_a is a per-episode draw independent of the critic and of the tasks (Eq. (10.1)), there is no circularity, and at deployment — where no adversary is run — q_η infers the *effective* perturbation level of the environment from the trajectory statistics, a well-posed inference in the same sense as the context encoder inferring μ_i.

Following VariBAD [59], the latents are sampled only inside these reconstruction losses; the execution *actor* and the execution *critic* are both conditioned on the whole belief b_i (Eq. (10.2)), never on a reparameterized sample,

$$a_i^{\text{exe}} \sim \pi_{\theta_x,i}(\cdot \mid o_i,\, b_i), \qquad b_i = \big\{(\mu_{z,i}, \Sigma_{z,i}),\ (\mu_\rho, \Sigma_\rho)\big\}. \tag{10.12}$$

By the perturbation-scoping invariant, the exploration action $a_i^{\text{exp}} \sim \pi_{\theta_e,i}(\cdot \mid o_i)$ and its critic Q_i^{exp} see neither $\check{\boldsymbol{a}}^{-i}$ nor b_i.

10.6. Factored Execution Critic (FMASAC)

With the joint belief $\boldsymbol{b}$ of Section 10.1, each agent has a per-agent execution critic $Q_i(\tau_i, a_i, \boldsymbol{b}; \zeta_i)$ that, VariBAD-style, receives the full posteriors rather than a single

sample; a mixing network satisfying the individual-global-max (IGM) property combines them as

$$Q_{\text{tot}}^{\zeta,\omega}(\tau, \boldsymbol{a}, s, \boldsymbol{b}) = g_\omega\big(s, \{Q_i(\tau_i, a_i, \boldsymbol{b}; \zeta_i)\}_{i=1}^n\big), \tag{10.13}$$

built only from the execution critics. The factored soft policy objective and soft critic loss follow FMASAC directly,

$$\mathcal{L}(\pi_{\theta_x}) = \mathbb{E}_{\pi_{\theta_x}}\Big[\alpha_{\text{exe}} \textstyle\sum_i \log \pi_{\theta_x,i}(a_i \mid o_i, b_i) - Q_{\text{tot}}^{\zeta,\omega}(\tau, (a_i, \check{\boldsymbol{a}}^{-i}), s, \boldsymbol{b})\Big], \tag{10.14}$$

$$\mathcal{L}_Q(\zeta, \omega) = \mathbb{E}_{\mathcal{D}_{\text{exe}}}\Big[\big(Q_{\text{tot}}^{\zeta,\omega}(\tau, \check{\boldsymbol{a}}, s, \boldsymbol{b}) - y_{\text{tot}}\big)^2\Big], \tag{10.15}$$

with the bootstrapped target y_{tot} computed from target networks $\bar{\zeta}, \bar{\theta}_x, \bar{\omega}$ in the usual soft way, and α_{exe} adapted against a minimum-entropy constraint H_0, independently of α_{exp}. Per Section 10.5, the joint action that enters Q_{tot} is the M3DDPG-perturbed one of Eq. (10.9): $(a_i, \check{\boldsymbol{a}}^{-i})$ in agent i's actor gradient, and the fully perturbed $\check{\boldsymbol{a}}$ in the critic target.

10.7. Buffer Semantics

Following CCM, the two replay buffers serve distinct roles. The exploration buffer $\mathcal{D}_{\text{exp}}$ holds transitions $(o_{i,t}, a_{i,t}, o_{i,t+1})$ tagged with the agent's active regime label $\mu_i(t)$, and drives the exploration policy and the contrastive encoder. The execution buffer $\mathcal{D}_{\text{exe}}$ holds transitions $(o_{i,t}, a_{i,t}, o_{i,t+1}, r_{i,t}, \mu_i(t))$; it also receives every exploration transition with its intrinsic reward stripped, so that execution-side context inference sees the full trajectory. The intrinsic reward r^{aux}, the sampled contexts z_i, and the MAAL perturbation $\check{\boldsymbol{a}}^{-i}$ of Eq. (10.9) are never stored — all depend on the current parameters and are recomputed at sample time. In particular, the context encoder is fed only adversary-free transitions (Section 10.8).

10.8. Disentangling the Context and Perturbation Latents

Each per-agent context z_i (from q_ϕ) and the single shared perturbation latent ρ (from q_η) encode independent generative factors: z_i carries agent i's local regime μ_i, drawn from $p(\mu)$ and switching at $\tilde{\vartheta}_i$, whereas ρ carries the adversary's *budget* ϵ_a, one per-episode team-wide draw from p_ϵ made independently of every μ_i (Eq. (10.1)). Since $\epsilon_a \perp \mu_i$ in the sampler, the posteriors should share no information, and the target is a generative-independence guarantee, not only an architectural one. The encoders are in any case separate networks with disjoint parameters (ϕ, ψ) and η.

Data path (primary). The context encoder is conditioned only on adversary-free trajectories — exploration transitions in $\mathcal{D}_{\text{exp}}$, which the perturbation-scoping invariant keeps free of the adversary, and clean-action execution transitions in $\mathcal{D}_{\text{exe}}$. The MAAL perturbation $\breve{\boldsymbol{a}}^{-i}$ is a critic-side quantity, recomputed at update time and never written to a buffer (Section 10.7), so there is no path from the adversary to the flow input $x_{i,t} = (o_{i,t}, a_{i,t})$. By construction each z_i is then a measurable function of variables independent of ϵ_a and $\breve{\boldsymbol{a}}^{-i}$, so $I(z_i; \rho) = 0$ follows from the computation graph and the sampler, not from optimization.

Residual linear leakage (cleanup). Second-order dependence can still arise: q_η pools the execution histories, which are regime-correlated, so ρ can absorb some μ_i-variance; and if the two encoders share upstream features, gradients from $\mathcal{L}_\rho$ (supervised by ϵ_a) can shape regime-relevant directions. As cleanup we add a cross-covariance penalty on the encoder *means*, over a minibatch of size B,

$$C_{z,\rho} = \frac{1}{B}\sum_{b=1}^{B}(\mu_{z,b} - \bar{\mu}_z)(\mu_{\rho,b} - \bar{\mu}_\rho)^\top, \qquad \mathcal{L}_{\text{cov}} = \left\|C_{z,\rho}\right\|_F^2, \tag{10.16}$$

with $\bar{\mu}_z = \frac{1}{B}\sum_b \mu_{z,b}$ (and likewise $\bar{\mu}_\rho$); over a minibatch of agent-transitions, $\mu_{z,b}$ is that agent's context posterior mean $\mu_{i,t}$ of Eq. (10.3) and $\mu_{\rho,b}$ is the team budget posterior mean of q_η at the same step. Only the cross-covariance *block* is penalized, not the full joint covariance, so the marginal variances of z and ρ stay free and the representations keep their capacity. The penalty acts on the posterior means, not the reparameterized samples, so it targets systematic dependence rather than the injected sampling noise, and the pairs $(\mu_{z,t}, \mu_{\rho,t})$ are taken at the same step t, already available since both posteriors are recomputed every step. The Phase-1 representation objective is then

$$\mathcal{L}_{\text{rep}} = \mathcal{L}_{\text{enc}} + \mathcal{L}_\rho + \lambda_{\text{cov}}\,\mathcal{L}_{\text{cov}}, \tag{10.17}$$

with a small weight λ_{cov}.

Why it works. If the context encoder's input carries no adversarial signal, each z_i is a function of variables independent of ϵ_a and $\breve{\boldsymbol{a}}^{-i}$, so $I(z_i; \rho) = 0$ by construction. Were adversarial-driven variance to enter its input, the reconstruction term of $\mathcal{L}_{\text{enc}}$ would reward z_i for explaining it, with three costs: the codes $\{c_{i,t}\}$ stop being exchangeable within a regime, so the Gaussian posterior (Eq. (10.3)) and the closed-form information gain (Eq. (10.5)) are misspecified; the exploration reward then credits variance reduction that is really "learning the budget level"; and the Phase-2 detector, which reads the same $c_{i,1:T}$, sees adversarial jitter that inflates its false-alarm rate. $\mathcal{L}_{\text{cov}}$ removes only *linear* dependence — two variables can satisfy

Cov = 0 and still be dependent — which is why it is cleanup and not the primary defense; a nonlinear guarantee would need an adversarial mutual-information bound or a kernel (HSIC) term, both heavier and less stable, and unnecessary once the data path is clean.

10.9. Two-Phase Training and Deployment

Phase 1 (Algorithm 10.1) jointly trains the context encoder, the perturbation branch, and both policies on the episodes of Section 10.1, using each agent's *known* switch time $\tilde{\vartheta}_i$ to reset that agent's context posterior. Phase 2 (Algorithm 10.2) freezes everything from Phase 1 and trains a single (shared parameters, per-agent input) InDiD-style transformer change-point detector, supervised by the ground-truth $\tilde{\vartheta}_i$ and reusing the exchangeable embeddings $c_{i,1:T}$ as its input. At deployment (Algorithm 10.3) the switch times are unknown and the detector's own per-agent reset signal both re-initializes that agent's context posterior and hands control back to its exploration policy for a bounded window.

10.9.1. Meta-Test: Reset-Triggered Re-Exploration

At deployment the regime identities and switch times are unknown and no gradient updates are performed (all of $\phi, \psi, \eta, \{\theta_{e,i}\}, \{\theta_{x,i}\}, \{\zeta_i\}, \omega, \xi$ are frozen). Each agent begins in Explore mode; at every step the detector produces $p_{i,t} = h_\xi(c_{i,1:t})$, and a reset fires for agent i under the standard stopping rule $\tau_{\text{detect}} = \inf\{t : p_{i,t} \geq C\}$ for threshold C. A reset (i) re-initializes that agent's context posterior to its prior, since its pre-switch context is no longer valid, and (ii) hands that agent back to its exploration policy for a bounded window, so that fresh, informative transitions are collected for its new regime before execution resumes. Other agents are unaffected.

10.10. Summary of the Proposed Architecture

The framework is the sequence

$$(x_{i,t}, y_{i,t}) \xrightarrow{f_\phi} c_{i,t} \xrightarrow{\text{Eq. (10.3)}} (\mu_{z,i}, \Sigma_{z,i}); \qquad x^{\text{exe}} \xrightarrow{q_\eta} (\mu_\rho, \Sigma_\rho); \qquad b_i = \big\{(\mu_{z,i}, \Sigma_{z,i}), (\mu_\rho, \Sigma_\rho)\big\}$$
$$\to \big(\pi_{\theta_{e,i}} \text{ via } r^{\text{aux}}_{i,t};\ \ \pi_{\theta_{x,i}}(o_i, b_i),\ \check{\boldsymbol{a}}^{-i},\ \boldsymbol{b} \to Q_{\text{tot}}\big) \xrightarrow{h_\xi} p_{i,t} \to \text{reset}_i.$$

It addresses the four challenges of the gap analysis: *task adaptation* (the exchangeable contrastive context), *efficient exploration* (the closed-form information-gain reward), *cooperative policy learning* (the factored execution critic under CTDE), and *robustness* (the scoped M3DDPG worst-case teammate-action perturbation). The two structural invariants keep the intrinsic-reward exploration stream and the task-return execution stream from contaminating each other. The contribution is this combination and its coupling; the constituent mechanisms — CCM, the exchangeable posterior, FMASAC, and InDiD — are each taken from prior work.

10.11. Planned Evaluation

[**Future work**] The method will be evaluated in a Search-and-Rescue task, chosen because it carries both settings of Chapter 3; a first implementation of that environment is described in Section 10.12. A small team of robot agents searches an arena with obstacles and victims at unknown locations, under partial observation and distance-limited communication. *Bounded disturbances*: noise on an agent's pose and local view, a chance that a commanded move is replaced by a random one, message dropout, and, in the hardest setting, one agent that fails. *Task change*: the obstacle and victim layout is resampled between episodes, and, within an episode, the conditions in one agent's region change at a random step $\tilde{\vartheta}_i$ while the other agents' regions are stationary; some layouts recur. The hardest condition applies a bounded perturbation and a within-episode per-agent switch together (RQ4).

The eight scenario dimensions of Section 9.3 set the magnitude, duration, and rate of these changes, whether they are adversarial or natural, which agents they affect, the observability, the communication reliability, and the adaptation budget. The provisional metrics of Section 9.4 are measured on the same runs; their exact definitions are to be fixed at the empirical stage. Baselines are plain FMASAC [39] for the nominal and adaptation references and exemplars from Table 8.2: standalone M3DDPG [41] as the robust exemplar, a detector-based adaptive exemplar, and a single-agent meta-RL exemplar applied per agent. Ablations replace the exchangeable posterior with CCM's mean pooling, replace the closed-form information gain (Eq. (10.5)) with the InfoNCE bound-difference estimator, remove the M3DDPG perturbation (nominal critic target) or replace its one-step MAAL step with a random teammate-action perturbation, fix the budget ϵ_a instead of sampling it (the perturbation-budget encoder then degenerates and the policy loses its ρ-conditioning), replace the InDiD detector with a classical online detector, and merge the two training phases, to show that each choice earns its place.

10.12. First Implementation of the Evaluation Environment

[**Infrastructure**] This section reports the first implemented piece of the evaluation of Section 10.11: a Search-and-Rescue environment in the multi-robot coordination task class of Section 9.3.1, its partial-observability model, its speed-sensitive reward, a domain-randomization hook for the sim-to-real axis, and a reproduction of the distributionally robust exemplar of Table 8.2. Nothing here is trained to convergence or evaluated for results; every component below is validated only by the smoke tests listed at the end of the section. The layered non-stationarity, the adaptive-side baseline, and the shared metric that the full comparison of Section 10.11 needs are not yet built.

10.12.1. Simulator and Training Stack

The environment is built on VMAS [67], a batched, GPU-vectorized continuous-control multi-agent simulator, and trained through BenchMARL [68] on TorchRL [69]. A vectorized simulator is chosen over the grid-world sketch of Section 10.11 for three reasons: the many-seed sweeps that the benchmark dials of Section 9.3 require are cheap when hundreds of environment copies step in one batched tensor operation; the continuous action interface matches the soft actor–critic backbone of Section 10.6; and the value-factorization baseline below reuses BenchMARL's maintained VDN/QMIX implementations rather than a bespoke one. The environment exposes both a discrete and a continuous action interface; the FMASAC backbone of this chapter uses the continuous one, while the value-factorization baseline uses the discrete one, as those methods require.

10.12.2. Arena and Scenario

A `Map` is a bounded, optionally walled arena holding agents (circles), obstacles (collidable boxes and spheres), and victims (point targets, each with a scalar `weight`). It is built either from a YAML configuration or generated at random, with a save-to-YAML and reload round trip so that a generated layout can be frozen for repeated runs. A `Scenario` wraps a `Map` into a VMAS world. Each victim's number of *required rescuers* is

$$k_v = \left\lceil \mathrm{weight}_v \,/\, 10 \right\rceil, \tag{10.18}$$

so a heavier victim can be rescued only when more agents are present at once; this is the coordination requirement the task turns on, and it sits on the *affected-agent* axis

of Section 9.3. Health, rescue state, and per-victim counters are stored as tensors of shape (batch_dim,), one entry per environment copy, so that the vectorized batch does not share episode state across copies.

10.12.3. Partial Observability: Two Disjoint Sensing Channels

Each agent observes

$$o_i = \left[\, \mathrm{pos}(2),\ \mathrm{vel}(2),\ \mathrm{lidar}(n_{\mathrm{rays}}),\ \mathrm{ears}(2\,n_{\mathrm{victims}}) \,\right], \tag{10.19}$$

with the two exteroceptive channels deliberately made asymmetric, in the sense of the partial-observability discussion of Chapter 4.

Lidar (VMAS's ray sensor, $n_{\mathrm{rays}} = 12$ rays, range 3.0) returns a bounded-range distance per ray, filtered to obstacle landmarks only — victims and other agents are excluded. This is the channel agents use to navigate and avoid collisions: the arena geometry is sensed directly and locally.

Ears (`EaredAgent`) perceive victims through a binaural cue: for each victim, a left and a right distance are computed from two virtual ear points offset from the agent's centre, and only that pair enters the observation — never a relative position vector. This models sound-based victim detection and is the *only* channel that carries victim information. The design is intentional: the environment geometry is fully and directly observable, whereas the actual objective is available only through a weaker, indirect cue. The task therefore instantiates the degraded-observation setting of the *observability* dial of Section 9.3, with the disturbance-relevant signal (victim location) the hardest part of the state to infer.

10.12.4. Rescue Mechanics and Speed-Sensitive Reward

A victim starts at $h_v = 100$ and, while active (not yet rescued and not dead), loses `decay_rate` = 0.2 health per simulation step. It becomes *rescued* the first time k_v agents are simultaneously within `rescue_range` = 0.5 of it — a sticky, one-shot covering event, following the covering mechanic of VMAS's own discovery scenario. The rescuing agents receive

$$r_v^{\mathrm{rescue}} = \texttt{rescue\_reward} \cdot \frac{h_v^{\text{at rescue}}}{100}, \tag{10.20}$$

proportional to the victim's *remaining* health at the moment of rescue, and every agent additionally receives a shared per-step penalty $-\,\texttt{decay\_penalty} \cdot \left(\sum_v \Delta h_v\right)$ for total health lost that step. The episode ends when every victim is either rescued

Table 10.1. – Search-and-Rescue environment parameters and their default values.

Parameter	Default	Meaning
n_{rays}	12	lidar rays per agent
lidar range	3.0	maximum lidar reading
k_v	$\lceil \text{weight}_v/10 \rceil$	agents needed to rescue victim v
`rescue_range`	0.5	distance within which an agent counts as present
h_v (initial)	100	victim health at episode start
`decay_rate`	0.2	health lost per active step
`rescue_reward`	config	scale of the rescue bonus, Eq. (10.20)
`decay_penalty`	config	scale of the shared health-loss penalty

or dead ($h_v \leq 0$). Because Eq. (10.20) pays less for a slower rescue even before the shared penalty is counted, response speed is a first-class objective of the task — the quantity that the recovery-time and tracking-speed metrics of Section 9.4 are meant to measure. Table 10.1 lists the environment parameters and their defaults.

10.12.5. Domain-Randomization Hook

A physics-task sampler draws four world parameters — `drag`, `linear_friction`, `angular_friction`, and `agent_mass` — from configurable uniform ranges, and the scenario applies the sampled values to the VMAS world and agents at construction. This is the intended entry point for the sim-to-real and adversarial-domain-randomization axis (Section 5.7) and for the MAML-style task distributions of Chapter 7: a per-reset draw gives a static-per-episode task, and a return-minimizing sampler would give the ADR setting of the *adversarial-vs-natural* dial. Sampling is implemented and tested; no training run uses it yet.

10.12.6. Distributionally Robust Baseline

The robust-MARL exemplar of Table 8.2 is DrIGM [46], ρ-contamination robust value factorization. Its reference implementation in the `robust-coMARL` code base is tied to a discrete `MultiDiscrete` building-energy environment with a fixed observation dimension and a single-environment Python training loop, none of which fits this batched, continuous-friendly VMAS task. Rather than porting that environment-specific code, the identical robust Bellman target is reproduced on the BenchMARL/TorchRL stack already in use. `RobustQmix` and `RobustVdn` subclass BenchMARL's stock `Qmix` and `Vdn` and change one thing: the TD(0) value estimator's dis-

count becomes $\gamma(1-\rho)$ in place of γ. Since TorchRL's TD(0) target is $r+\gamma V(s')(1-\text{done})$, this reproduces `robust-coMARL`'s $r + \gamma(1-\rho)\max_a Q(s',a)(1-\text{done})$ exactly, with no reimplementation of the mixer, loss, or replay machinery, and it stays batched. A `RescueTaskClass` (a BenchMARL `VmasClass`) wraps the `Scenario` as a task, building a fresh `Scenario` per environment so that the train and evaluation environments do not share one mutable object. Setting $\rho = 0$ recovers plain VDN/QMIX, so the same code provides both the non-robust reference and the robust variant by sweeping a single parameter.

10.12.7. Validation Status

[**Smoke-tested**] The following were checked directly; none is an experimental result.

- *Rescue mechanics.* Forcing k_v agents onto a victim confirms **rescued** = true, reward exactly `rescue_reward` $\cdot\, h_v^{\text{at rescue}}/100$, health frozen after rescue, and episode **done** = true; an unreached victim decays and never triggers the bonus.
- *Lidar.* Short readings next to an obstacle, `max_range` readings when clear, and victims never registering on the ray sensor.
- *Shapes.* Observation and reward tensors match n_{agents} and n_{victims} on a fixed YAML map and on randomly generated maps, including a save-to-YAML and reload round trip.
- *Rendering.* A camera-origin bug and a viewer-zoom bug were fixed: VMAS's auto-fit camera squares `viewer_zoom` once it exceeds about 1, so scaling it to the map size zoomed out by the square of the intended factor; the default is restored, with an explicit `fit_map` option that computes the correct $\sqrt{\text{half-extent}}$.
- *Training pipeline.* A few end-to-end BenchMARL iterations run without error, and the registered value-estimator discount is confirmed equal to $\gamma(1-\rho)$ (e.g. $0.99 \cdot (1-0.3) = 0.693$).

10.12.8. What Remains

[**Future work**] Four pieces stand between this infrastructure and the comparison of Section 10.11. *(i)* No adaptive-MARL baseline — change-point detection or sliding-window re-estimation from Chapter 6 — is wired up, so the robust-versus-adaptive comparison on shared ground has not been run. *(ii)* No unbounded structured drift (a per-step variation budget, Section 6.2) and no bounded adversarial perturbation

are layered on the environment yet; it is currently a static or reset-time-randomized task, not the "both at once" benchmark. *(iii)* The shared metric of Section 9.4 — worst-case degradation paired with tracking speed — is not implemented. *(iv)* Nothing has been trained to convergence or evaluated. This section is the first of the steps that Section 10.11 lists, not a set of findings.

10.13. What Would Confirm or Refute the Method

The method is supported if, on the same runs, it (i) matches plain FMASAC's nominal return with no adversarial perturbation and no agent's regime switched, (ii) exceeds standalone M3DDPG's worst-case return under perturbation, (iii) has a shorter recovery time after an agent's regime switch than the detector-based exemplar, and (iv) keeps (ii) and (iii) when a perturbation and a per-agent switch occur together, where each ablation degrades. It is weakened if the nominal return drops below plain FMASAC (the MAAL perturbation made the execution policy over-conservative), if the InDiD detector's delay or false-alarm rate is no better than a classical detector's, or if the exchangeable posterior gives no advantage over mean pooling on recovery time.

10.14. Limitations of the Design

The exact information gain of Eq. (10.5) and the $O(1)$ posterior update of Eq. (10.3) hold only under the exchangeable-Gaussian assumption on the flow codes; if the flow cannot make $\{c_{i,t}\}$ exchangeable within a regime, the method falls back to the weaker InfoNCE estimator. Training is two-phase, so the detector cannot shape the representation it consumes, and the InDiD detector needs ground-truth switch times $\tilde{\vartheta}_i$, available only from a simulator (Chapter 6). Per-agent contexts z_i and per-agent switch times $\tilde{\vartheta}_i$ let the environment change for one agent while the others are stationary, but the perturbation budget is still assumed *shared* — one ϵ_a for the whole team — which rules out an adversary that targets one agent's action channel harder than the rest. The adversary follows M3DDPG's MAAL approximation [41]: a single linearized gradient step for the inner minimization, which is only a local, first-order worst case and can under-estimate a strong joint deviation of the teammate actions. The perturbation-budget encoder is supervised by the sampled radius ϵ_a during Phase 1 and infers the effective budget from trajectory statistics at deployment; this is a well-posed inference, but its calibration under a test-time perturbation regime outside p_ϵ is unverified. The method is also heavy: an invertible flow, two variational encoder–decoder pairs whose posteriors both feed the critic, two full SAC stacks

(only one of them factored), and a transformer detector. It inherits FMASAC's monotonic-mixer restriction (Section 4.9) and CTDE's need for a centralized state at training time. It is specified but not implemented, so the claim that its parts reinforce each other is a hypothesis until the evaluation of Section 10.11 is run.

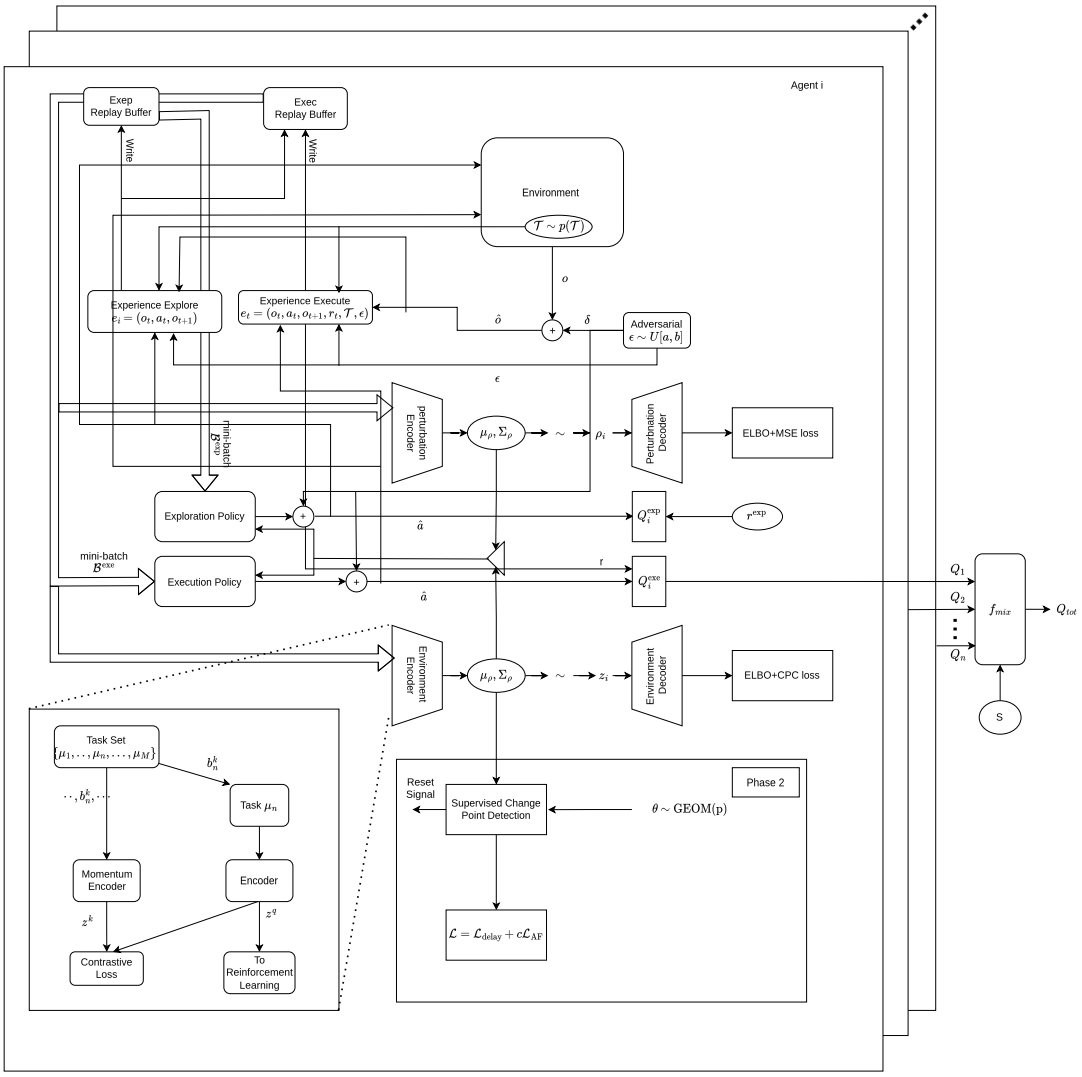

Figure 10.1. – Agent-level data flow. Task-conditioned rollouts feed two disjoint replay buffers (Explore/Exec). Each agent's environment encoder maintains a per-agent exchangeable posterior over its local context z_i (ELBO + CPC loss); one team-shared perturbation encoder maintains a single posterior over the budget latent ρ (ELBO + $(\epsilon_a - \hat{\epsilon})^2$ loss on the episode's sampled adversary radius ϵ_a). VariBAD-style, the execution actor and critic both take the full posteriors of the two latents, not samples. Phase 2 trains a supervised change-point detector whose reset signal re-initializes the context posterior and, at test time, returns control to the exploration policy. The diagram's "$\epsilon \sim U[a, b]$" block is realized by the one-step MAAL step of Eq. (10.9).

Algorithm 10.1 Phase 1 — joint context, exploration, and execution training

Require: task distribution $p(\mu)$; switch parameter p_{sw}; budget distribution p_ϵ; horizon H; encoders $f_\phi, f_{\bar{\phi}}, q_\eta$ and decoders p_ψ, p_η; policies $\{\pi_{\theta_{e,i}}\}, \{\pi_{\theta_{x,i}}\}$; critics $\{Q_i^{\text{exp}}\}, \{Q_i^{\text{exe}}\}$ with targets and mixer g_ω; buffers $\mathcal{D}_{\text{exp}}, \mathcal{D}_{\text{exe}}$; hyperparameters $(\nu, \kappa), \tau_m, \alpha_{\text{adv}}, \lambda_{\text{CPC}}, \lambda_{\text{cov}}$

Ensure: trained $\phi, \psi, \eta, \{\theta_{e,i}\}, \{\theta_{x,i}\}, \{\zeta_i\}, \omega$

for each training iteration **do**
  For each i: sample $\mu_{1,i}, \mu_{2,i} \sim p(\mu)$, $\vartheta_i \sim \text{GEOM}(p_{\text{sw}})$, $\tilde{\vartheta}_i \leftarrow \min(\vartheta_i, H)$; sample one $\epsilon_a \sim p_\epsilon$
  Initialize each context posterior $\mu_{i,0} \leftarrow 0$, $\sigma_{i,0}^2 \leftarrow \kappa$
  for $t = 1, \ldots, H$ **do**
    for each agent i **do**
      Active regime $\mu_i(t) \leftarrow \mu_{1,i}$ if $t < \tilde{\vartheta}_i$ else $\mu_{2,i}$
      $(\mu_\rho, \Sigma_\rho) \leftarrow q_\eta(x_t^{\text{exe}})$; $b_i \leftarrow \{(\mu_{i,t-1}, \sigma_{i,t-1}^2 I), (\mu_\rho, \Sigma_\rho)\}$
      $a_i^{\text{exp}} \sim \pi_{\theta_{e,i}}(\cdot \mid o_i)$; $a_i^{\text{exe}} \sim \pi_{\theta_{x,i}}(\cdot \mid o_i, b_i)$
    end for
    Execute the joint action; observe r_i, o_i'
    Store $(o_i, a_i^{\text{exp}}, o_i')$ tagged $\mu_i(t)$ in $\mathcal{D}_{\text{exp}}$, and intrinsic-reward-free in $\mathcal{D}_{\text{exe}}$
    Store $(o_i, a_i^{\text{exe}}, o_i', r_i, \mu_i(t))$ in $\mathcal{D}_{\text{exe}}$
    for each agent i **do**
      $c_{i,t} \leftarrow f_\phi(y_{i,t} \mid x_{i,t})$; update $(\mu_{i,t}, \sigma_{i,t}^2)$ by Eq. (10.3)
      if $t = \tilde{\vartheta}_i$ **then** reset $\mu_{i,t} \leftarrow 0$, $\sigma_{i,t}^2 \leftarrow \kappa$ ▷ ground-truth reset
    end for
  end for
  Sample same-/cross-regime batches from $\mathcal{D}_{\text{exp}}$; compute $\mathcal{L}_{\text{enc}}$ (Eq. (10.4)) and the batched means $(\mu_{z,b}, \mu_{\rho,b})_{b=1}^B$
  $r_{i,t}^{\text{aux}} \leftarrow \frac{1}{2}\log(\sigma_{i,t-1}^2/\sigma_{i,t}^2)$ (Eq. (10.5)); $r_{i,t}^e \leftarrow r_{i,t}^{\text{env}} + \alpha_{\text{exp}} r_{i,t}^{\text{aux}}$
  Update each $\pi_{\theta_{e,i}}$, Q_i^{exp} by SAC on r_i^e; adapt α_{exp}
  Minibatch $\mathcal{B}^{\text{exe}} \subset \mathcal{D}_{\text{exe}}$; recompute each z_i with current ϕ
  MAAL step: $\breve{\boldsymbol{a}}^{-i} \leftarrow \Pi_{\epsilon_a}[\hat{\boldsymbol{a}}^{-i} - \alpha_{\text{adv}} \nabla_{\boldsymbol{a}^{-i}} Q_{\text{tot}}^{\bar{\zeta},\bar{\omega}}(\tau, (a_i, \hat{\boldsymbol{a}}^{-i}), s, \boldsymbol{b})]$ (Eq. (10.9))
  $(\mu_\rho, \Sigma_\rho) \leftarrow q_\eta(x^{\text{exe}})$; $\rho \sim q_\eta$; $\hat{\epsilon} \leftarrow p_\eta(\rho)$; descend $\mathcal{L}_{\text{rep}} = \mathcal{L}_{\text{enc}} + \mathcal{L}_\rho + \lambda_{\text{cov}}\mathcal{L}_{\text{cov}}$ (Eqs. (10.11), (10.16), (10.17)) in ϕ, ψ, η; $\bar{\phi} \leftarrow \tau_m \bar{\phi} + (1 - \tau_m)\phi$
  With the team belief $\boldsymbol{b}$ (Eq. (10.2)), evaluate y_{tot} and the soft-policy objective at the MAAL-perturbed action
  Descend $\mathcal{L}_Q$ in $\{\zeta_i\}, \omega$; update $\{\theta_{x,i}\}$ by the factored soft-policy objective (Eq. (10.14)); adapt α_{exe}
  Periodically Polyak-update $\bar{\zeta}, \bar{\theta}_x, \bar{\omega}$
end for
return $\phi, \psi, \eta, \{\theta_{e,i}\}, \{\theta_{x,i}\}, \{\zeta_i\}, \omega$

Algorithm 10.2 Phase 2 — supervised change-point detection

Require: frozen encoder f_ϕ; InDiD transformer h_ξ with a causal, local-window attention mask; labeled episodes $\{(\tau_i, \tilde{\vartheta}_i)_{i=1}^n\}$; false-alarm weight λ_{FA}
Ensure: trained detector ξ

for each training iteration **do**
  Sample or replay a labeled episode; for each agent i:
    frozen embeddings $c_{i,1:H} \leftarrow f_\phi(\tau_{i,1:H})$; $p_{i,t} \leftarrow h_\xi(c_{i,1:t})$ for $t = 1, \dots, H$, under the mask
  $\mathcal{L}_{\text{CPD}} \leftarrow \sum_i \left[\mathcal{L}_{\text{delay}}(p_{i,1:H}, \tilde{\vartheta}_i) + \lambda_{\text{FA}}\, \mathcal{L}_{\text{FA}}(p_{i,1:H}, \tilde{\vartheta}_i)\right]$
  Descend $\mathcal{L}_{\text{CPD}}$ in ξ
end for
return ξ

Algorithm 10.3 Meta-test — change-point-triggered re-exploration (per agent)

Require: frozen $\phi, \psi, \eta, \{\theta_{e,i}\}, \{\theta_{x,i}\}, \{\zeta_i\}, \omega, \xi$; threshold C; re-exploration budget K; uncertainty floor $\sigma^2_{\min}$

For each i: $\mu_i \leftarrow 0$, $\sigma_i^2 \leftarrow \kappa$, $\text{mode}_i \leftarrow$ Explore, $k_i \leftarrow 0$
for $t = 1, 2, \dots$ **do**
  for each agent i **do**
    if mode_i = Explore **then**
      $a_i \sim \pi_{\theta_{e,i}}(\cdot \mid o_i)$; $k_i \leftarrow k_i + 1$
    else
      $(\mu_\rho, \Sigma_\rho) \leftarrow q_\eta(x^{\text{exe}})$; $b_i \leftarrow \{(\mu_i, \sigma_i^2 I), (\mu_\rho, \Sigma_\rho)\}$; $a_i \sim \pi_{\theta_{x,i}}(\cdot \mid o_i, b_i)$
    end if
  end for
  Execute the joint action; observe r_i, o_i'
  for each agent i **do**
    $c_{i,t} \leftarrow f_\phi(y_{i,t} \mid x_{i,t})$; update $(\mu_{i,t}, \sigma_{i,t}^2)$ by Eq. (10.3); $p_{i,t} \leftarrow h_\xi(c_{i,1:t})$
    if $p_{i,t} \geq C$ **then**
      Reset $\mu_i \leftarrow 0$, $\sigma_i^2 \leftarrow \kappa$, $\text{mode}_i \leftarrow$ Explore, $k_i \leftarrow 0$ ▷ change point for agent i
    end if
    if mode_i = Explore **and** ($k_i \geq K$ **or** $\sigma_{i,t}^2 \leq \sigma^2_{\min}$) **then**
      $\text{mode}_i \leftarrow$ Execute
    end if
  end for
end for

11. Conclusion

This thesis proposed a MARL method that is robust and fast-adapting at once without a nominal-performance cost, and grounded it in a structured review that organizes the robust and adaptive literatures into one framework and turns that framework into the seven design requirements the method meets (Section 9.2). The four research questions of Section 1.4 are answered as follows.

RQ1: what is the problem? A deployed MARL policy faces changes from its training conditions through seven channels, listed in Table 4.2: the observation, action, and communication channels; the transition and reward functions; the identity and behaviour of the other agents; and the identity of the task. Behind all of them is one structural fact (Section 4.7). Each agent's effective transition kernel is an average over the other agents' policies, so a MARL policy's environment is non-stationary whenever any other agent is still learning, even with fixed physics and no adversary. The channels split into those that a designer treats as bounded and static and those that are unbounded over time but structured. Chapter 3 formalizes the two settings, constrained and unconstrained uncertainty, that this split produces.

RQ2: how is bounded uncertainty handled, and at what cost? There are four families (Chapter 5, Table 5.1). *Adversarial training* (M3DDPG) builds hard cases with an explicit adversary and pays for it with unstable training. *Regularization* (SA-MDP, ERNIE, MIR3, RADIAL-RL) replaces the adversary with a penalty that limits the policy's sensitivity to perturbation; it appears in four forms. *Distributionally robust* MARL (DrIGM) optimizes the worst case over a divergence ball of transition kernels while keeping decentralized execution. *Sim-to-real* methods (ADR) specialize adversarial training to the transition channel and the offline transfer step. All four optimize a worst case over a set that is fixed before training. Their common failure mode is a real disturbance that exceeds the set, and their common cost, which is the subject of RQ3, is nominal performance given up for a worst-case floor.

RQ3: can one policy be good in both disturbed and nominal conditions? The review shows that the nominal-performance cost of robustification is real and is recorded for every robust family. Worst-case optimization fixes the policy against a

case that, at deployment with no disturbance, is not occurring. No reviewed method is shown to avoid this cost while remaining robust. Chapter 10 argues that the cost comes from *hedging* rather than *adapting*: a policy conditioned on an online adaptation state can behave nominally when that state infers the nominal task and defend only when it does not. The proposed method realizes this with an M3DDPG worst-case (MAAL) teammate-action perturbation, scoped to the execution stream, whose inferred radius — as a full posterior, VariBAD-style — conditions both the execution policy and the centralized critic, so conservatism is scaled to the perturbation that is actually inferred and the exploration stream is left untouched. The method is specified in full; its evaluation is future work.

RQ4: can the policy adapt quickly when the whole environment changes? There are three online-re-estimation families (Chapter 6) and three meta-learning families (Chapter 7). Change-point detection and sliding-window estimation adapt after detecting a change or continuously discounting old data, with a reaction lag set by a detector's confidence or a window's length. Meta-learning amortizes the adaptation procedure over a task distribution and adapts from a few transitions with no retraining. Chapter 8 shows that these relate to the robust mechanisms as one construction: a set of models the policy carries, frozen for robustness or re-estimated for adaptability. Section 8.3.1 makes this exact for DrIGM and SWUCRL2-CW, whose frozen-limit sets are the same total-variation-ball construction, matching at radius $\rho = \eta/2$ and identical under full observability. Meta-learning is the fastest of the mechanisms, but it has been built almost entirely for a single agent. Bringing a contrastive context encoder, with an exchangeable-process posterior, into a team learner with FMASAC's credit assignment and a representation-learned change-point detector that triggers re-exploration is the core of the proposed design.

Contribution and next step. The central contribution is the proposed method of Chapter 10; the supporting contributions are the one framework for the two literatures, the mechanism-level comparison on common axes, the curated shortlist of exemplar methods, the four-part evaluation-gap analysis, and the seven design requirements R1–R7 (Section 9.2) that connect the review to the method. The method combines, on the FMASAC cooperative backbone, a per-agent contrastive context encoder with an exchangeable-process Gaussian posterior, separate information-gain exploration and task-reward execution policies, an M3DDPG-style worst-case (MAAL) teammate-action perturbation with a single team-shared variational encoder that infers the adversary's budget, scoped to the execution stream, and a second-phase, representation-learned change-point detector that resets an agent's context posterior and re-triggers its exploration when that agent's regime switches. The contribution is the combination, not any single part. The reviewed evidence is

used to argue that a method along these lines could obtain nominal performance, robustness, and fast adaptation together. Implementing it and running the Search-and-Rescue evaluation of Section 10.11 is the next stage of the work.

Central conclusion. Robust MARL and adaptive MARL answer the same question, how a policy survives an environment that does not stay fixed, with two assumptions about how it moves: bounded and prepared for, or unbounded and tracked. Read as one framework, they are closer in mechanism than their separate literatures suggest. The practical gap is not a missing algorithm for either property on its own, but a policy that holds both at once without giving up the nominal case. This thesis maps that gap and proposes a method to close it, leaving its implementation and evaluation as the next step.

A. Additional Meta-Reinforcement Learning Methods

Chapter 7 develops one exemplar per meta-RL family: MAML for policy-parameter-gradient methods (§7.2), RL^2 for black-box methods (§7.3), and PEARL, with VariBAD as a contrast, for task-inference methods (§7.4). These are the exemplars Chapter 8 compares directly against the robust MARL taxonomy. Each family's own literature is larger than one exemplar. This appendix lists the further methods that were read during the survey but not carried into the main text, so that the breadth of the literature is on record without letting any single mechanism dominate the comparison. Each row states the method's place in its family, its distinguishing mechanism, and why it is not an exemplar in Chapter 7. None of these methods is required to follow Chapters 7–11. Every characterization below is the one given by the method's own source paper.

Table A.1. – Additional meta-RL methods by family. "Direction" names the sub-problem the method addresses within its family; "Mechanism" is its distinguishing idea; the last column states why it is not carried as an exemplar in Chapter 7.

Method	Direction	Mechanism	Why not an exemplar
Policy-parameter-gradient family (exemplar: MAML, §7.2)			
ANIL [70]	Which parameters adapt	Adapt only the task-specific output layer; the feature extractor is shared and frozen in the inner loop	A partial-adaptation variant of MAML's mechanism, not a distinct one
BOIL [71]	Which parameters adapt	Adapt only the feature extractor (representation change), freezing the head	Same mechanism as MAML/ANIL with a different frozen subset
CAVIA [72]	Which parameters adapt	Adapt a small set of task-specific context parameters; policy weights fixed	Reduces MAML's inner-loop dimensionality; mechanism unchanged

Table A.1 – continued from previous page

Method	Direction	Mechanism	Why not an exemplar
Meta-SGD [73]	Learn the optimizer	Learn a per-parameter inner-loop update vector instead of a fixed step size	Learns the step, not the adaptation principle
Meta-Curvature [74]	Learn the optimizer	Learn a linear transform of the gradient that captures parameter correlations	Refinement of Meta-SGD; same bi-level structure
WarpGrad [75]	Learn the optimizer	Learn a parameter-space warp so optimization runs in a better geometry	Changes the geometry, not the adaptation mechanism
DiCE [76]	Meta-gradient estimation	MagicBox operator giving correct higher-order gradients through stochastic graphs	An estimator fix for MAML-style objectives, not a method
Loaded DiCE [77]	Meta-gradient estimation	Reformulate DiCE so value baselines / GAE reduce variance without breaking higher-order terms	Variance reduction on top of DiCE
No-DiCE [78]	Meta-gradient estimation	Correct the bias from data-distribution dependence across adaptation steps	Bias correction on top of DiCE
MAESN [79]	Exploration for adaptation	Condition the policy on a learned latent variable inducing temporally coherent exploration	Adds structured exploration to MAML; not a new adaptation route
E-MAML [56]	Exploration for adaptation	Assign meta-credit to pre-adaptation trajectories by their effect on the adapted policy	Modifies MAML's meta-objective for exploration credit only
Black-box family (exemplar: RL^2, §7.3)			
SNAIL [80]	Attention backbone	Temporal convolutions interleaved with causal attention over the interaction history	Replaces RL^2's recurrence with a different sequence model; same adaptation-via-input principle
TrMRL [81]	Attention backbone	Attention-only transformer over the history; no recurrence or convolution	Same principle as SNAIL with a pure-attention architecture

Table A.1 – continued from previous page

Method	Direction	Mechanism	Why not an exemplar
MRA [82]	External memory	Explicit write / read of working-memory states, retrieved by nearest-neighbour weighting	External-memory variant of history conditioning
Been There, Done That [83]	External memory	Differentiable neural dictionary with a reinstatement gate that restores prior cell state on task recurrence	Episodic-recall variant; specific to recurring tasks
Episodic Planning Network [84]	External memory	Memory used for planning in novel environments, beyond weighted retrieval	Task-solving mechanism narrower than the family exemplar
Control of Memory [85]	External memory	Structured, addressable external memory with explicit read / write, applied to control	Architecture study predating the meta-RL framing, not a standalone adaptation method
Differentiable Plasticity / Neuromodulation [86], [87]	Self-modifying weights	Hebbian trace added to fixed weights, gated by a learned neuromodulatory signal	Weight-space adaptation; peripheral to the robustness/adaptability argument
Self-Referential Weight Matrix [88]	Self-modifying weights	Network updates its own full weight matrix from its own activations	Extreme weight-space variant; no MARL use
MetODS [89]	Self-modifying weights	Outer-product synaptic self-modification inside an explicit RL formulation	Weight-space variant of SRWM
BIMRL [90]	Self-modifying weights	Brain-inspired stack combining Hebbian, neuromodulatory, and memory modules	Integrative architecture, not a single mechanism
Hebbian Plasticity in Random Networks [91]	Self-modifying weights	Evolve only Hebbian rules; random fixed initial weights, no backprop outer loop	Training-signal variant; evolutionary outer loop out of scope
Task-inference family (exemplars: PEARL, VariBAD, §7.4)			
ELUE [60]	Belief-based inference	Decouple encoder training from the actor / critic; a conditional autoencoder on raw transitions	Fixes a specific PEARL off-policy limitation; same belief-inference mechanism

Table A.1 – continued from previous page

Method	Direction	Mechanism	Why not an exemplar
DREAM [92]	Decoupled exploration	Explore-then-execute with a consistency argument and a sample-complexity result	Exploration decomposition on top of task inference
MetaCURE [93]	Decoupled exploration	Empowerment-driven intrinsic reward for adaptation under sparse task reward	Exploration objective specialised to sparse reward
CCM [63]	Decoupled exploration	Contrastive objective on how exploratory experience is encoded, plus information-gain collection	Encoder / exploration refinement of PEARL
ESCP [94]	Discriminative context	Context-sensitive policy trained to react to sudden within-episode change	Relaxes the fixed-task-per-episode assumption; discriminative rather than variational
PD-VF [95]	Direct regression	Policy-dynamics value function; adapt by regression after a few transitions, no gradient step	Regresses to a value function rather than inferring a task belief
FLAP [96]	Direct regression	Linear task representation enabling near-instant adaptation	Linear-representation special case of direct regression
BRUNO [65]	Bayesian task belief	Exchangeable stochastic process placed on learned per-step latents; its conjugate posterior-predictive is maintained by closed-form recursion rather than a learned RNN	Architecturally principled but published only for single-agent meta-RL; this thesis adopts it as the per-agent context encoder (Chapter 10)
Neural Predictive Belief [97]	Representation validation	Learn a belief state from raw observations by prediction alone; probe what it encodes	A methodology for validating belief representations, not an adaptation method

Two further points from this literature carry into the main argument. First, black-box methods are known to be hard to train: the outer-loop optimization must

discover both a representation and an implicit learning rule at once, and performance degrades sharply when the deployment task distribution differs from the meta-training one. Second, ESCP is the one method in this list that explicitly targets change *within* an episode rather than between episodes, which places it closest to the adaptive MARL setting of Chapter 6; it is noted here rather than developed because it remains single-agent.

B. Contrastive Log-ratio Upper Bound (CLUB)

MIR3 (§5.5) estimates the mutual information between an agent's observation history and its selected action with the Contrastive Log-ratio Upper Bound (CLUB) estimator [43]. CLUB is a differentiable upper bound on mutual information that can be optimized by stochastic gradient descent, which makes it usable inside a deep RL training loop.

Given two random variables X and Y, the CLUB estimator satisfies

$$I(X;Y) \leq \mathbb{E}_{p(x,y)}\left[\log q_\phi(y \mid x)\right] - \mathbb{E}_{p(x)p(y)}\left[\log q_\phi(y \mid x)\right], \tag{B.1}$$

where $q_\phi(y \mid x)$ is a neural network approximating the conditional $p(y \mid x)$. The first expectation is over *positive pairs* drawn from the joint distribution, observations and actions that occurred together; the second is over *negative pairs* formed from independent marginals. Given a minibatch $\{(x_i, y_i)\}_{i=1}^N$ the empirical estimator is

$$\hat{I}_{\text{CLUB}} = \frac{1}{N^2} \sum_{i=1}^{N} \sum_{j=1}^{N} \left[\log q_\phi(y_i \mid x_i) - \log q_\phi(y_j \mid x_i)\right]. \tag{B.2}$$

In MIR3 this estimates $I(h;a)$ between history h and action a, and enters the regularized objective

$$J(\pi) = \mathbb{E}[R] - \lambda\, I(h;a), \tag{B.3}$$

so the policy is pushed to keep only the information needed for decision making and to discard redundant dependence on teammate behaviour, without training against an explicit adversary. The conditional model q_ϕ is fitted by maximizing the conditional log-likelihood on positive pairs, in alternation with the policy update that minimizes the estimated bound.

C. Total Variation Distance

Total variation distance (TVD) quantifies the difference between two probability distributions, and is the divergence used to define DrIGM's ambiguity set (§5.6) and, with a fixed floor, SWUCRL2-CW's confidence region (§6.5). For a measurable space $(\Omega, \mathcal{F})$ and probability measures P, Q,

$$d_{\mathrm{TV}}(P, Q) = \sup_{A \in \mathcal{F}} |P(A) - Q(A)|, \tag{C.1}$$

the largest difference in probability the two assign to any event. For discrete distributions this is equivalent to

$$d_{\mathrm{TV}}(P, Q) = \frac{1}{2} \sum_{x} |P(x) - Q(x)| = \frac{1}{2} \|P - Q\|_1, \tag{C.2}$$

and for densities p, q, $d_{\mathrm{TV}}(P, Q) = \frac{1}{2} \int |p(x) - q(x)| \, \mathrm{d}x$. The identity $d_{\mathrm{TV}} = \frac{1}{2} \|\cdot\|_1$ is what Section 8.3.1 uses to match DrIGM's TV radius ρ against SWUCRL2-CW's ℓ_1 floor η: a TV ball of radius ρ is an ℓ_1 ball of radius 2ρ, so the two coincide at $\rho = \eta/2$.

TVD is related to the Kullback–Leibler divergence by Pinsker's inequality $d_{\mathrm{TV}}(P, Q) \leq \sqrt{\frac{1}{2} D_{\mathrm{KL}}(P \| Q)}$, and by $d_{\mathrm{TV}}(P, Q) \leq \sqrt{1 - e^{-D_{\mathrm{KL}}(P \| Q)}}$; both are used to move between divergence choices when deriving robustness guarantees.

D. Exponential-Family Distributions

BOCPD's within-run predictive model (§6.4) is taken from the exponential family so that the run parameters can be integrated out in closed form. A distribution is in the exponential family if

$$p(x \mid \omega) = h(x) \exp\big(\phi(x)^\top \omega - A(\omega)\big), \tag{D.1}$$

with base measure h, natural parameter ω, sufficient statistics $\phi(x)$, and log-partition function $A(\omega) = \log \int h(x) \exp(\phi(x)^\top \omega)\, \mathrm{d}x$. The Gaussian, Bernoulli, Poisson, Gamma, and Dirichlet distributions are all members.

Maximum likelihood. For $\mathcal{D} = \{x_1, \ldots, x_n\}$ the score equation is moment matching,

$$\nabla_\omega A(\omega) = \mathbb{E}_{p(x|\omega)}[\phi(x)] = \frac{1}{n} \sum_{i=1}^{n} \phi(x_i). \tag{D.2}$$

Conjugate prior and posterior. The conjugate prior $p_F(\omega \mid \alpha, \nu) \propto \exp(\omega^\top \alpha - \nu A(\omega))$ is closed under observation: after a dataset X of size n,

$$p(\omega \mid X, \alpha, \nu) = p_F\big(\omega \,\big|\, \alpha + \textstyle\sum_{i=1}^{n} \phi(x_i),\ \nu + n\big), \tag{D.3}$$

so the posterior stays in the same family and its parameters update by adding the summed sufficient statistics and the count. This closure is exactly what makes BOCPD's recursive run-length update constant-time per step: each candidate run carries a set of exponential-family hyperparameters that are updated by accumulation.

E. Use of Generative AI Tools

In accordance with the Declaration of Authorship, Table E.1 names every generative AI tool used in the preparation of this thesis and the purpose for which it was used. [FILL IN: verify this table matches your actual usage and your institution's required disclosure format before submission; adjust or extend as needed.]

Table E.1. – Generative AI tools used in the preparation of this thesis.

Tool	**Purpose**
Claude (Anthropic), Sonnet 5	Drafting, restructuring, and editing thesis text under the author's direction; converting source material into LaTeX; technical formalization and verification of mathematical claims against cited source papers; identifying and fixing errors in derivations, cross-references, and citations; literature search and citation verification for newly added sources; document formatting and template migration. All generated content was reviewed, and in many cases directed and revised, by the author, who is responsible for the information and statements in this thesis.

List of Tables

List of Figures

Bibliography

[1] K. Zhang, Z. Yang, and T. Başar, "Multi-agent reinforcement learning: A selective overview of theories and algorithms," in *Handbook on Reinforcement Learning and Control*, Springer, 2021, pp. 321–384.

[2] S. V. Albrecht, F. Christianos, and L. Schäfer, *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. MIT Press, 2024. [Online]. Available: `https://www.marl-book.com`.

[3] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," *arXiv preprint arXiv:1412.6572*, 2014.

[4] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," *arXiv preprint arXiv:1706.06083*, 2017.

[5] H. Zhang, H. Chen, C. Xiao, *et al.*, "Robust deep reinforcement learning against adversarial perturbations on state observations," *Advances in Neural Information Processing Systems*, vol. 33, pp. 21 024–21 037, 2020.

[6] P. Hernandez-Leal, M. Kaisers, T. Baarslag, and E. M. de Cote, "A survey of learning in multiagent environments: Dealing with non-stationarity," *arXiv preprint arXiv:1707.09183*, 2017.

[7] Y. Wang and S. Zou, "Online robust reinforcement learning with model uncertainty," in *Advances in Neural Information Processing Systems*, vol. 34, 2021.

[8] Z. U. Farhat, D. Ghosh, G. K. Atia, and Y. Wang, "Sample-efficient distributionally robust multi-agent reinforcement learning via online interaction," *arXiv preprint arXiv:2508.02948*, 2025.

[9] P. Auer, T. Jaksch, and R. Ortner, "Near-optimal regret bounds for reinforcement learning," *Advances in neural information processing systems*, vol. 21, 2008.

[10] C. Tessler, Y. Efroni, and S. Mannor, "Action robust reinforcement learning and applications in continuous control," in *International Conference on Machine Learning*, PMLR, 2019, pp. 6215–6224.

[11] F. Wu, L. Li, Z. Huang, Y. Vorobeychik, D. Zhao, and B. Li, "Crop: Certifying robust policies for reinforcement learning through functional smoothing," in *International Conference on Learning Representations*, 2022.

[12] G. N. Iyengar, "Robust dynamic programming," *Mathematics of Operations Research*, vol. 30, no. 2, pp. 257–280, 2005.

[13] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.

[14] A. Garivier and E. Moulines, "On upper-confidence bound policies for non-stationary bandit problems," in *International Conference on Algorithmic Learning Theory*, Springer, 2011, pp. 174–188.

[15] L. Li, R. Yang, and D. Luo, "Focal: Efficient fully-offline meta-reinforcement learning via distance metric learning and behavior regularization," in *International Conference on Learning Representations*, 2021.

[16] J. Moos, K. Hansel, H. Abdulsamad, S. Stark, D. Clever, and J. Peters, "Robust reinforcement learning: A review of foundations and recent advances," *Machine Learning and Knowledge Extraction*, vol. 4, no. 1, pp. 276–315, 2022.

[17] L. Pinto, J. Davidson, R. Sukthankar, and A. Gupta, "Robust adversarial reinforcement learning," in *International conference on machine learning*, PMLR, 2017, pp. 2817–2826.

[18] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *International conference on machine learning*, PMLR, 2017, pp. 1126–1135.

[19] Y. Duan, J. Schulman, X. Chen, P. L. Bartlett, I. Sutskever, and P. Abbeel, "Rl^2: Fast reinforcement learning via slow reinforcement learning," *arXiv preprint arXiv:1611.02779*, 2016.

[20] K. Rakelly, A. Zhou, C. Finn, S. Levine, and D. Quillen, "Efficient off-policy meta-reinforcement learning via probabilistic context variables," in *Proceedings of the 36th International Conference on Machine Learning*, K. Chaudhuri and R. Salakhutdinov, Eds., ser. Proceedings of Machine Learning Research, vol. 97, PMLR, Sep. 2019, pp. 5331–5340.

[21] M. L. Littman, "Markov games as a framework for multi-agent reinforcement learning," in *Machine learning proceedings 1994*, Elsevier, 1994, pp. 157–163.

[22] J. Hu and M. P. Wellman, "Nash q-learning for general-sum stochastic games," *Journal of machine learning research*, vol. 4, no. Nov, pp. 1039–1069, 2003.

[23] R. J. Aumann, "Subjectivity and correlation in randomized strategies," *Journal of Mathematical Economics*, vol. 1, no. 1, pp. 67–96, 1974.

[24] A. Greenwald, K. Hall, R. Serrano, *et al.*, "Correlated q-learning," in *ICML*, vol. 3, 2003, pp. 242–249.

[25] S. Hart and A. Mas-Colell, "A general class of adaptive strategies," *Journal of Economic Theory*, vol. 98, no. 1, pp. 26–54, 2001.

[26] V. Mnih, K. Kavukcuoglu, D. Silver, *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[27] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, no. 3-4, pp. 229–256, 1992.

[28] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016. [Online]. Available: `https://arxiv.org/abs/1506.02438`.

[29] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in *International conference on machine learning*, PMLR, 2015, pp. 1889–1897.

[30] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[31] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, "Deterministic policy gradient algorithms," in *Proceedings of the 31st International Conference on Machine Learning*, E. P. Xing and T. Jebara, Eds., ser. Proceedings of Machine Learning Research, vol. 32, Bejing, China: PMLR, 22–24 Jun 2014, pp. 387–395. [Online]. Available: `https://proceedings.mlr.press/v32/silver14.html`.

[32] T. P. Lillicrap, J. J. Hunt, A. Pritzel, *et al.*, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015.

[33] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *International conference on machine learning*, PMLR, 2018, pp. 1587–1596.

[34] T. Haarnoja, A. Zhou, K. Hartikainen, *et al.*, "Soft actor-critic algorithms and applications," *arXiv preprint arXiv:1812.05905*, 2018.

[35] P. Sunehag, G. Lever, A. Gruslys, *et al.*, "Value-decomposition networks for cooperative multi-agent learning," *arXiv preprint arXiv:1706.05296*, 2017.

[36] T. Rashid, M. Samvelyan, C. S. De Witt, G. Farquhar, J. Foerster, and S. Whiteson, "Monotonic value function factorisation for deep multi-agent reinforcement learning," *Journal of Machine Learning Research*, vol. 21, no. 178, pp. 1–51, 2020.

[37] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[38] C. Yu, A. Velu, E. Vinitsky, *et al.*, "The surprising effectiveness of ppo in cooperative multi-agent games," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[39] L. Yue, R. Yang, J. Zuo, M. Yan, X. Zhao, and M. Lv, "Factored multi-agent soft actor-critic for cooperative multi-target tracking of uav swarms," *Drones*, vol. 7, no. 3, p. 150, 2023.

[40] J. Lin, K. Dzeparoska, S. Q. Zhang, A. Leon-Garcia, and N. Papernot, "On the robustness of cooperative multi-agent reinforcement learning," in *2020 IEEE Security and Privacy Workshops (SPW)*, 2020, pp. 62–68. doi: 10.1109/SPW50608.2020.00027.

[41] S. Li, Y. Wu, X. Cui, H. Dong, F. Fang, and S. Russell, "Robust multi-agent reinforcement learning via minimax deep deterministic policy gradient," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 33, 2019, pp. 4213–4220.

[42] A. Bukharin, Y. Li, Y. Yu, *et al.*, "Robust multi-agent reinforcement learning via adversarial regularization: Theoretical foundation and stable algorithms," *Advances in neural information processing systems*, vol. 36, pp. 68 121–68 133, 2023.

[43] P. Cheng, W. Hao, S. Dai, J. Liu, Z. Gan, and L. Carin, "CLUB: A contrastive log-ratio upper bound of mutual information," in *Proceedings of the 37th International Conference on Machine Learning*, H. D. III and A. Singh, Eds., ser. Proceedings of Machine Learning Research, vol. 119, PMLR, 13–18 Jul 2020, pp. 1779–1788. [Online]. Available: https://proceedings.mlr.press/v119/cheng20b.html.

[44] S. Li, R. Xu, J. Xiu, *et al.*, "Robust multi-agent reinforcement learning by mutual information regularization," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 10, pp. 18 118–18 132, 2025. doi: 10.1109/TNNLS.2025.3577259.

[45] T. Oikarinen, W. Zhang, A. Megretski, L. Daniel, and T.-W. Weng, "Robust deep reinforcement learning through adversarial loss," *Advances in Neural Information Processing Systems*, vol. 34, pp. 26 156–26 167, 2021.

[46] C. Qu, C. Yeh, K. Panaganti, E. Mazumdar, and A. Wierman, "Distributionally robust cooperative multi-agent reinforcement learning with value factorization," in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: https://openreview.net/forum?id=2T3LOpqIOO.

[47] S. Chen, G. Liu, Z. Zhou, K. Zhang, and J. Wang, "Robust multi-agent reinforcement learning method based on adversarial domain randomization for real-world dual-uav cooperation," *IEEE Transactions on Intelligent Vehicles*, vol. 9, no. 1, pp. 1615–1627, 2024. doi: 10.1109/TIV.2023.3307134.

[48] O. Besbes, Y. Gur, and A. Zeevi, "Stochastic multi-armed-bandit problem with non-stationary rewards," *Advances in neural information processing systems*, vol. 27, 2014.

[49] D. Agudelo-España, S. Gomez-Gonzalez, S. Bauer, B. Schölkopf, and J. Peters, "Bayesian online prediction of change points," in *Proceedings of the 36th Conference on Uncertainty in Artificial Intelligence (UAI)*, J. Peters and D. Sontag, Eds., ser. Proceedings of Machine Learning Research, vol. 124, PMLR, Mar. 2020, pp. 320–329. [Online]. Available: https://proceedings.mlr.press/v124/agudelo-espana20a.html.

[50] R. Alami, M. Mahfoud, and E. Moulines, "Restarted bayesian online change-point detection for non-stationary markov decision processes," in *Conference on Lifelong Learning Agents*, PMLR, 2023, pp. 715–744.

[51] B. C. Da Silva, E. W. Basso, A. L. Bazzan, and P. M. Engel, "Dealing with non-stationary environments using context detection," in *Proceedings of the 23rd international conference on Machine learning*, 2006, pp. 217–224.

[52] P. Gajane, R. Ortner, and P. Auer, "A sliding-window algorithm for markov decision processes with arbitrarily changing rewards and transitions," *arXiv preprint arXiv:1805.10066*, 2018.

[53] W. C. Cheung, D. Simchi-Levi, and R. Zhu, "Reinforcement learning for non-stationary markov decision processes: The blessing of (more) optimism," in *International conference on machine learning*, PMLR, 2020, pp. 1843–1854.

[54] M. Al-Shedivat, T. Bansal, Y. Burda, I. Sutskever, I. Mordatch, and P. Abbeel, "Continuous adaptation via meta-learning in nonstationary and competitive environments," *arXiv preprint arXiv:1710.03641*, 2017.

[55] J. Rothfuss, D. Lee, I. Clavera, T. Asfour, and P. Abbeel, "Promp: Proximal meta-policy search," *arXiv preprint arXiv:1810.06784*, 2018.

[56] B. C. Stadie, G. Yang, R. Houthooft, *et al.*, "Some considerations on learning to explore via meta-reinforcement learning," *arXiv preprint arXiv:1803.01118*, 2018.

[57] J. X. Wang, Z. Kurth-Nelson, D. Tirumala, *et al.*, "Learning to reinforcement learn," *arXiv preprint arXiv:1611.05763*, 2016.

[58] J. Subramanian, A. Sinha, R. Seraj, and A. Mahajan, "Approximate information state for approximate planning and reinforcement learning in partially observed systems," *Journal of Machine Learning Research*, vol. 23, no. 12, pp. 1–83, 2022.

[59] L. Zintgraf, K. Shiarlis, M. Igl, *et al.*, "Varibad: A very good method for bayes-adaptive deep rl via meta-learning," *arXiv preprint arXiv:1910.08348*, 2019.

[60] T. Imagawa, T. Hiraoka, and Y. Tsuruoka, "Off-policy meta-reinforcement learning with belief-based task inference," *IEEE Access*, vol. 10, pp. 49 494–49 507, 2022. doi: 10.1109/ACCESS.2022.3170582.

[61] Z. Zhao, Y. Fu, J. Chai, Y. Zhu, and D. Zhao, "Meta learning task representation in multiagent reinforcement learning: From global inference to local inference," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 8, pp. 14 908–14 921, 2025. doi: 10.1109/TNNLS.2025.3540758.

[62] E. Rivas, S. Saika, A. Bakht, A. Piplai, N. D. Bastian, and A. Shah, "Adapting under fire: Multi-agent reinforcement learning for adversarial drift in network security," in *Proceedings of the 22nd International Conference on Security and Cryptography (SECRYPT)*, 2025, pp. 547–554.

[63] H. Fu, H. Tang, J. Hao, *et al.*, "Towards effective context for meta-reinforcement learning: An approach based on contrastive learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, 2021, pp. 7457–7465.

[64] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 9726–9735. doi: 10.1109/CVPR42600.2020.00975.

[65] I. Korshunova, J. Degrave, J. Dambre, A. Gretton, and F. Huszar, "Exchangeable models in meta reinforcement learning," in *4th Lifelong Machine Learning Workshop at ICML 2020*, 2020. [Online]. Available: https://openreview.net/forum?id=TZFlzejFPT.

[66] E. Romanenkova, A. Stepikin, M. Morozov, and A. Zaytsev, "InDiD: Instant disorder detection via representation learning," *arXiv preprint arXiv:2106.02602*, 2021.

[67] M. Bettini, R. Kortvelesy, J. Blumenkamp, and A. Prorok, "VMAS: A vectorized multi-agent simulator for collective robot learning," in *International Symposium on Distributed Autonomous Robotic Systems (DARS)*, ser. Springer Proceedings in Advanced Robotics, vol. 28, Springer, 2022, pp. 42–56.

[68] M. Bettini, A. Prorok, and V. Moens, "BenchMARL: Benchmarking multi-agent reinforcement learning," *Journal of Machine Learning Research*, vol. 25, no. 217, pp. 1–10, 2024.

[69] A. Bou, M. Bettini, S. Dittert, *et al.*, "TorchRL: A data-driven decision-making library for PyTorch," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[70] A. Raghu, M. Raghu, S. Bengio, and O. Vinyals, "Rapid learning or feature reuse? towards understanding the effectiveness of maml," *arXiv preprint arXiv:1909.09157*, 2019.

[71] J. Oh, H. Yoo, C. Kim, and S.-Y. Yun, "Boil: Towards representation change for few-shot learning," *arXiv preprint arXiv:2008.08882*, 2020.

[72] S. Whiteson, "Fast context adaptation via meta- learning," 2019.

[73] Z. Li, F. Zhou, F. Chen, and H. Li, "Meta-sgd: Learning to learn quickly for few-shot learning," *arXiv preprint arXiv:1707.09835*, 2017.

[74] E. Park and J. B. Oliva, "Meta-curvature," *Advances in neural information processing systems*, vol. 32, 2019.

[75] S. Flennerhag, A. A. Rusu, R. Pascanu, F. Visin, H. Yin, and R. Hadsell, "Meta-learning with warped gradient descent," *arXiv preprint arXiv:1909.00025*, 2019.

[76] J. Foerster, G. Farquhar, M. Al-Shedivat, T. Rocktäschel, E. Xing, and S. Whiteson, "Dice: The infinitely differentiable monte carlo estimator," in *International Conference on Machine Learning*, PMLR, 2018, pp. 1529–1538.

[77] G. Farquhar, S. Whiteson, and J. Foerster, "Loaded dice: Trading off bias and variance in any-order score function gradient estimators for reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[78] R. Vuorio, J. A. Beck, G. Farquhar, J. N. Foerster, and S. Whiteson, "No DICE: An investigation of the bias-variance tradeoff in meta-gradients," in *Deep RL Workshop NeurIPS 2021*, 2021. [Online]. Available: `https://openreview.net/forum?id=WB2dwqWZ6kU`.

[79] A. Gupta, R. Mendonca, Y. Liu, P. Abbeel, and S. Levine, "Meta-reinforcement learning of structured exploration strategies," *Advances in neural information processing systems*, vol. 31, 2018.

[80] N. Mishra, M. Rohaninejad, X. Chen, and P. Abbeel, "A simple neural attentive meta-learner," *arXiv preprint arXiv:1707.03141*, 2017.

[81] L. C. Melo, "Transformers are meta-reinforcement learners," in *international conference on machine learning*, PMLR, 2022, pp. 15 340–15 359.

[82] M. Fortunato, M. Tan, R. Faulkner, *et al.*, "Generalization of reinforcement learners with working and episodic memory," *Advances in neural information processing systems*, vol. 32, 2019.

[83] S. Ritter, J. Wang, Z. Kurth-Nelson, *et al.*, "Been there, done that: Meta-learning with episodic recall," in *International conference on machine learning*, PMLR, 2018, pp. 4354–4363.

[84] S. Ritter, R. Faulkner, L. Sartran, A. Santoro, M. Botvinick, and D. Raposo, "Rapid task-solving in novel environments," *arXiv preprint arXiv:2006.03662*, 2020.

[85] J. Oh, V. Chockalingam, H. Lee, *et al.*, "Control of memory, active perception, and action in minecraft," in *International conference on machine learning*, PMLR, 2016, pp. 2790–2799.

[86] T. Miconi, J. Clune, and K. O. Stanley, "Differentiable plasticity: Training plastic neural networks with backpropagation," *arXiv preprint arXiv:1804.02464*, 2018.

[87] T. Miconi, A. Rawal, J. Clune, and K. O. Stanley, "Backpropamine: Training self-modifying neural networks with differentiable neuromodulated plasticity," *arXiv preprint arXiv:2002.10585*, 2020.

[88] K. Irie, I. Schlag, R. Csordás, and J. Schmidhuber, "A modern self-referential weight matrix that learns to modify itself," in *International conference on machine learning*, PMLR, 2022, pp. 9660–9677.

[89] M. Chalvidal, T. Serre, and R. VanRullen, "Meta-reinforcement learning with self-modifying networks," *Advances in Neural Information Processing Systems*, vol. 35, pp. 7838–7851, 2022.

[90] S. R. R. Rohani, S. Hedayatian, and M. S. Baghshah, "Bimrl: Brain inspired meta reinforcement learning," in *2022 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, IEEE, 2022, pp. 9048–9053.

[91] E. Najarro and S. Risi, "Meta-learning through hebbian plasticity in random networks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 20 719–20 731, 2020.

[92] E. Z. Liu, A. Raghunathan, P. Liang, and C. Finn, "Explore then execute: Adapting without rewards via factorized meta-reinforcement learning," *arXiv preprint arXiv:2008.02790*, 2020.

[93] J. Zhang, J. Wang, H. Hu, *et al.*, "Metacure: Meta reinforcement learning with empowerment-driven exploration," in *Proceedings of the 38th International Conference on Machine Learning*, M. Meila and T. Zhang, Eds., ser. Proceedings of Machine Learning Research, vol. 139, PMLR, 18–24 Jul 2021, pp. 12 600–12 610. [Online]. Available: `https://proceedings.mlr.press/v139/zhang21w.html`.

[94] F.-M. Luo, S. Jiang, Y. Yu, Z. Zhang, and Y.-F. Zhang, "Adapt to environment sudden changes by learning a context sensitive policy," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 36, 2022, pp. 7637–7646.

[95] R. Raileanu, M. Goldstein, A. Szlam, and R. Fergus, "Fast adaptation to new environments via policy-dynamics value functions," in *Proceedings of the 37th International Conference on Machine Learning*, 2020, pp. 7920–7931.

[96] M. Peng, B. Zhu, and J. Jiao, "Linear representation meta-reinforcement learning for instant adaptation," *arXiv preprint arXiv:2101.04750*, 2021.

[97] Z. D. Guo, M. G. Azar, B. Piot, B. A. Pires, and R. Munos, "Neural predictive belief representations," *arXiv preprint arXiv:1811.06407*, 2018.

Declaration Of Authorship

I, Abdullah Alalwani, hereby affirm, that I have prepared the present work without the unauthorized help of third parties and without the use of any aids other than those specified. The data and concepts taken directly or indirectly from sources are marked with a reference to the source. This thesis has not been submitted to an examination board in the same or a similar form, or published in any other way, either at home or abroad. I have documented the use of all generative AI tools authorized for the production of this thesis in the appendix of this thesis in tabular form by naming the tool and the purpose for which it was used. All verbatim or paraphrased content from generative AI tools has been marked accordingly. I am aware that any attempt at undocumented use of generative AI tools will be considered an attempt to cheat under the examination regulations. I confirm that I have not adopted content generated using AI tools without reflection and that, as the author, I am responsible for the information and statements in this thesis.

Ilmenau, [FILL IN]

Abdullah Alalwani